



VESTORA

ملف مذاكرة · مادة تحضير شخصية للمناقشة

## Team Leader

المعمارية، الدفع والتفاوض، الأدمن، والفرونت — أضخم نصيب تقني في المشروع

هذا الملف للمذاكرة فقط. ليس تقريرًا ولا وثيقة تُسلّم — الغرض منه أن تفهم جزءك من المشروع فهمًا كاملاً وتقدر تشرحه وتدافع عنه.

# المحتويات

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>1. Your Responsibility, Exactly</b>                              | <b>5</b>  |
| 1.1. The One-Line Answer                                            | 5         |
| 1.2. What You Own, in Detail                                        | 5         |
| 1.3. What You Deliberately Gave Away                                | 6         |
| <b>2. Opening the Defense — Your First Two Minutes</b>              | <b>6</b>  |
| 2.1. The Opening                                                    | 7         |
| 2.2. If They Ask for One Thing to Show                              | 7         |
| 2.3. The Three Numbers to Have Ready                                | 7         |
| <b>3. The Product in Two Pages</b>                                  | <b>7</b>  |
| 3.1. What Vestora Is                                                | 7         |
| 3.2. The Money Vocabulary — The Most Important Thing in the Product | 8         |
| <b>4. System Architecture</b>                                       | <b>8</b>  |
| 4.1. The Shape                                                      | 8         |
| 4.2. Three Architectural Decisions You Made                         | 8         |
| <b>5. The Team Division</b>                                         | <b>9</b>  |
| <b>6. The Data Model — Your Architectural Decisions</b>             | <b>10</b> |
| 6.1. The Core Shape                                                 | 11        |
| 6.2. Three Users, One Table                                         | 11        |
| 6.3. The Money Trail                                                | 12        |
| <b>7. Payments — The Most Careful Code in the Project</b>           | <b>13</b> |
| 7.1. The Two Non-Negotiable Rules                                   | 13        |
| 7.2. Event Before Effect — Rule 5                                   | 13        |
| 7.3. Idempotency by Index, Not by Code                              | 13        |
| 7.4. The Reconciliation Alarm                                       | 14        |
| 7.5. The Fee Is Frozen at Checkout                                  | 14        |
| <b>8. Tranche Funding — Paying a Commitment in Instalments</b>      | <b>14</b> |
| 8.1. The Problem                                                    | 14        |
| 8.2. The Fix — Sum, Not Existence                                   | 15        |
| 8.3. Four Functions That Make It Work                               | 15        |
| 8.4. The Double-Counting Guard                                      | 15        |
| <b>9. The Payment State Machine</b>                                 | <b>16</b> |
| 9.1. Six States                                                     | 16        |
| 9.2. Four Cancel Reasons                                            | 16        |
| 9.3. The Sweeper — Closing What Nobody Finished                     | 17        |
| <b>10. The Deal Room and Term Sheets</b>                            | <b>17</b> |
| 10.1. What Lives in One Screen                                      | 17        |
| 10.2. The Term Sheet Service                                        | 17        |
| <b>11. The Dashboards — Five Controllers, One Question Each</b>     | <b>18</b> |
| 11.1. One Request, Everything                                       | 18        |
| 11.2. The Numbers on These Screens                                  | 20        |
| 11.3. Venture Insights — One Project at a Time                      | 22        |
| <b>12. Three Roles, Three Control Rooms</b>                         | <b>23</b> |
| 12.1. What the Sidebars Tell You                                    | 25        |
| <b>13. Messaging, Notifications, and Real-Time</b>                  | <b>26</b> |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| 13.1. The Pieces .....                                        | 26        |
| 13.2. Presence Is Scoped, Not Broadcast .....                 | 27        |
| 13.3. Why Notifications Go Through a Queue .....              | 28        |
| <b>14. The Investment Flow — End to End .....</b>             | <b>28</b> |
| <b>15. One Deal Room, Read End to End .....</b>               | <b>31</b> |
| 15.1. What This One Screen Proves .....                       | 31        |
| 15.2. The Timeline Is the Audit Trail .....                   | 32        |
| <b>16. The Admin Surface — Six Controllers .....</b>          | <b>32</b> |
| 16.1. Why Six Files and Not One .....                         | 33        |
| 16.2. The Primary Admin .....                                 | 33        |
| 16.3. Insights Return Parts, Not Percentages .....            | 34        |
| <b>17. Backend Infrastructure .....</b>                       | <b>34</b> |
| 17.1. The Stripe Guard — Rule 10 .....                        | 34        |
| 17.2. Two Providers, One Interface .....                      | 35        |
| 17.3. Rate Limiting .....                                     | 35        |
| 17.4. Cross-Cutting Services .....                            | 35        |
| <b>18. The Frontend Foundation You Built .....</b>            | <b>36</b> |
| 18.1. The Transport Layer — One Door .....                    | 36        |
| 18.2. The Type Contract .....                                 | 37        |
| 18.3. The Design System in Code .....                         | 37        |
| 18.4. Motion and 3D — Nine Components .....                   | 37        |
| <b>19. The Landing Page and Onboarding .....</b>              | <b>38</b> |
| 19.1. The Landing — Twelve Components .....                   | 38        |
| 19.2. Onboarding — 590 Lines .....                            | 39        |
| <b>20. The Admin Interface — Thirteen Pages .....</b>         | <b>40</b> |
| <b>21. Bilingual and RTL — The Whole Product, Twice .....</b> | <b>43</b> |
| 21.1. The Contract .....                                      | 43        |
| 21.2. Direction Is a System Concern, Not a Page Concern ..... | 43        |
| 21.3. The Honest-Copy Rule Lives Here Too .....               | 44        |
| <b>22. Authorization — The Model You Designed .....</b>       | <b>44</b> |
| 22.1. Four Levels .....                                       | 44        |
| 22.2. Viewer-Relative Visibility .....                        | 45        |
| 22.3. The Ownership Chain .....                               | 45        |
| <b>23. The Admin Screens — A Gallery .....</b>                | <b>45</b> |
| 23.1. Revenue and Reconciliation .....                        | 46        |
| 23.2. Security and Activity .....                             | 47        |
| <b>24. Terms You Must Know .....</b>                          | <b>48</b> |
| <b>25. The Money Model, Proven on Screen .....</b>            | <b>49</b> |
| 25.1. Before — Committed but Not Funded .....                 | 49        |
| 25.2. After — The Numbers Line Up .....                       | 50        |
| 25.3. The Platform Side .....                                 | 51        |
| <b>26. Endpoints Reference — Your Areas .....</b>             | <b>51</b> |
| 26.1. Payments — <code>api/payments</code> .....              | 51        |
| 26.2. Deal Room — <code>api/deals</code> .....                | 52        |
| 26.3. Admin — <code>api/admin</code> .....                    | 53        |
| <b>27. Request and Response Examples .....</b>                | <b>53</b> |
| 27.1. Creating a Funding Request .....                        | 53        |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| 27.2. Opening a Checkout .....                               | 54        |
| 27.3. Tranche Refusal — A Real Guard .....                   | 54        |
| <b>28. The Counter-Offer — Negotiating the Number .....</b>  | <b>54</b> |
| 28.1. Why It Exists .....                                    | 55        |
| 28.2. The Flow .....                                         | 55        |
| <b>29. Running the Whole Thing .....</b>                     | <b>55</b> |
| 29.1. The Two Commands .....                                 | 55        |
| 29.2. The Five-Minute Demo .....                             | 56        |
| <b>30. The Documentation Suite .....</b>                     | <b>57</b> |
| <b>31. The Schema History — Twenty-Nine Migrations .....</b> | <b>57</b> |
| 31.1. The Story in Phases .....                              | 57        |
| <b>32. More Screens You Built .....</b>                      | <b>59</b> |
| <b>33. Governance — The Rules You Enforced .....</b>         | <b>60</b> |
| <b>34. Known Limitations .....</b>                           | <b>61</b> |
| <b>35. Doctor Questions — The Leader Edition .....</b>       | <b>62</b> |
| <b>36. Answering For Your Team .....</b>                     | <b>65</b> |
| 36.1. The Backend Member .....                               | 65        |
| 36.2. The Frontend Member .....                              | 65        |
| 36.3. The Database Member .....                              | 65        |
| 36.4. The UI/UX Member .....                                 | 65        |
| 36.5. The Testing Member .....                               | 66        |
| <b>37. Suggested Study Plan .....</b>                        | <b>66</b> |
| <b>38. Final Code Map — Where to Find Everything .....</b>   | <b>67</b> |
| <b>39. Cheat Sheet — Review Before You Walk In .....</b>     | <b>68</b> |



# 1. Your Responsibility, Exactly

## 1.1. The One-Line Answer

إنت قائد الفريق — ومعنى ده حاجتين مع بعض: صممت المعمارية ووزعت الشغل على خمس أعضاء، وكمان بنيت أضخم نصيب تقني في المشروع بنفسك.

### اعرف ده كويس

الجملة اللي تقولها لو اتسألت «إنت عملت إيه؟»:

«أنا صممت معمارية النظام ووزعت الفريق، وبنيت الأجزاء اللي محدش غيري ينفج يمسكها: نظام الدفع كامل بالتمويل على دفعات، غرفة الصفقة والتفاوض على الشروط، لوحة الأدمن بستة كنترولرز، البنية التحتية للباكند، وطبقة الفرونت المشتركة كلها — اللاندينج، نظام التصميم في الكود، طبقة API، والترجمة.»

## 1.2. What You Own, in Detail

| المجال               | الأسطر | الملفات                                                                                                                                                                                                                   |
|----------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| غرفة الصفقة والتفاوض | 1553   | <code>DealRoomController</code> (1287) · <code>TermSheetService</code> (266)                                                                                                                                              |
| الدفع والتمويل       | 2188   | <code>PaymentService</code> (1259) · <code>PaymentsController</code> (486) · <code>FundingMath</code> (443) — والمزودين والسويكر                                                                                          |
| الأدمن كامل          | 2093   | <code>AdminController</code> (720) · <code>AdminRevenue</code> (434) · <code>AdminUserOverview</code> (141) · <code>AdminSearch</code> (187) · <code>AdminInsights</code> (238) · <code>AdminModeration</code> (373)      |
| اللوحات              | 1839   | <code>InvestorController</code> (690) · <code>FounderDashboard</code> (415) · <code>InvestorDashboard</code> (300) · <code>Capital</code> (257) · <code>VentureInsights</code> (177)                                      |
| الرسائل والإشعارات   | 676    | <code>MessagesController</code> (358) · <code>NotificationController</code> (318) · <code>ChatHub</code> · <code>PresenceTracker</code> · والطاير والworker                                                               |
| البنية التحتية       | 750    | <code>Program.cs</code> (400) · <code>ServiceResult</code> · <code>UtcDateTimeConverter</code> · <code>StageLog</code> · <code>TrustSignals</code> · <code>FileUploadSecurityService</code> · <code>PipelineStages</code> |

**جدول 1** نصيبك في الباكند: أكثر من 9000 سطر — تلتين الباكند كله.

| المجال       | الحجم              | التفصيل                                                                                                                 |
|--------------|--------------------|-------------------------------------------------------------------------------------------------------------------------|
| الترجمة      | 4896 سطر           | <code>lib/i18n/dictionaries.ts</code> — كل نص ظاهر في المنتج، بلغتين                                                    |
| عقد الأنواع  | 1736 سطر           | <code>lib/types/api.ts</code> — كل شكل بيانات بين الفرونت والباكند                                                      |
| نظام التصميم | 775 سطر · 129 توكن | <code>app/globals.css</code> — الألوان والخطوط والحركة والزجاج                                                          |
| صفحات الأدمن | 13 صفحة            | كل <code>app/(app)/admin/*</code> — من النظرة العامة للتسوية                                                            |
| اللاندينج    | 12 مكّون           | الهيرو، القصة، الإحصائيات، الشهادات، «إزاي بيشتغل»                                                                      |
| الحركة وال3D | 9 مكّونات          | <code>depth-scene</code> · <code>tilt</code> · <code>magnetic</code> · <code>cursor</code> · <code>reveal</code> وغيرهم |
| طبقة الAPI   | 17 ملف             | <code>lib/api/*</code> — الناقل الوحيد المسموح بيه في المشروع                                                           |

| المجال      | الحجم   | التفصيل                                    |
|-------------|---------|--------------------------------------------|
| الأونبوردنج | 590 سطر | <code>app/(app)/onboarding/page.tsx</code> |

**جدول 2** نصيبك في الفرونت: البنية المشتركة التي كل الأعضاء يبنوا فوقها.

#### اعرف ده كويس

الأرقام التي تحفظها: أكثر من 9000 سطر باكند (تلتين الباكند)، 13 صفحة أدمن في الفرونت، 129 توكن تصميم، 4896 سطر ترجمة. ودي مش أرقام للتفاخر — دي إجابة سؤال «إنت عملت إيه بالطبط؟» برقم يتقاس بدل كلام عام.

### 1.3. What You Deliberately Gave Away

#### مش جزءك

التوزيع ده قرار قيادي، مش حاجة حصلت بالصدفة:

- المصادقة والحسابات — عضو الباكند. أعماق نظام أمني في المشروع، وسبته له عشان يبقى عنده جزء يقدر يدافع عنه لوحده بعمق.
- المشاريع والاكتشاف — عضو الباكند. أكبر كنترولر (1180 سطر).
- صفحة التصفح ونماذج الحساب — عضو الفرونت.
- مخطط قاعدة البيانات وميكانيكاها — عضو الداتابيز.
- التصميم البصري ونظام التصميم — عضو الـ UI/UX (صممه، وإنت نفذته كودًا).
- الاختبارات — عضو التستينج.

لو الدكتور سأل «ليه سبت الحاجات دي؟» الإجابة: «عشان كل عضو يبقى عنده جزء حقيقي يقدر يشرحه ويدافع عنه، مش حنة صغيرة من شغل حد ثاني.»

#### اختبر نفسك

##### 1. إيه دورك في المشروع؟

صممت المعمارية ووزعت الفريق، وبنيت الدفع وغرفة الصفقة والأدمن والبنية التحتية للباكند وطبقة الفرونت المشتركة.

##### 2. قد إيه نصيبك التقني؟

أكثر من 9000 سطر في الباكند — تلتين الباكند كله — بالإضافة لطبقة الفرونت المشتركة و13 صفحة أدمن.

##### 3. ليه مسكتش المصادقة والمشاريع بنفسك؟

قرار قيادي: كل عضو لازم يبقى عنده جزء حقيقي يقدر يدافع عنه بعمق، مش حنة من شغل حد ثاني.

## 2. Opening the Defense — Your First Two Minutes

أول دقيقتين بيحدّوا نبرة المناقشة كلها. الفصل ده بيديك اللي تقوله بالطبط.

## 2.1. The Opening

اعرف ده كويس

«فيستورا منصّة تمويل من ناحيتين: المؤسّس بينشر مشروع، والمستثمر بيلتزم بمبلغ، واللاتين بيتفاوضوا في غرفة صفقة، وبعدين الفلوس بتدفع — على دفعات لو حبوا — في صندوق تجريبي. أنا قائد الفريق: صمّمت المعمارية، ووَزّعت الشغل على خمس أعضاء، وبنيت أضخم نصيب تقني — الدفع، غرفة الصفقة، الأدمن، البنية التحتية، وطبقة الفرونت المشتركة. وأهم قرار في المنتج كله مفرداتي: إحنا بنفّرق بين اهتمام، والتزام، وتمويل — وعمرنا ما بنقول جُمعت فلوس على فلوس ما وصلتش.»

## 2.2. If They Ask for One Thing to Show

اعرف ده كويس

افتح شاشة تمويل المؤسّس بعد التسوية (في فصل «المودل المالي مثبت على الشاشة»). الأربع أرقام عليها بيشرحوا فلسفة المنتج كلها في ثلاث ثواني: اهتمام، ملتزم، ممول، صافي بعد الرسوم.

## 2.3. The Three Numbers to Have Ready

| الرقم     | معناه                                  |
|-----------|----------------------------------------|
| 9000+ سطر | نصيبك في الباكد — تلتين الباكد كله.    |
| 29 ترحيل  | عمر السكيما من أول نسخة لآخر واحدة.    |
| 76 اختبار | كلهم ناجحين — 48 على المنطق و28 تكامل. |

جدول 3 ثلاث أرقام تحفظهم بالظبط.

خد بالك

ماتبدأش بالتقنيات. «استخدمنا NET و Next.js» جملة أي مشروع يقولها. ابدأ بـ\*المشكلة (منصّات بتقول جُمع على وعود)\* والقرار (مفردات صارمة مفروضة بالكود) — والتقنية تيجي في سياقها لما يسأل.

## 3. The Product in Two Pages

قبل أي تفصيلة تقنية — لازم تعرف تشرح إيه المنتج ده في نص دقيقة.

### 3.1. What Vestora Is

اعرف ده كويس

منصّة تمويل من ناحيتين. المؤسّس (Innovator) بينشر مشروع محتاج تمويل، والمستثمر (Investor) بيطلب إنه يدعمه، والمؤسّس بيوافق، وبعدين اللاتين بيتفاوضوا في غرفة صفقة خاصة، ولما يتفقوا المؤسّس بيطلب الفلوس والمستثمر بيدفع في صندوق تجريبي. والأدمن بيراجع ويوفّق.

### 3.2. The Money Vocabulary — The Most Important Thing in the Product

| المصطلح                  | معناه بالطبط                                                               |
|--------------------------|----------------------------------------------------------------------------|
| <b>Interest</b> (اهتمام) | مستثمر طلب يدعم مشروع. مجرد طلب — مفيش أي التزام.                          |
| <b>Committed</b> (ملتزم) | المؤسس وافق على الطلب. التزام على الورق، <b>مش فلوس وصلت</b> .             |
| <b>Funded</b> (ممول)     | عملية دفع <b>Succeeded</b> فعليًا. ده <b>بس</b> اللي يتقال عليه فلوس وصلت. |

**جدول 4** التسلسل: **Interest ≥ Committed ≥ Funded** — وعمره ما بينعكس.

**خد بالك**  
دي القاعدة رقم 1 في **AGENTS.md**، وهي أهم قرار منتج في المشروع كله.  
السبب: منصّات تمويل كتير بتقول «تم جمع 500 ألف» وهي تقصد «ناس وعدوا بـ 500 ألف». الفرق بين الاثنين هو الفرق بين حقيقة وادّعاء — وفيستورا **بترفض** تخلط بينهم في أي شاشة أو أي حساب.  
لو الدكتور سأل سؤال واحد بس عن المنتج، غالبًا هيبقي ده.

## 4. System Architecture

### 4.1. The Shape

| الطبقة          | التقنية والدور                                               |
|-----------------|--------------------------------------------------------------|
| الواجهة         | Next.js 16 · React 19 · TypeScript · Tailwind v4             |
| API             | NET 8 Web API. — الكنترولرز والخدمات                         |
| الوصول للبيانات | EF Core — <b>AppDbContext</b> مباشرة، <b>Repository</b> مفيش |
| قاعدة البيانات  | SQL Server — بعيدة ومشتركة بين الفريق                        |
| الوقت الفعلي    | SignalR — <b>ChatHub</b> للرسائل والإشعارات                  |
| الدفع           | مزود قابل للتبديل — محاكي أو Stripe <b>تجريبي فقط</b>        |

**جدول 5** ستّ طبقات، كل واحدة بقرار وراها.

### 4.2. Three Architectural Decisions You Made

**اعرف ده كويس**  
**القرار الأول: مفيش طبقة Repository.**  
الكنترولر بيكلم **AppDbContext** مباشرة. السبب: **DbContext** نفسه أصلًا Repository و Unit of Work — إضافة طبقة فوقه كانت هتبقى غلاف بينده على غلاف من غير ما يضيف حاجة.  
الخدمات موجودة **بس** لما يكون فيه منطق حقيقي مشترك: **AuthService**، **PaymentService**، **TermSheetService**. وده معيار واضح مش مزاج.

اعرف ده كويس

القرار الثاني: Status و Stage محورين منفصلين.

- Investment.Status = بوابة التمويل — «Approved» بس هي اللي بتتسبب في إجمالي المشروع.
- Investment.Stage = اللي بيحصل بين شخصين — New ← Reviewing ← Approved ← Contacted ← InDiscussion ← Committed ← Closed.

دمجهم كان هيجلي «الصفقة في مرحلة نقاش» و«الفلوس اتحسبت» نفس الحاجة — وهما مش نفس الحاجة خالص.

اعرف ده كويس

القرار الثالث: كل رقم مالي بيتحسب في FundingMath وبس.

قبل الملف ده، تعبير «إجمع الاستثمارات المعتمدة» كان مكتوب بإيد في **خمستاشر مكان** — التصقّح، المحفوظات، اللوحتين، تحليلات الأدمن، الترتيب، غرفة الصفقة. كانوا متفقين بالصدفة، وكلهم كانوا بيسمّوا موافقة المؤسس «raised».

دلوقتي فيه مصدر واحد. لو الرقم غلط، غلط في مكان واحد — مش في خمستاشر.

## 5. The Team Division

الفصل ده أهم فصل قيادي — الدكتور غالبًا هيسأل «مين عمل إيه؟».

| العضو        | نصيبه                                                                                                                                           |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| إنت — الليدر | المعمارية · الدفع والتفاوض · الأدمن · اللوحات · الرسائل · البنية التحتية · اللاندينج · نظام التصميم في الكود · طبقة API · الترجمة · الأونبوردنج |
| عضو الباكند  | المصادقة والحسابات كاملة · المشاريع والاكتشاف · القصة والتحديثات · التفاعل والمتابعة · الإيميل                                                  |
| عضو الفرونت  | صفحة التصقّح (أعقد صفحة) · نماذج الحساب الخمسة · صفحة المشروع · الإعدادات · دليل المستثمرين · البحث المحفوظ                                     |
| عضو الداتايز | مخطط قاعدة البيانات · العلاقات والفهارس · الحذف الناعم · الترحيلات · الميكانيكا كاملة                                                           |
| عضو UI/UX    | تصميم كل الشاشات · نظام التصميم (التوكنز والقواعد) · الخطوط والألوان · التصميم ثنائي اللغة · التسليم للفرونت                                    |
| عضو التستينج | 28 اختبار تكامل · توثيق 13 محاولة اختراق · خطة اختبار يدوي بـ 65 سيناريو                                                                        |

**جدول 6** ستّة أعضاء، وكل واحد عنده جزء حقيقي يدافع عنه.

اعرف ده كويس

المبدأ اللي وزعت به: كل عضو ياخذ مجال كامل يقدر يشرحه من أوله لآخره — مش حتّة صغيرة من شغل حد ثاني.

عشان كده عضو الباكند خد المصادقة كاملة (مش نصّها)، وعضو الفرونت خد التصقّح كامل (مش كام مكوّن منه). لو الدكتور سأل أي واحد فيهم، هيلقي قدامه حد فاهم مجاله كله مش جزء منه.

## خد بالك

فيه **حدود مشتركة** لازم تعرفها لأن الدكتور ممكن يمسك واحد منها:

- `AccountRules.cs` (عضو الباكند) لازم تفضل **مطابقة** لـ `rules.ts` (بتاعتك) — قاعدة مكتوبة في `AGENTS.md`.
- `TrustSignals` (بتاعك) بينده عليه كمنترولز عضو الباكند.
- لما عضو الباكند احتاج عمود جديد في الداتابيز، إنت اللي عملت الـ entity والمigration.
- التصميم بتاع الـ UI/UX انتقذ في الكود بإيدك في `globals.css`.

## اختبر نفسك

## 1. ليه مغيث Repository في المشروع؟

`DbContext` نفسه Repository و Unit of Work — طبقة فوقه غلاف على غلاف. الخدمات موجودة بس للمنطق المشترك الحقيقي.

## 2. إيه الفرق بين Status و Stage؟

Status بوابة التمويل — Approved بس بتتسبب Stage اللي بيحصل بين شخصين. دمجهم كان هيخلط النقاش بالفلوس.

## 3. ليه FundingMath موجود؟

قبله نفس الحساب كان مكتوب بإيد في 15 مكان واتفقوا بالصدفة، وكلهم سمّوا الموافقة «raised». دلوقتي مصدر واحد.

## 4. على أي أساس وزّعت الفريق؟

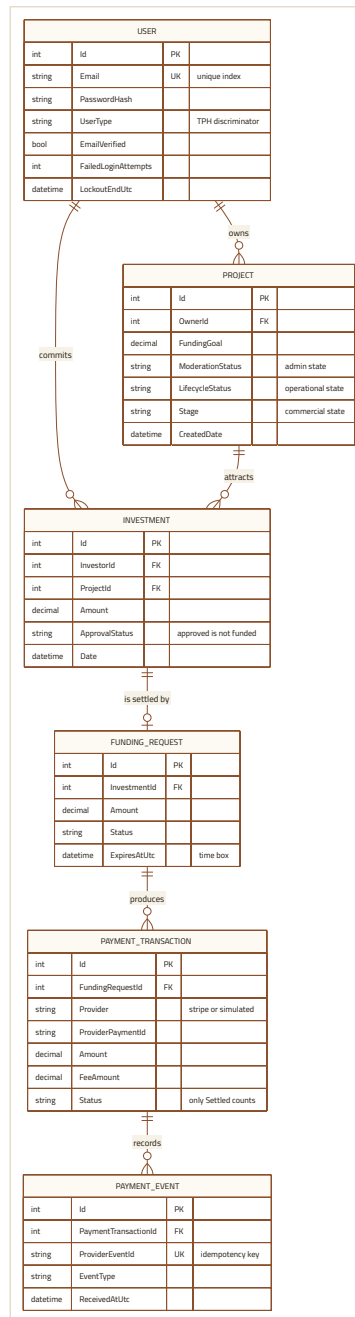
كل عضو ياخذ مجال كامل يقدر يشرحه من أوله لآخره، مش حتّة من شغل حد ثاني.

## 6. The Data Model — Your Architectural Decisions

عضو الداتابيز ماسك الميكانيكا كاملة (الفهارس، الترحيلات، الحذف الناعم). الفصل ده بيغطّي قراراتك إنت: شكل المخطط وليه.



## 6.3. The Money Trail



شكل 2 سلسلة التمويل: من العلاقة، لطلب التمويل، للمعاملة، لحدث المزود.

اعرف ده كويس

السلسلة دي أربع مستويات عن قصد:

PaymentEvent ← PaymentTransaction ← FundingRequest ← Investment

- **العلاقة** هي الاتفاق بين شخصين.
  - **طلب التمويل** هو المؤسس بيبطلب رقم معين — ويمكن يتكرر (دفعات).
  - **المعاملة** هي محاولة دفع واحدة — ويمكن تفشل وتتعاد.
  - **الحدث** هو تأكيد المزود — وده اللي بيمنع الدفع المزدوج.
- كل مستوى موجود لأن اللي فوقه ما ينفعش يمثلّه بأمانة.



## 7. Payments — The Most Careful Code in the Project

PaymentService (1259 سطر) — أخطر ملف في المشروع، لأنه يتعامل مع فلوس.

### 7.1. The Two Non-Negotiable Rules

اعرف ده كويس

التعليق في أول الملف يبلّغهم:

«التأكيد بيتقبل مرة واحدة بس، مفروض بفهرس فريد مش بكود حريص. والمتصفح اللي راجع لصفحة نجاح مش دليل — كل تسوية بتتقرّر بسؤال المزود أو بـ webhook موقع.»  
يعني: مغيث ثقة في العميل، ومغيث اعتماد على إن الكود «هيفتكر» يتأكد.

### 7.2. Event Before Effect — Rule 5

خد بالك

ده أهم كود في المشروع كله. الترتيب مقصود ومحمي بمعاملة قاعدة بيانات:

```
1 await using var tx = await _db.Database.BeginTransactionAsync(ct);
2
3 var evt = new PaymentEvent { ... Applied = false };
4 _db.PaymentEvents.Add(evt);
5 await _db.SaveChangesAsync(ct); // الحدث بيتسجل الأول ←
6
7 // وبعدين بس الأثر بيتطبق ...
```

التعليق بيشرح ليه المعاملة مش زخرفة: «حجز مفتاح عدم التكرار هو نفسه كتابة دائمة. لو العملية ماتت بين حجز المفتاح وتطبيق الأثر — نشر، اتصال اتقطع، مهلة انتهت — المفتاح هيعيش والتسوية لأ، ومغيث إعادة محاولة تقدر تسترجعها لأن إعادة المحاولة هي بالطبط الذي المفتاح بيمنعها دلوقتي. الدفعة كانت هتضيع بصمت.»

### 7.3. Idempotency by Index, Not by Code

اعرف ده كويس

الفهرس الفريد على (Provider, ProviderEventId) هو اللي بيمنع التكرار — مش فحص if في الكود. الفرق مهم: webhook مكرّر، وصفحة رجوع اتضغطت مرتين، و webhook بيسابق صفحة الرجوع — كلهم بيتصادموا عند الفهرس، قبل ما أي رقم يتحرّك، والخسران بيخرج بهدوء.

```
1 catch (DbUpdateException ex) when (IsUniqueViolation(ex))
2 {
3     await tx.RollbackAsync(ct);
4     return false; // مكرّر - اتجاهل بهدوء
5 }
```

## 7.4. The Reconciliation Alarm

خد بالك

فيه حالة واحدة الكود بيصرّح فيها بدل ما يتجاهل:

المزوّد يقول الفلوس اتأخذت، وفيستورا كانت قافلة المحاولة كفاشلة.

```
1 _logger.LogError(
2     "RECONCILIATION: provider reports transaction {Reference} as paid, but
Vestora " +
3     "already closed it as {Status}. Funding was NOT recorded.");
```

التعليق بيقول: «ده معناه إن الطرفين مختلفين على دفعة حقيقية، ولازم يبقى صوته عالي مش يتسجّل كتنكرار عادي.» وصف الحدث بيتحفظ كـ **CONFLICT** عشان التسوية تتعمل بالإيد من سجّل المراجعة. ودي إجابة ممتازة لو الدكتور سأل «إيه اللي بيحصل لو حصل خطأ في الدفع؟»

## 7.5. The Fee Is Frozen at Checkout

اعرف ده كويس

```
1 var feeRateBps = transaction.FeeRateBps > 0 ? transaction.FeeRateBps :
_settings.FeeRateBps;
```

النسبة اللي اتحسبت وقت فتح الدفع، مش اللي في الإعدادات دلوقتي. التعليق بيشرح: «قراءة الإعدادات الحية هنا كانت هتخلّي تغيير النسبة بين checkout والتسوية يغيّر عائد المؤسّس بعد ما اتعرض عليه رقم.» نافذة صغيرة، بس دي فلوس المؤسّس.

# 8. Tranche Funding — Paying a Commitment in Instalments

الفصل ده عن أحدث ميزة كبيرة في المشروع، وهي بتحلّ مشكلة حقيقية في تمويل الشركات.

## 8.1. The Problem

اعرف ده كويس

الشيكات الكبيرة في الواقع مبتتدفعش مرة واحدة. مستثمر بيلتزم بـ 50 ألف ممكن يدفعها على ثلاث دفعات حسب مراحل المشروع.

الكود القديم كان بيسأل: «هل نجحت دفعة واحدة؟» — ولو أيوه، يقفل الطلب ويعتبر العلاقة ممولة بالكامل. التعليق بيوصف الكارثة بالظبط: «كان هيعلن العلاقة مكتملة عند أول قسط، ويقفل الطلب نهائياً، ويسيب باقي الفلوس مستحيل تحصيلها — والرصيد لسه محسوب كالتزام.»

## 8.2. The Fix — Sum, Not Existence

اعرف ده كويس

```
1 public static bool IsFullySettled(decimal settled, decimal commitment) =>
2   commitment <= 0m ? settled > 0m : settled >= commitment;
```

السؤال اتغير من «هل فيه دفعة نجحت؟» لـ «هل مجموع اللي اتدفع وصل للالتزام؟». ودي حالة كلاسيكية لفرق بين `Any()` و `Sum()` — بس هنا الفرق بينهم فلوس ضايعة.

## 8.3. Four Functions That Make It Work

| الدالة                                 | بتعمل إيه                                                           |
|----------------------------------------|---------------------------------------------------------------------|
| <code>SettledForInvestmentAsync</code> | مجموع كل الدفعات الناجحة لعلاقة واحدة — عبر كل الدفعات.             |
| <code>CommitmentTargetAsync</code>     | الرقم المستهدف: الشروط المتفق عليها لو موجودة، وإلا الطلب الأصلي.   |
| <code>IsFullySettled</code>            | هل المجموع وصل للمستهدف؟                                            |
| <code>UnsettledCommitment</code>       | قد إيه فاضل ينفع يتطلب: <code>Max(0, commitment - settled)</code> . |

**جدول 7** أربع دوال في `FundingMath` بيبنوا عليها كل حسابات الدفعات.

خد بالك

**تفصيلة دقيقة جدًا — الشروط المتفق عليها بتغلب الطلب الأصلي:**

التعليق بيقول: «رقم المستثمر الأصلي هو اللي طلبه قبل ما حد يتكلم؛ وورقة الشروط هي اللي الاتنين قبلوها **بعدين**. وقياس الدفعات على الأول كان هيعلن العلاقة مكتملة والاتفاقية اللي هي قائمة عليه لسه نصّها مدفوع.» يعني لو المستثمر طلب 30 ألف، وبعد التفاوض اتفقوا على 50 ألف — الدفعات بتتقاس على الـ 50، مش الـ 30.

## 8.4. The Double-Counting Guard

اعرف ده كويس

أذكى سطرين في الميزة كلها:

```
1 var headroom = await FundingMath.HeadroomForAsync(...);
2 headroom = Math.Max(0m, headroom - settled);
```

التعليق بيشرح المشكلة: «الدفعات اللي اتدفعت خرجت من مساحة الجولة عن طريق المجموع المدفوع، بس **ماخرجتش من التزام العلاقة دي**، اللي لسه محسوب كامل. طرحهم هنا هو اللي بيمنع المؤسّس إنه يطلب 100% من الالتزام **مرتين** عن طريق تقسيمه.»

من غير السطر ده، مؤسّس يقدر يقسّم التزام 50 ألف لدفعتين ويطلب 50 ألف في كل واحدة — يعني 100 ألف من التزام 50.

## 9. The Payment State Machine

كل معاملة دفع بتعدي على حالات محدّدة — والانتقالات بينهم مش عشوائية.

### 9.1. Six States

| الحالة     | نهائية؟ | معناها                                         |
|------------|---------|------------------------------------------------|
| Initiated  | لا      | الجلسة افتتحت، المستثمر لسه ما دفعش.           |
| Processing | لا      | المزوّد بيعالج.                                |
| Succeeded  | أيوه    | الفلوس وصلت — دي الوحيدة اللي بتعني ممول.      |
| Failed     | أيوه    | الدفع اترفض.                                   |
| Cancelled  | أيوه    | اتلغى — بإرادة المستثمر أو بانتهاء المهلة.     |
| Refunded   | أيوه    | اتسترد — الاستثناء الوحيد للتعديل بعد النهاية. |

جدول 8 ست حالات، أربعة منهم نهائية.

اعرف ده كويس

الصفوف النهائية ثابتة — قاعدة رقم 5.

```
1 if (PaymentStatus.IsTerminal(transaction.Status))
2 {
3     evt.Outcome = $"ignored · already {transaction.Status}";
4     return false;
5 }
```

لو المزوّد بعت تأكيد لمعاملة خلصت خلاص، الحدث بيتسجّل بس مغيش أثر بيتطبّق. والاستثناء الوحيد `Succeeded → Refunded` وده فعل أدمن بيمشي في مسار ثاني خالص. ودي إجابة سؤال «طيب لو المزوّد بعت نفس التأكيد مرتين؟» — الحدث بيتسجّل للسجلّ، والأثر مبيتطبّقش ثاني.

### 9.2. Four Cancel Reasons

| السبب          | إمتى                                 |
|----------------|--------------------------------------|
| user_cancelled | المستثمر ضغط إلغاء بنفسه.            |
| abandoned      | فتح checkout وساب الصفحة.            |
| expired        | الطلب نفسه انتهت مهلته.              |
| round_closed   | المؤسّس قفل الجولة والطلب لسه مفتوح. |

جدول 9 مش كل إلغاء زي الثاني — والسبب بيتخزّن.

اعرف ده كويس

ليه نفرّق؟ لأن «المستثمر رفض» و«المستثمر نسي» و«الجولة قفلت» ثلاث قصص مختلفة تمامًا في التحليلات. لو كلهم `Cancelled` وخلاص، `AdminInsights` ماكانش هيقدر يقول «فين العلاقات بتقف».

### 9.3. The Sweeper — Closing What Nobody Finished

اعرف ده كويس

التعليق يشرح المشكلة بدقّة:

«حاجتين بيعغنوا لو اتسابوا. checkout المستثمر مشي منه بيفضل «Initiated» للأبد، ويمنع إعادة المحاولة اللي كانت هتخليه يجرب ثاني. وطلب تمويل محدش دفعه بيمسك حصّة من جولة كان ممكن مستثمرين تانيين ياخدوها.»

الحل: خدمة خلفية بتلفّ كل ثلاث دقائق. والتعليق بيوضّح ليه بطيئة: «دي مسك دفاتر، مش طابور — فبتشتغل كل كام دقيقة وماتكلّفش حاجة لما مافيش حاجة تتعمل.»

خد بالك

تفصيلة مهمة في إفعال الجولة: لما المؤسّس يقفل جولة، كل طلب مفتوح بيتسحب — ما عدا واحد المستثمر واقف قدامه بيدفع دلوقتي.

التعليق يشرح: «سحبه من تحته وسط الدفع بيتج عملية دفع مالهاش حاجة تتعلّق بيها. لو نجحت، التسوية بتعلّم الطلب مدفوع والسجلّ بيقرأ بأمانة: اتسحب، والدفعة اللي كانت ماشية خلصت برضه.»

## 10. The Deal Room and Term Sheets

أكبر ملف في الباكند كله. (1287 سطر) DealRoomController

### 10.1. What Lives in One Screen

| القسم           | الوظيفة                                            |
|-----------------|----------------------------------------------------|
| حالة الالتزام   | الأرقام: المتفق عليه، المدفوع، الفاضل.             |
| الجدول الزمني   | كل مرحلة عدّت عليها العلاقة، بتاريخها ومين حرّكها. |
| الشروط          | TermSheet — الاقتراح والقبول والاستبدال.           |
| الأسئلة         | المستثمر ببسأل، المؤسّس بيجاب.                     |
| طلبات المستندات | المستثمر بيطلب مستند، المؤسّس بيرفعه.              |
| المحادثة        | رسائل مباشرة بين الطرفين.                          |

جدول 10 ست أقسام في شاشة واحدة — أعقد شاشة في المنتج.

### 10.2. The Term Sheet Service

اعرف ده كويس

ليه خدمة منفصلة مش كود في الكنترولر؟ التعليق بيجاب:

«موجودة كخدمة لنفس سبب الدفع: دي المكان الوحيد اللي فيه اتفاق معلن بيتحوّل لحقيقة باقي النظام بيقرأها. طلب التمويل بيفحصها. خط الأنايب بيتحرّك عليها. كنترولرين بيكتبوا نسختهم من «الطرفين وافقوا» هو بالطبط شكل الباك اللي الكود ده دفع تمنه مرة قبل كده في إجماليات التمويل.»

اعرف ده كويس

المقترح يقبل ورقته بنفسه تلقائياً:

«إنك تقترح حاجة وبعدين حد يسألك هل موافق عليها — ده نموذج محدث المفروض يملاه. موافقة الطرف الثاني هي اللي بتقرر أي حاجة.»

**والورقة الحية بتتسبدل مش بتترفض:** «إعادة التفاوض هي الحالة الطبيعية في النقطة دي من الصفقة، وإجبار المقترح إنه يسحب الأول كان هيسيب نافذة العلاقة فيها بلا شروط خالص.»

## 11. The Dashboards — Five Controllers, One Question Each

اللوحات عندك **1839 سطر** موزعين على خمس كينترولرز. كل واحد بيجاب سؤال مختلف تمامًا.

| الكنترولر                   | الأسطر | بيجاب سؤال                                                      |
|-----------------------------|--------|-----------------------------------------------------------------|
| InvestorController          | 690    | كل حاجة عن مستثمر: المشاريع اللي دعمها، ملخص استثماراته، نشاطه. |
| FounderDashboardController  | 415    | «إيه حالة مشاريعي كلها؟» — كل مؤشر ورسم بياني في طلب واحد.      |
| InvestorDashboardController | 300    | «إيه حالة استثماراتي كلها؟»                                     |
| CapitalController           | 257    | دليل المستثمرين — للمؤسس اللي بيدور على تمويل.                  |
| VentureInsightsController   | 177    | «إيه حالة المشروع ده بالتحديد؟»                                 |

**جدول 11** خمس لوحات، كل واحدة بسؤالها.

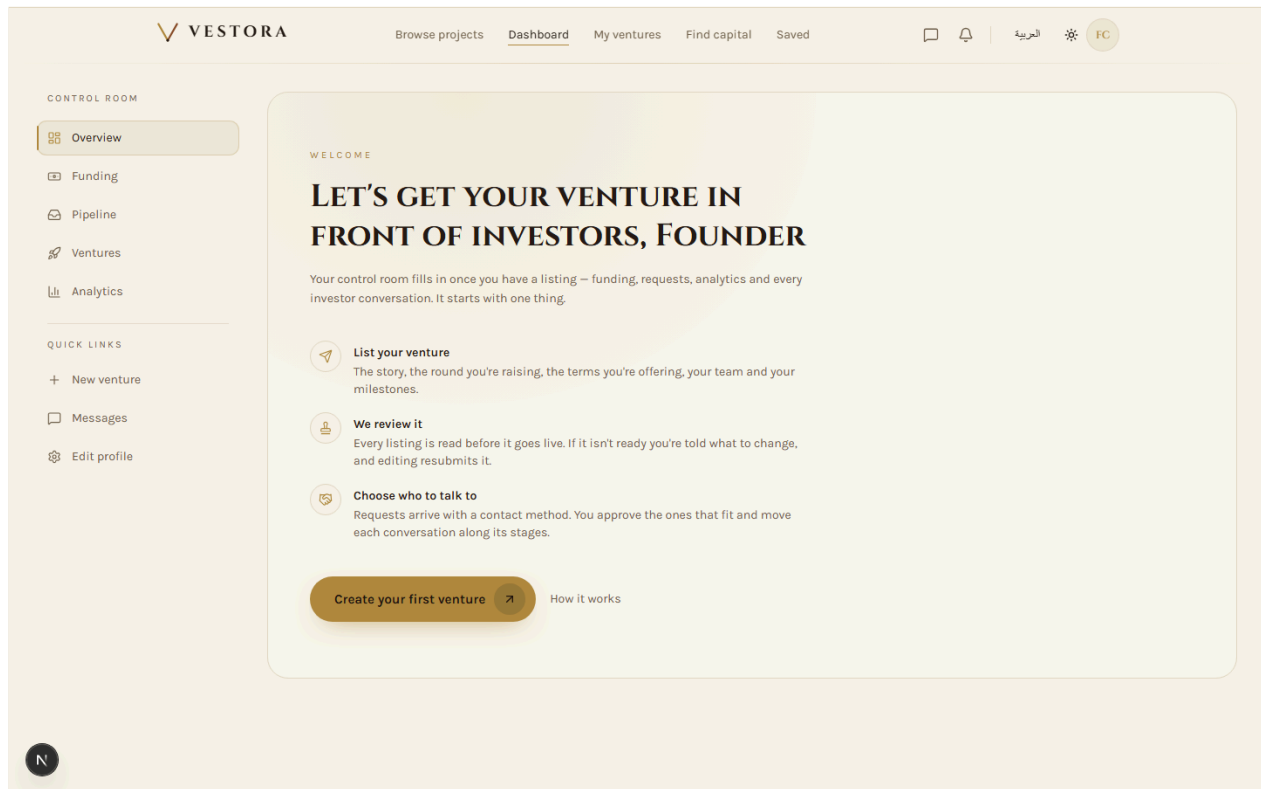
### 11.1. One Request, Everything

اعرف ده كويس

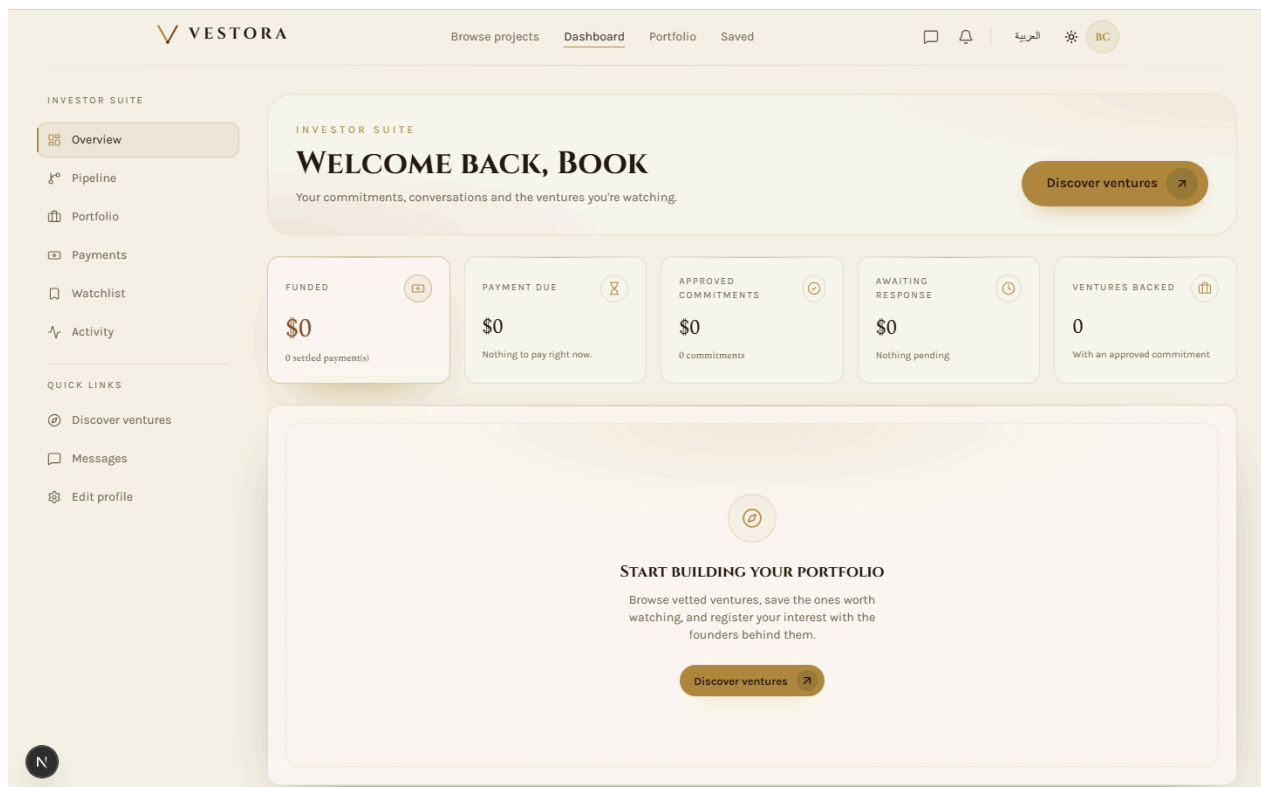
التعليق على **FounderDashboardController** بيشرح المبدأ:

«نموذج قراءة مجمّع للوحة تحكم المؤسس. طلب واحد بيرجع كل مؤشر وكل سلسلة رسم بياني وكل طابور اللي النظرة العامة محتاجها.»

ليه ده مهم؟ لأن البديل كان الصفحة تبعت ست أو سبع طلبات منفصلة — وكل واحد فيهم رحلة شبكة كاملة، والصفحة تفضل بتترسم على مراحل قدام المستخدم.



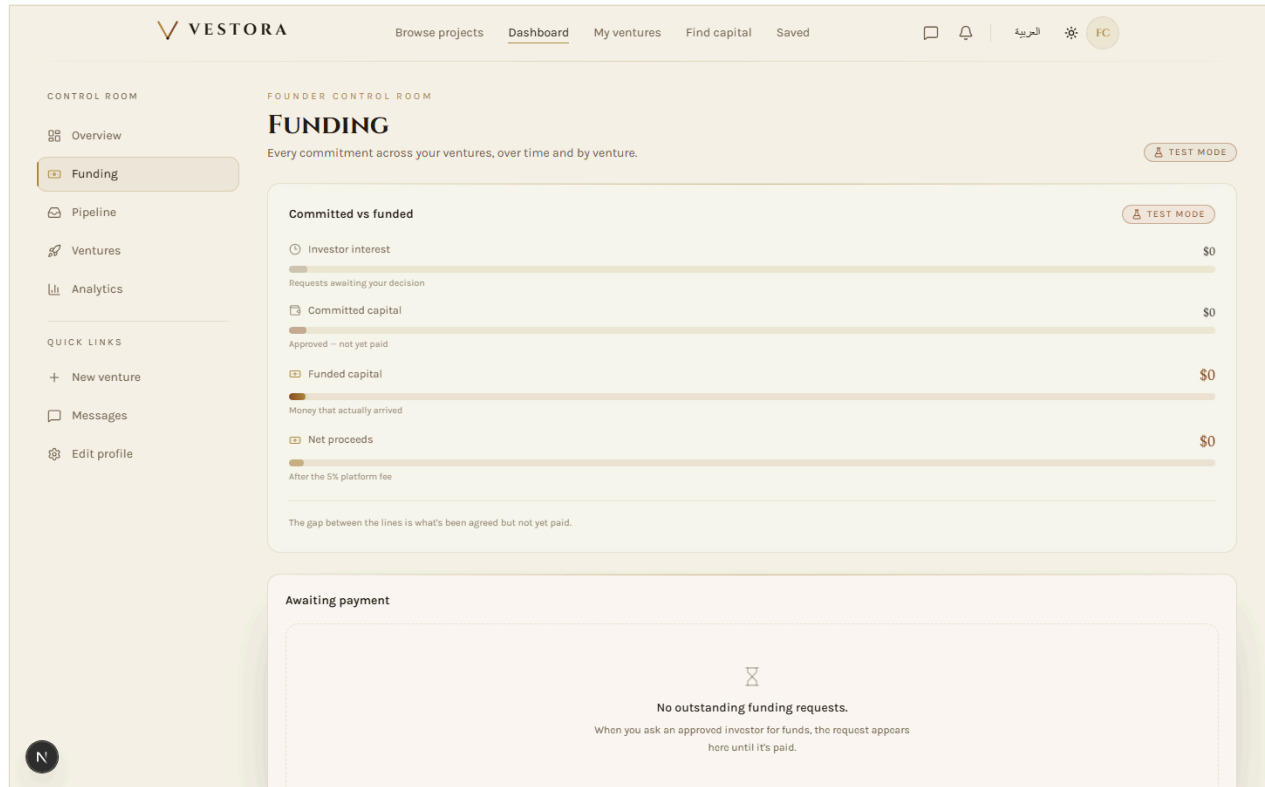
شكل 3 لوحة المؤسس: كل المؤشرات والرسوم في طلب واحد.



شكل 4 لوحة المستثمر — نفس المبدأ، سؤال مختلف.

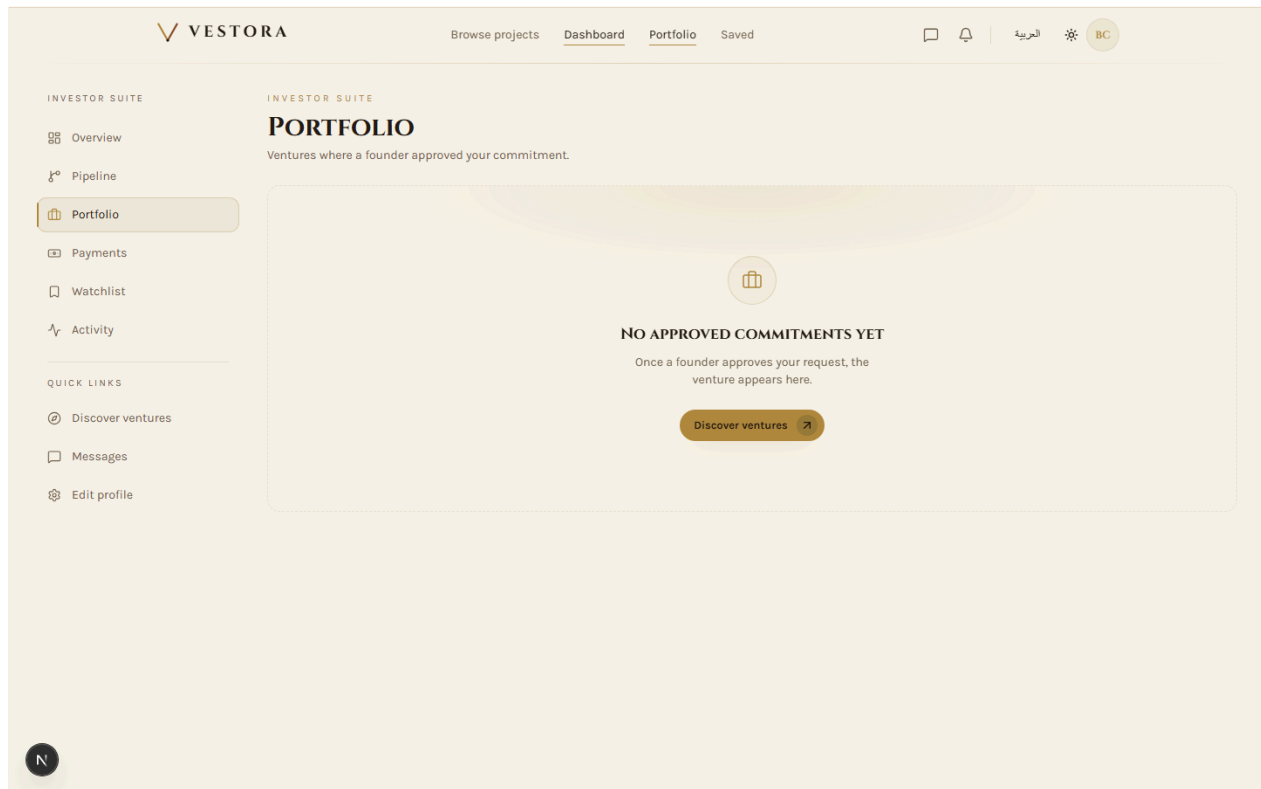
## 11.2. The Numbers on These Screens

خد بالك  
كل رقم في اللوحات دي بييجي من **FundingMath** — مفيش حساب محلي في أي كترولر منهم. ولاحظ إن **FounderDashboardController** بيأخد **PaymentService** في الـ constructor عشان الأرقام المالية تيجي من نفس المصدر بالظبط.  
ودي القاعدة رقم 2 مطبقة عملياً: لو الرقم في اللوحة اختلف عن الرقم في التصفح، يبقى فيه حساب اتكتب في مكان ثاني — وده بالظبط اللي **FundingMath** اتعمل عشان يمنعه.

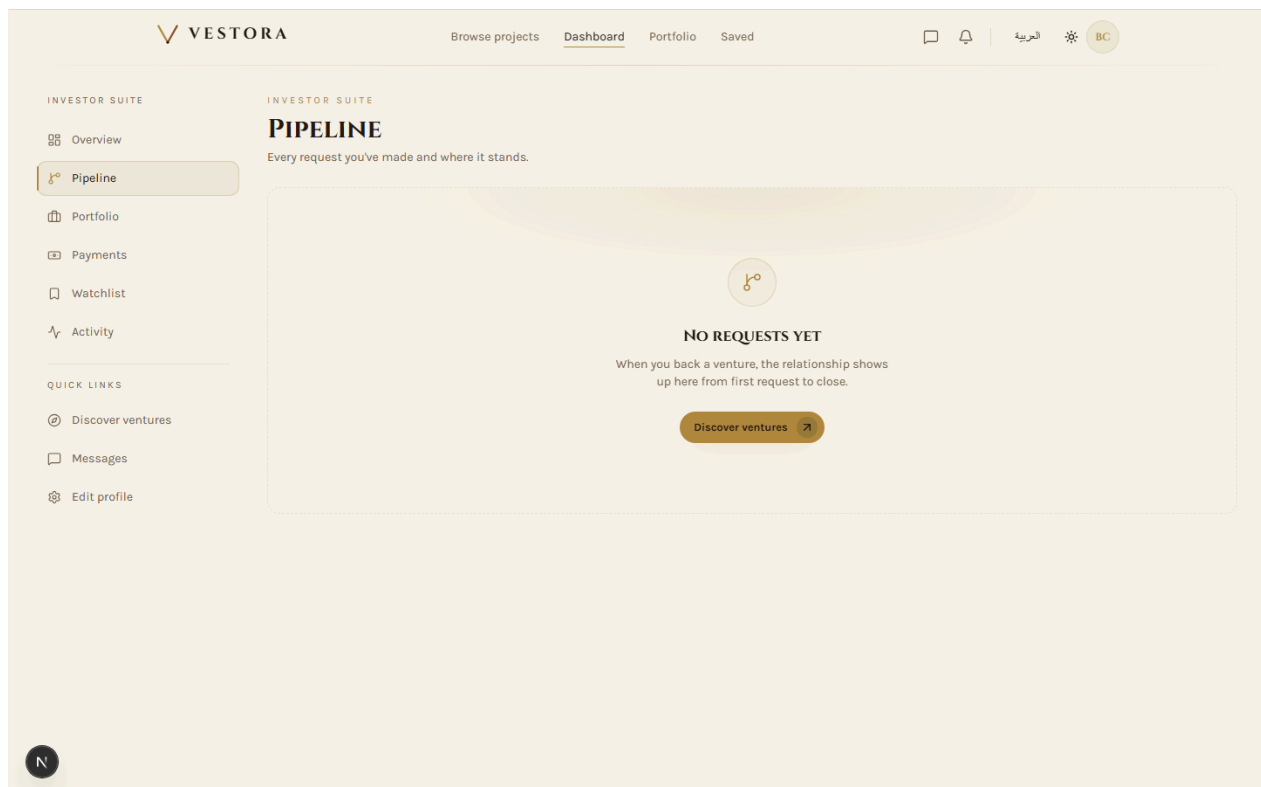


**شكل 5** شاشة التمويل: الاهتمام والملتزم والممول وصافي العائد — أربعة أرقام منفصلة، مش رقم واحد مجمّع.





شكل 6 محفظة المستثمر: المشاريع التي التزم فيها وحالة كل واحدة.

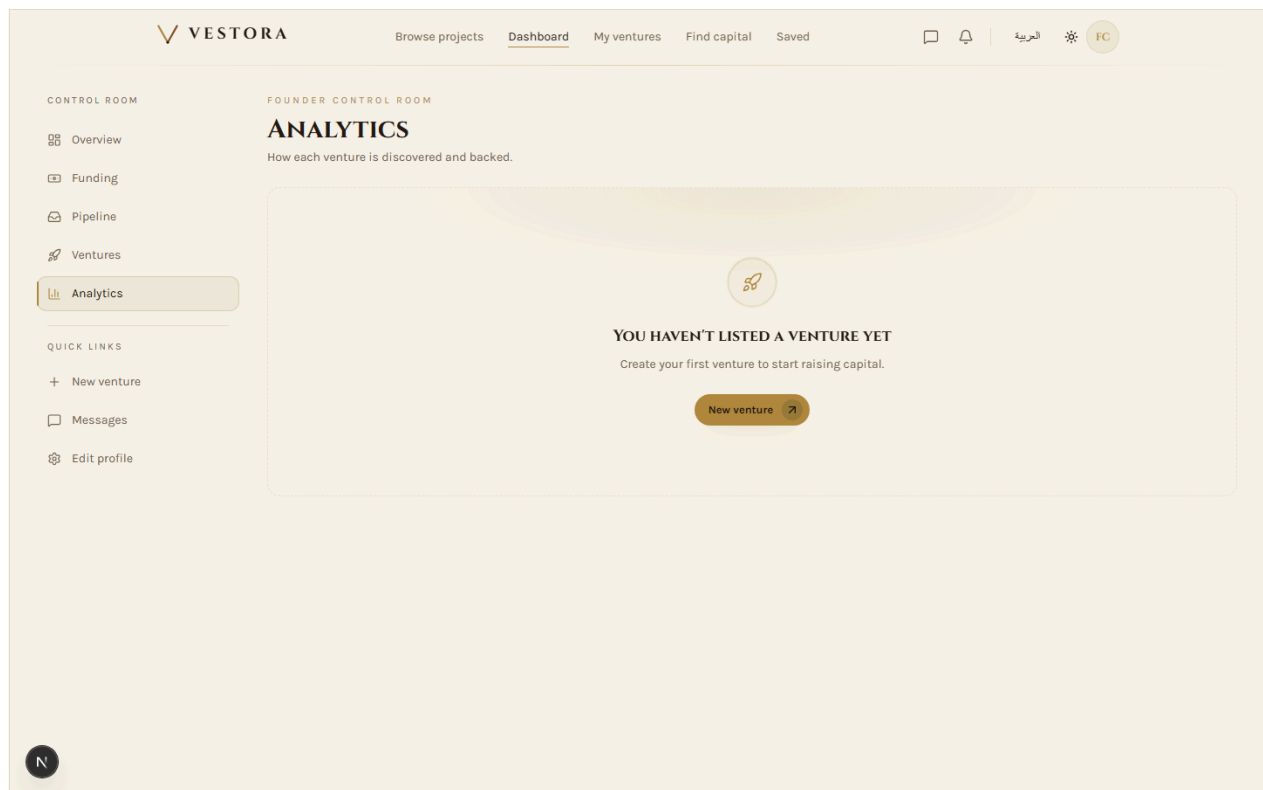


شكل 7 خط أنابيب المستثمر: كل علاقة وفي أي مرحلة هي دلوقتي.

#### اعرف ده كويس

خط الأنابيب ده هو **Investment.Stage** معروض بصريًا — والمراحل بتتحرك عن طريق **StageLog** وبس. لو الدكتور سأل «إزاي بتتبع حالة الصفقة؟» — دي الشاشة، ودي الخدمة وراها.

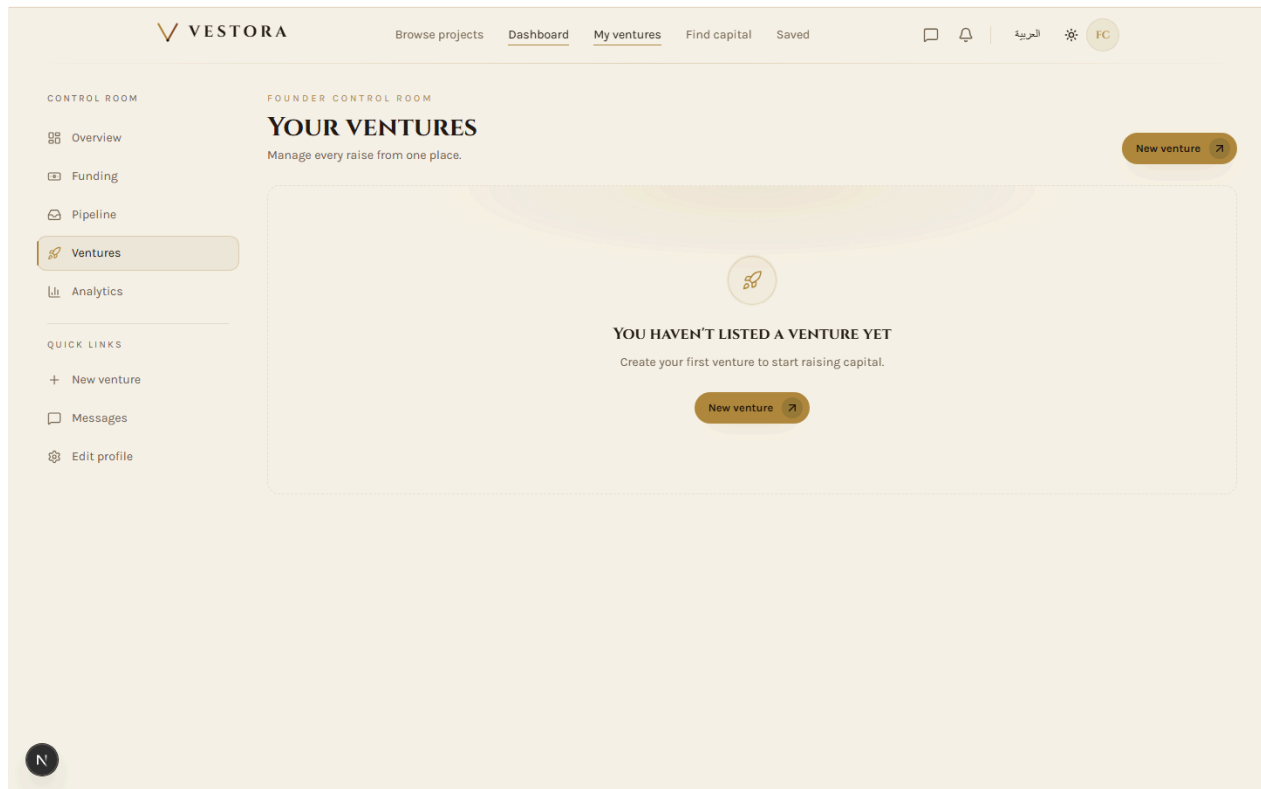
## 11.3. Venture Insights — One Project at a Time



**شكل 8** تحليلات المؤسّس: المشاهدات، ومنحنى الالتزامات، ومنحنى التمويل الفعلي — **منفصلين**.

### اعرف ده كويس

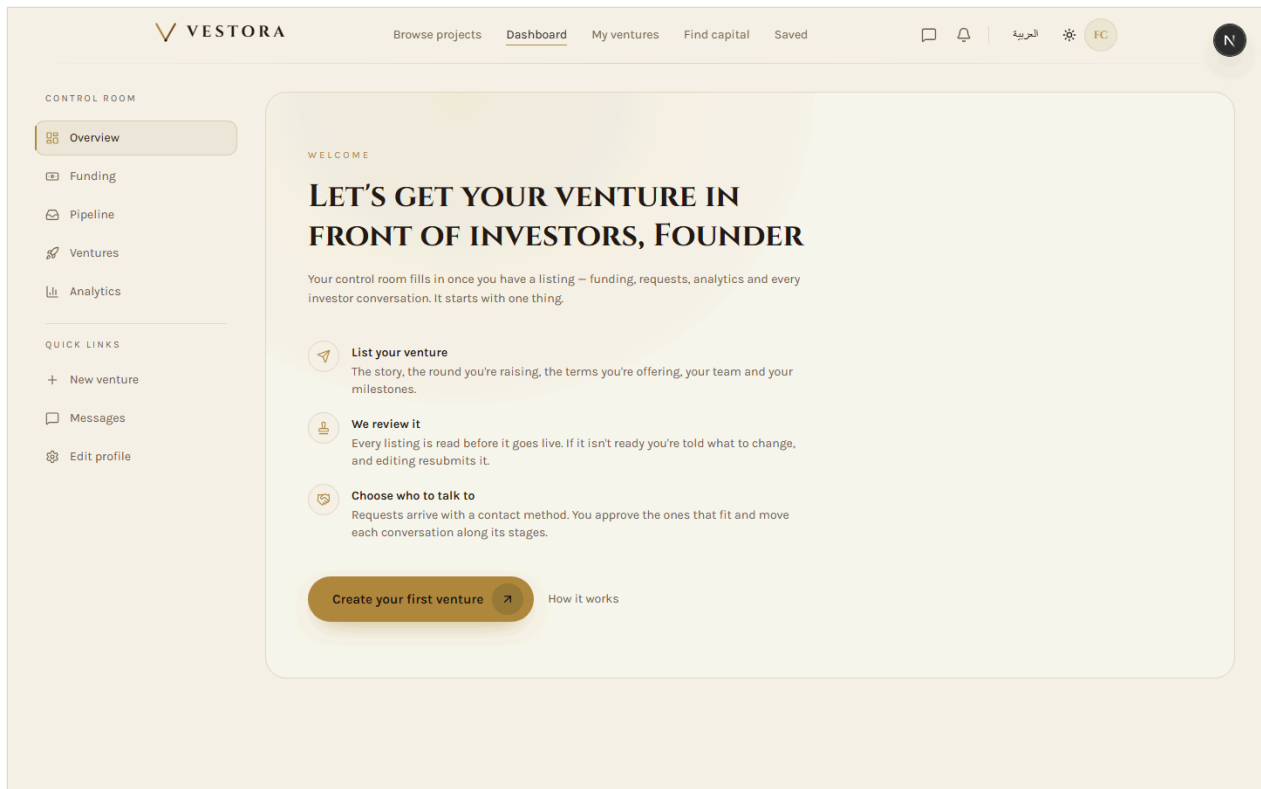
المنحنيين المنفصلين دول هما المفردات المالية معروضة بيانياً. التعليق في الكود بيشرح ليه: «تحليلات المؤسّس كانت بترسم الموافقات وتسمّي الخط **جمع**». دلوقتي فيه منحني للوعد ومنحنى للفلوس اللي وصلت، والمسافة بينهم هي **بالظبط** اللي لسه ما اتدفعش. ونسبة التحويل بقت بتقيس «الزوار اللي **مؤلوا فعلاً**، مش اللي اتوافق عليهم — نسبة أقسى بكتير وأصدق بكتير».



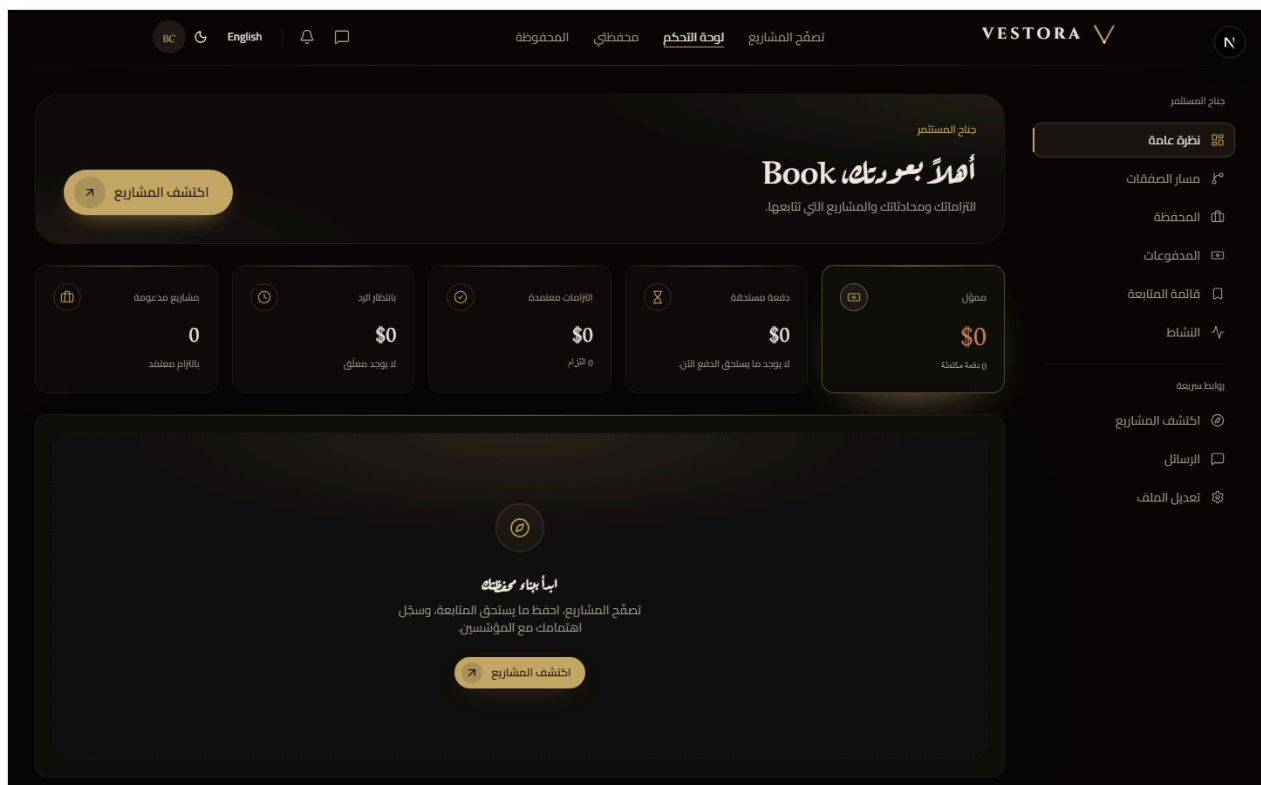
**شكل 9** كل مشاريع المؤسس في مكان واحد، بحالة كل واحد.

## 12. Three Roles, Three Control Rooms

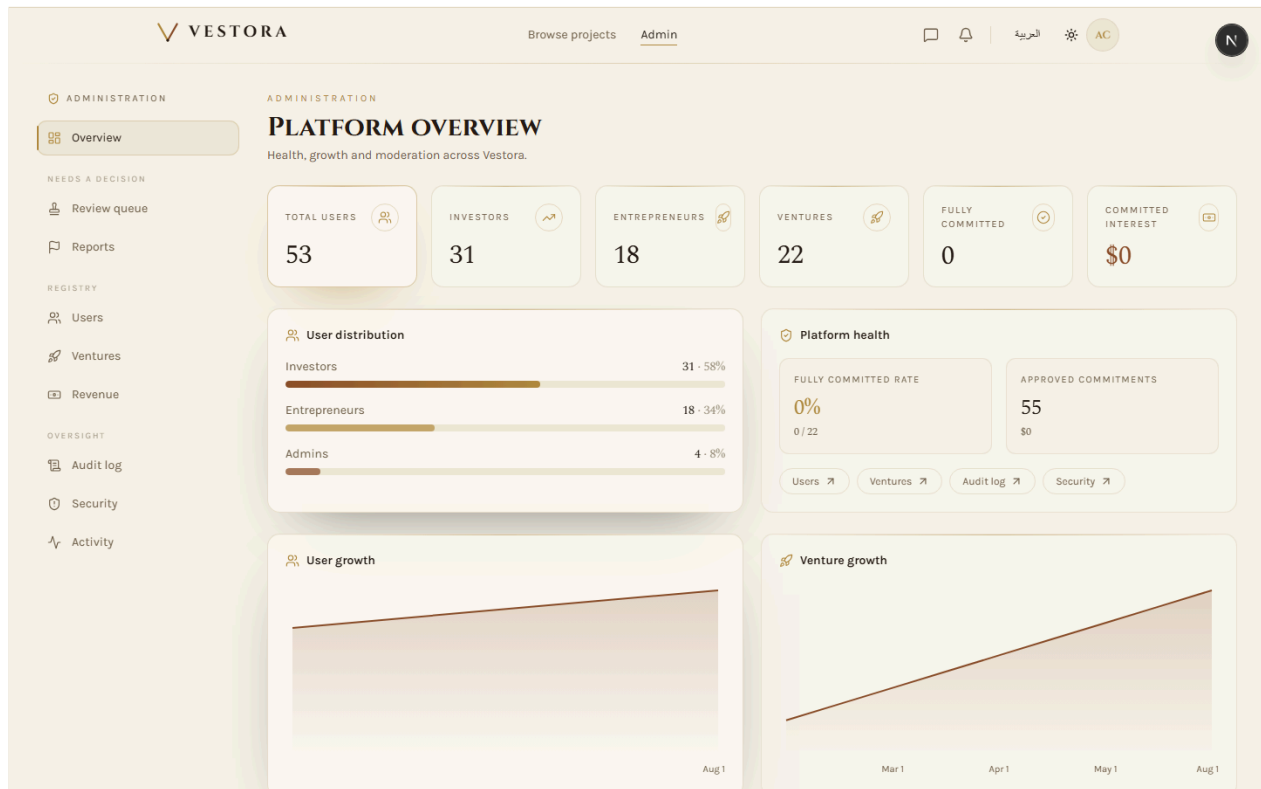
نفس المنتج يتعرض بثلاث أشكال مختلفة تمامًا حسب الدور. الفصل ده بيقرنهم.



شكل 10 لوحة المؤسّس — والشريط الجانبي يقول: Overview · Funding · Pipeline · Ventures · Analytics.



شكل 11 لوحة المستثمر بالعربي والوضع الداكن — Overview · Pipeline · Portfolio · Payments · Watchlist · Activity.



شكل 12 لوحة الأدمن — Administration · Needs a decision · Registry · Oversight

## 12.1. What the Sidebars Tell You

| الدور    | الشريط الجانبي ينعكس إيه                                      |
|----------|---------------------------------------------------------------|
| المؤسس   | «إزاي بجمع فلوس؟» — التمويل، خط الأنابيب، مشاريعي، التحليلات. |
| المستثمر | «فلوسي فين؟» — المحفظة، المدفوعات، المتابعة، خط الأنابيب.     |
| الأدمن   | «إيه اللي محتاج قرار؟» — وده أول قسم في الشريط، قبل السجلات.  |

جدول 12 التنقل نفسه بيقول للمستخدم إيه دوره.

### اعرف ده كويس

لاحظ إن شريط الأدمن مقسوم بعناوين: **NEEDS A DECISION** قبل **REGISTRY** قبل **OVERSIGHT**. الترتيب ده **مقصود**: الأدمن أول ما يدخل يشوف اللي محتاج تصرف منه، مش قوائم البيانات. وده نفس منطق تبويب «يحتاج قرارك» في الإشعارات.

### خد بالك

التلات لوحات دول بيستخدموا نفس المكوّنات — نفس الكارت، نفس الشريط الجانبي، نفس الحالة الفاضية. اللي بيتغير هو المحتوى بس. ودي إجابة لو الدكتور سأل «عملت تلات واجهات مختلفة؟» — لأ، واجهة واحدة بتلات تكوينات. لو كانوا تلاتة منفصلين، أي تعديل في التصميم كان لازم يتعمل تلات مرات.

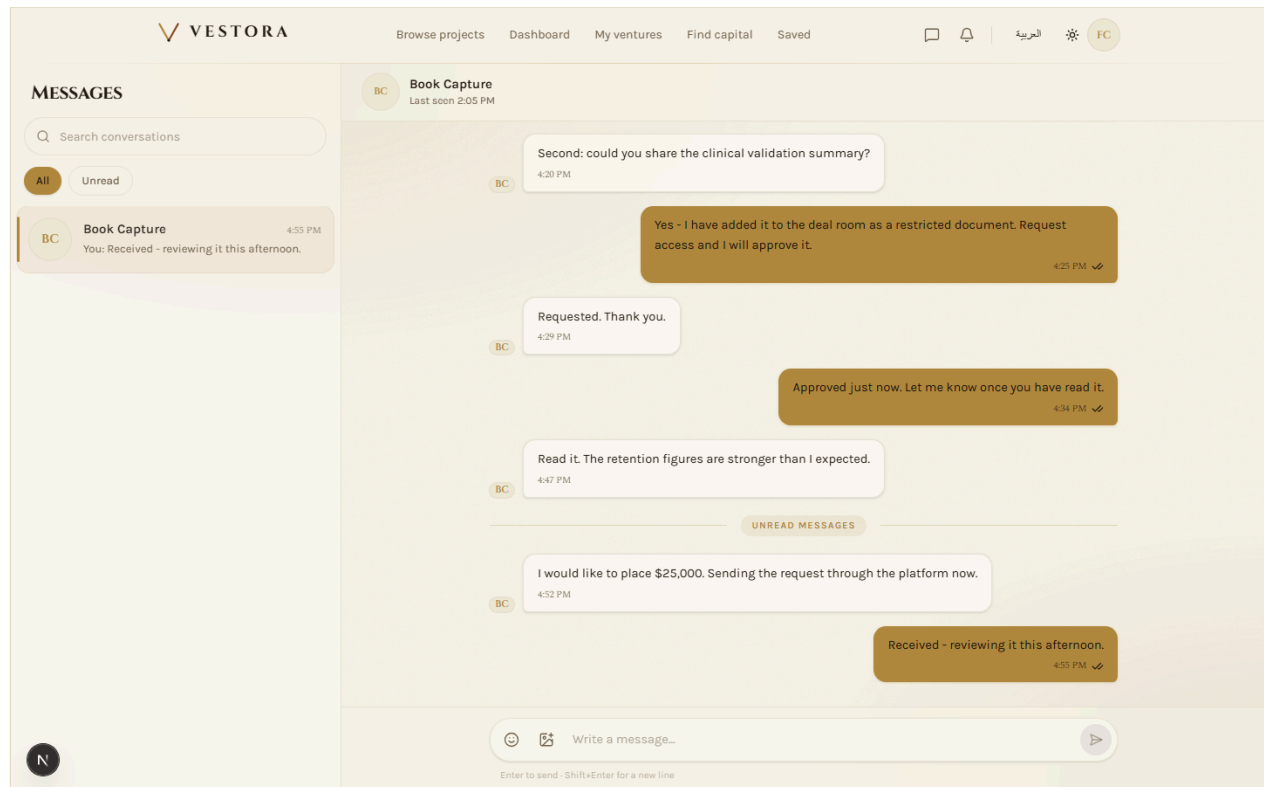
## 13. Messaging, Notifications, and Real-Time

سطر 676 بين الرسائل والإشعارات، وفوقهم طبقة SignalR كاملة.

### 13.1. The Pieces

| المكوّن                                    | دوره                                      |
|--------------------------------------------|-------------------------------------------|
| <code>MessagesController</code> (358)      | المحادثات والمرفقات وعلامات القراءة.      |
| <code>NotificationController</code> (318)  | قائمة الإشعارات وتعليمها كمقروءة.         |
| <code>ChatHub</code>                       | قناة SignalR — الرسائل والإشعارات لحظيًا. |
| <code>PresenceTracker</code> (57)          | مين متصل دلوقتى.                          |
| <code>NotificationFanOutQueue</code> (34)  | طابور في الذاكرة.                         |
| <code>NotificationFanOutWorker</code> (61) | خدمة خلفية بتفرّغ الطابور.                |
| <code>NotificationPush</code> (29)         | شكل موحد للدفع عبر SignalR.               |

جدول 13 سيع قطع بتشتغل مع بعض.



شكل 13 محادثة حقيقية بين مؤسس ومستثمر — فقاعات وأوقات وعلامات قراءة.

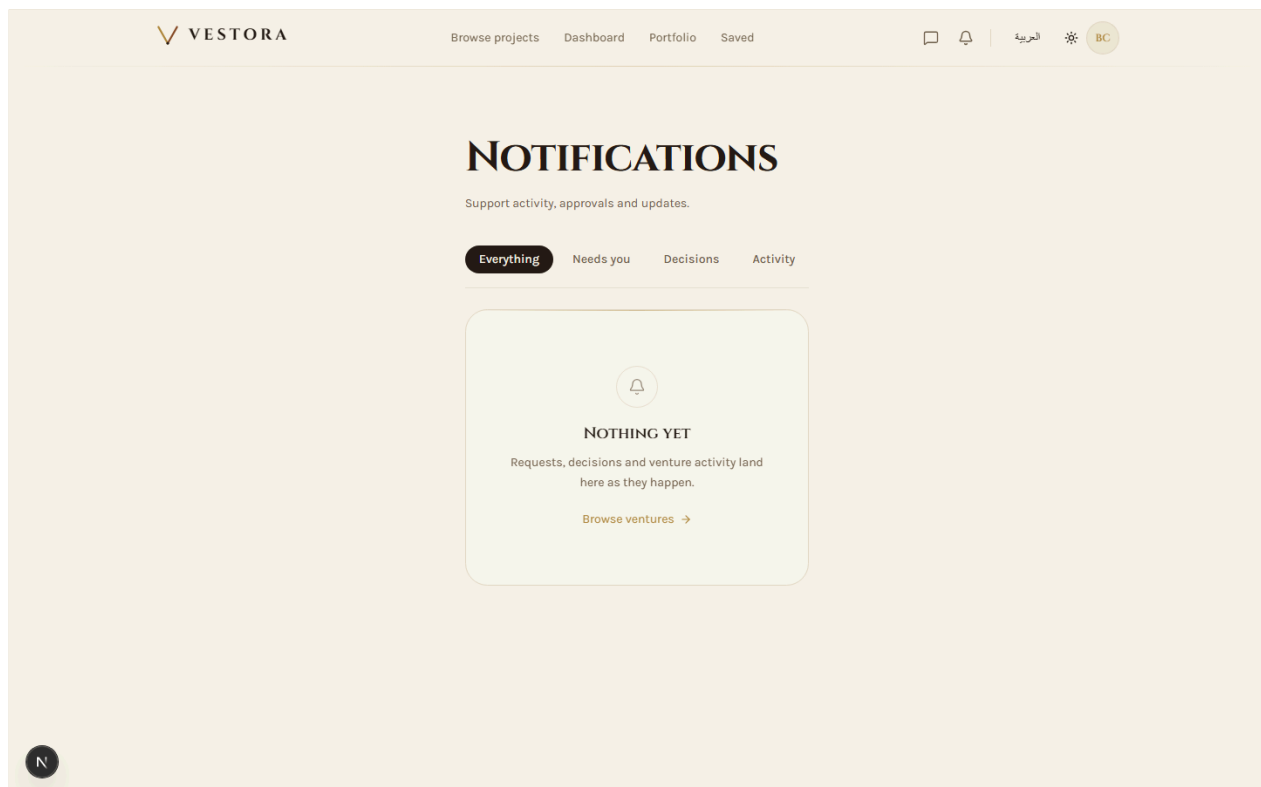
## 13.2. Presence Is Scoped, Not Broadcast

اعرف ده كويس

تفصيلة خصوصية ذكية في ChatHub :

- ```
1 // Distinct users this user has a conversation with – the audience for
2 // presence changes (so only people you actually talk to see your dot move).
```

يعني حالة «متصل الآن» بتوصل بس للناس اللي فعلاً بينك وبينهم محادثة — مش لكل المستخدمين. من غير ده، أي حد كان هيقدر يراقب إمتى أي حد تاني بيدخل ويخرج.



شكل 14 الإشعارات بتبويباتها: الكل · يحتاج قرارك · القرارات · النشاط.

اعرف ده كويس

التبويب الثاني اسمه «يحتاج قرارك» مش «غير مقروء» — وده قرار مقصود.

في منتج قايم على قرارات بين طرفين (المؤسس يوافق، المستثمر يلتزم، الأدمن يراجع)، الأولوية للي محتاج فعل منك، مش للي محتاج قراءة. نفس المبدأ اللي في شريط الأدمن الجانبي.

### 13.3. Why Notifications Go Through a Queue

#### اعرف ده كويس

لما المؤسّس ينشر تحديث، الإشعار بيروح لكل المستثمرين المعتمدين + المتابعين  
 1. اللي حافطين المشروع. لو ده حصل **داخل** الطلب، المؤسّس كان هيسنتّي لحد ما  
 كل إشعار يتحفظ ويتبعث — والوقت بيزيد مع كل مستلم.  
 التعليق على الـ worker بيشرح التصميم: «كل عنصر بياخد **DbContext** خاص بيه (الطلب اللي حطّه في  
 الطابور خلص من زمان)، وبيتحفظ ويتدفع عبر **SignalR** بشكل مستقل، عشان فشل واحد ما يسقطش  
 الباقي.»  
 ودي نقطة قوية: **العزل مقصود** — إشعار واحد يفشل ما يمنعش التسعة والتسعين الباقيين.

## 14. The Investment Flow — End to End

الفصل ده بيمشي في **رحلة استثمار كاملة** من أولها لآخرها. لو الدكتور قال «اشرحلي إزاي المنتج بيشغل»،  
 ده اللي تمشي عليه.

| الخطوة      | اللي بيحصل                                                  | الحالة                             |
|-------------|-------------------------------------------------------------|------------------------------------|
| 1. النشر    | المؤسّس ينشر مشروع. يبدأ <b>مخفي</b> لحد ما الأدمن يراجع.   | PendingReview                      |
| 2. المراجعة | الأدمن بيوافق (أو يرفض بسبب مكتوب).                         | Approved                           |
| 3. الاكتشاف | المشروع بيظهر في التصفّح للجمهور.                           | —                                  |
| 4. الاهتمام | المستثمر بيطلب يدعم. <b>مجرد اهتمام.</b>                    | Stage: New · Status: Pending       |
| 5. الموافقة | المؤسّس بيوافق. دلوقتي بس بيتحسب <b>ملتزم.</b>              | Stage: Approved · Status: Approved |
| 6. التفاوض  | غرفة الصفقة: أسئلة، مستندات، واقتراح شروط.                  | Stage: InDiscussion                |
| 7. الاتفاق  | الطرفين بيقبلوا ورقة الشروط.                                | TermSheet: Accepted                |
| 8. الطلب    | المؤسّس بيطلب الفلوس (أو أول دفعة منها).                    | FundingRequest: Open               |
| 9. الدفع    | المستثمر بيفتح checkout ويدفع في الصندوق التجريبي.          | Transaction: Initiated             |
| 10. التسوية | السيرفر بيسأل المزود ويطبّق النتيجة. <b>دلوقتي بس ممول.</b> | Stage: Committed · Succeeded       |
| 11. الإقفال | المؤسّس بيقل الجولة ويعلن النتيجة.                          | LifecycleStatus: Closed            |

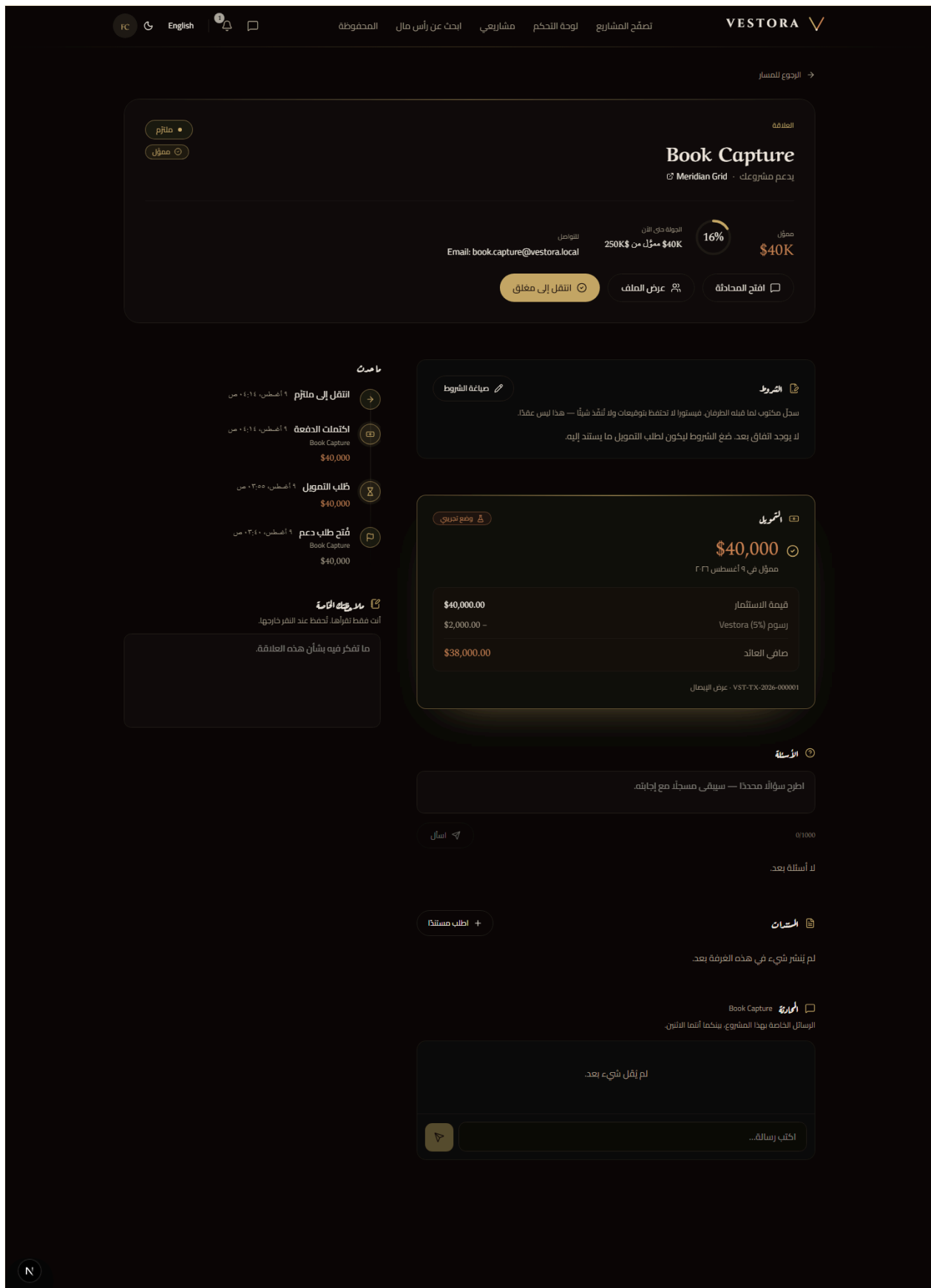
**جدول 14** إحدى عشر خطوة — من فكرة لفلوس وصلت.

#### اعرف ده كويس

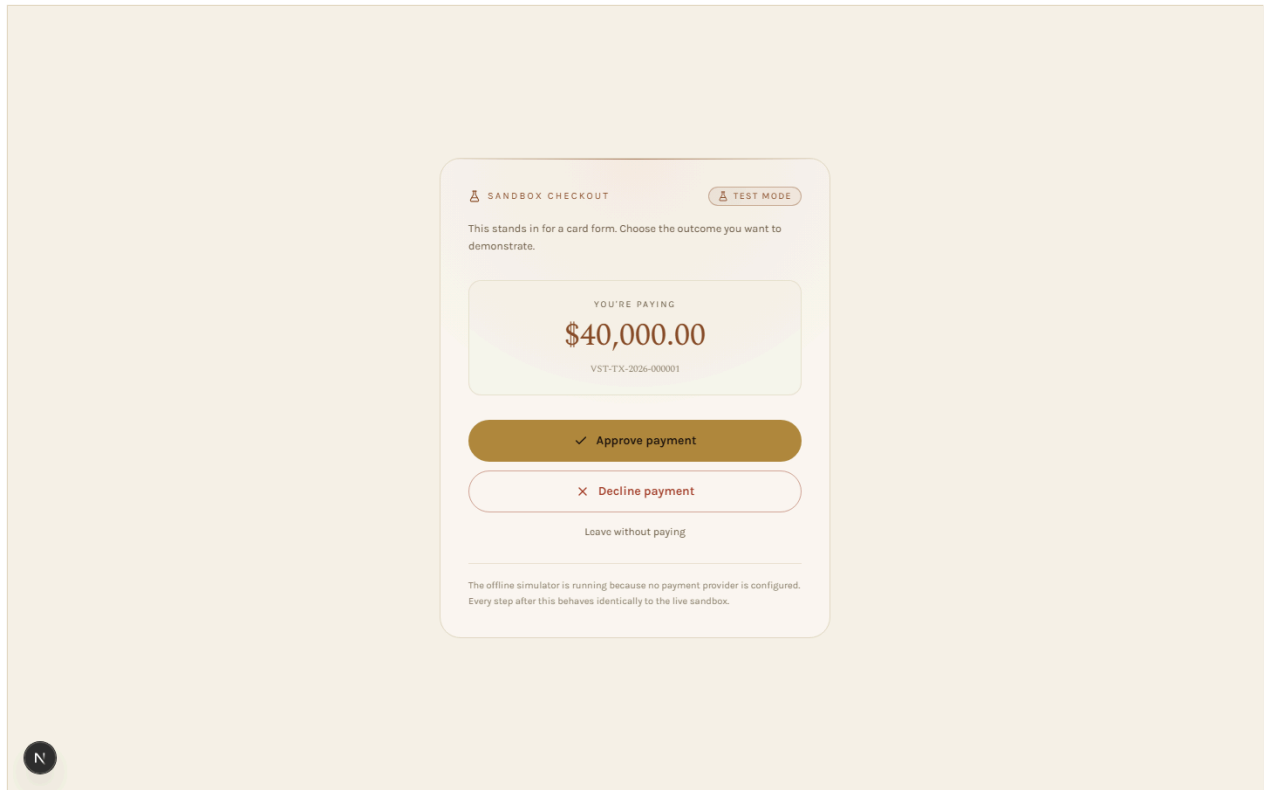
**أهم نقطتين في الرحلة دي تعرف تجاوب عنهم:**

- بين خطوة 5 و10 المشروع بيقول «ملتزم» **مش** «ممول». الفلوس ماوصلتش لسه.
- خطوة 10 هي الوحيدة اللي بتخلّي الرقم «ممول» — ولازم تعدي على **PaymentEvent** الأول.

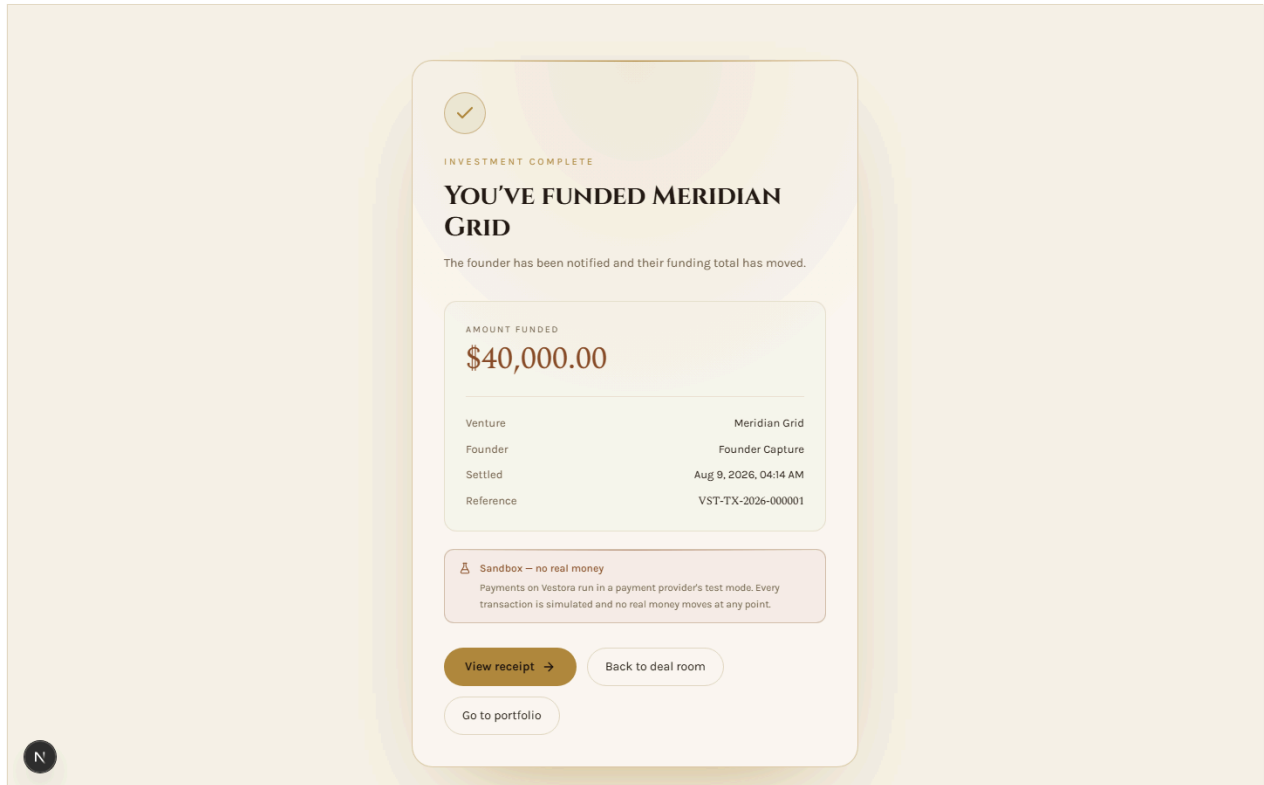




**شكل 15** غرفة الصفقة (خطوة 6-7): الأسئلة والشروط والمستندات والمحادثة في شاشة واحدة.



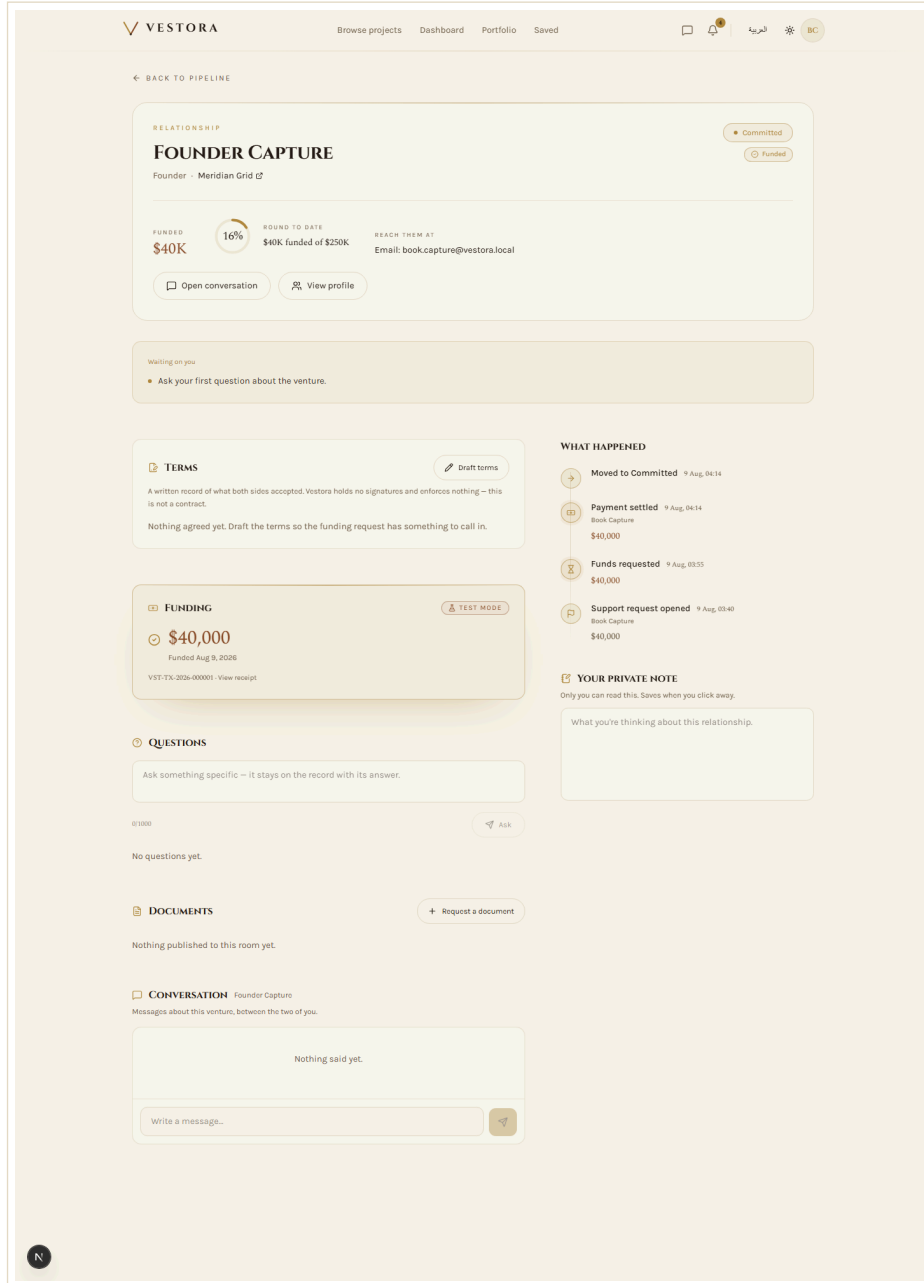
شكل 16 الدفع (خطوة 9): شارة الصندوق التجريبي واضحة وكبيرة.



شكل 17 التسوية (خطوة 10): تأكيد بمرجع فريد، والتنبؤ الصندوقي فاضل بعد النجاح.

## 15. One Deal Room, Read End to End

الصورة دي بتلخص المشروع كله في شاشة واحدة — تستاهل تتقرا سطر سطر.



شكل 18 غرفة الصفقة من عين المستثمر بعد ما الفلوس وصلت: الشارتين، نسبة الجولة، الخط الزمني، والملاحظة الخاصة.

### 15.1. What This One Screen Proves

بيثبت إيه

العنصر في الصورة

المحورين المنفصلين — العلاقة ملتزمة والفلوس وصلت.  
مش نفس الحاجة، والأتنين معروضين.

شارتين: **Funded** + **Committed**

| العنصر في الصورة                                   | بيّث إليه                                                                |
|----------------------------------------------------|--------------------------------------------------------------------------|
| 16% · \$40K funded of \$250K                       | نسبة الجولة محسوبة من FundingMath — ممول على مطلوب، مش ملتزم على مطلوب.  |
| VST-TX-2026-000001                                 | مرجع بشري لكل معاملة — يتقال في التليفون.                                |
| TEST MODE                                          | الصراحة الصندوقية حتى جوه غرفة الصفقة.                                   |
| «Vestora holds no signatures and enforces nothing» | قاعدة رقم 8 مكتوبة في الواجهة نفسها — المنصة مبتدعيش إنها عقد.           |
| الملاحظة الخاصة                                    | «Only you can read this» — كل طرف عنده ملاحظاته، والتاني عمره ما يشوفها. |
| جدول 15 ست حاجات في شاشة واحدة.                    |                                                                          |

## 15.2. The Timeline Is the Audit Trail

| اعرف ده كويس                                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| قسم What happened على اليمين هو InvestmentStageEvent معروض بصريًا:                                                                                                                                                                                    |
| <ul style="list-style-type: none"> <li>Support request opened — \$40,000 (Aug, 03:40 9)</li> <li>Funds requested — \$40,000 (Aug, 03:55 9)</li> <li>Payment settled — \$40,000 (Aug, 04:14 9)</li> <li>Moved to Committed — (Aug, 04:14 9)</li> </ul> |
| لاحظ إن آخر اثنين في نفس الدقيقة — لأن SettleAsync بيحرك المرحلة جوه نفس المعاملة اللي بتسجل الدفع.                                                                                                                                                   |
| والتعليق في الكود بيوضح تفصيلة دقيقة: «المستثمر هو الفاعل: هو اللي عمل ده، وتاريخ ينسب للمؤسس لحظة وصول الغلوس كان هيوصف فعل الشخص الغلط.»                                                                                                            |

| خد بالك                                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| الخط الزمني ده مش مكتوب بالإيد في الكنترولر — هو ناتج StageLog.MoveAsync اللي هي الطريقة الوحيدة المسموحة لتحريك المرحلة. عشان كده مستحيل يكون فيه انتقال حصل من غير ما يظهر هنا. |

## 16. The Admin Surface — Six Controllers

الأدمن عندك 2093 سطر موزعين على ستّة كترولرز، كل واحد بسبب.

| الكترولر                    | الأسطر | مسؤول عن                                            |
|-----------------------------|--------|-----------------------------------------------------|
| AdminController             | 720    | المستخدمين، الأدوار، الإنشاء والتعليق، والbootstrap |
| AdminRevenueController      | 434    | الإيرادات والرسوم والتسوية المالية                  |
| AdminUserOverviewController | 373    | حساب واحد وكل اللي متعلق بيه — جديد                 |
| AdminModerationController   | 238    | طابور المراجعة: الموافقة والرفض بسبب                |
| AdminInsightsController     | 187    | التحليلات التاريخية: فين العلاقات بتقف — جديد       |

## الكنترولر

## الأسطر

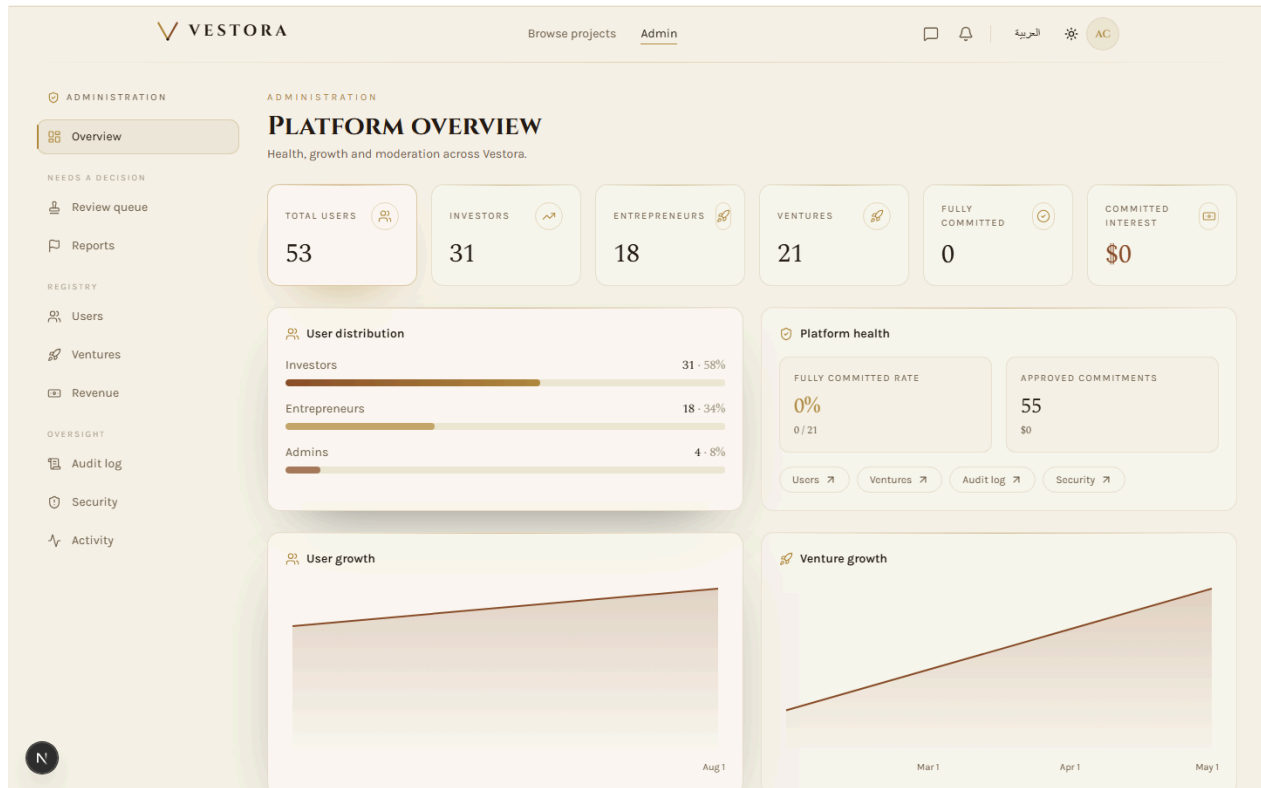
## مسؤول عن

AdminSearchController

141

بحث شامل عبر كل الكيانات — جديد

جدول 16 ستّة كنترولرز — ثلاثة منهم اتضافوا في آخر تحديث.



شكل 19 لوحة الأدمن الرئيسية: عدّادات وتوزيعات ورسوم بيانية بألوان هوية المنتج.

## 16.1. Why Six Files and Not One

اعرف ده كويس

التعليق في AdminUserOverviewController يشرح المبدأ:

«متساب بره AdminController عن قصد: الكنترولر ده بيدير قائمة المستخدمين والأفعال اللي بتعمل عليها، وده بيقرا عبر نص المخطط عشان يجاوب سؤال مختلف. نفس التقسيمة بتاعة المراجعة والإيرادات.»

يعني التقسيم بالسؤال اللي بيتجاوب عليه، مش بالكيان.

## 16.2. The Primary Admin

خد بالك

مش كل أدمن يقدر يعمل أدمن جديد — الأدمن الأساسي بس.

التعليق يشرح: «أي أدمن يقدر يعمل أدمن ثاني كان معناه إن عضوية «مين يقدر يعمل أي حاجة على المنصة دي» بيقرّرها أول واحد يدوس.»

والعلم ده في البيانات مش في الكود: «إيميل مكتوب في الكود بيتكسر أول ما العنوان ده يتغيّر أو الحساب بتاعه يحصله مشكلة، ومينفعش يتسلّم لحد ثاني من غير نشر جديد.»

اعرف ده كويس

## الـ Bootstrap — إزاي أول أدمن بيتعمل أصلاً؟

```
1 [HttpPost("bootstrap")]
2 [AllowAnonymous]
```

نقطة مفتوحة، بس محمية بتلات طبقات: **مفتاح سرّي** من الإعدادات، **rate limiting**، وأهم حاجة — بتتقفل للأبد أول ما أي أدمن يوجد:

```
1 if (await _context.Users.AnyAsync(u => u.UserType == "Admin"))
2     return Conflict("An administrator already exists. Bootstrap is no longer available.");
```

ودي إجابة سؤال «إزاي أول أدمن بيتعمل من غير أدمن موجود؟» — سؤال ذكي ممكن يتسأل.

## 16.3. Insights Return Parts, Not Percentages

اعرف ده كويس

تفصيلة إحصائية مهمة من `AdminInsightsController`:

«كل نسبة هنا بترجع كبسط ومقام. عند خمسين حساب، رفض واحد بيحرك معدل الموافقة كذا نقطة، والنسبة المجردة بتشيل دقة الأعداد اللي وراها ماتقدرش تتحملها — فالأجزاء بتتبع والواجهة بتبدأ بيها.»

يعني بدل «معدل الموافقة 67%» بتقول «12 من 18». وده نفس مبدأ «مراجعة لا موثقة» — ماتدعّيش دقة مش موجودة.

## 17. Backend Infrastructure

`Program.cs` (400 سطر) وكل اللي بيخلّي الباكند يشتغل أصلاً.

## 17.1. The Stripe Guard — Rule 10

خد بالك

التطبيق **بيرفض يشتغل** لو المفتاح مش تجريبي:

```
1 if (!string.IsNullOrWhiteSpace(stripeKey) && !stripeKey.StartsWith("sk_test_"))
2 {
3     throw new InvalidOperationException(
4         "Payments:Stripe:SecretKey must be a test key (sk_test_...). " +
5         "Vestora must never process real payments.");
6 }
```

التعليق بيشرح: «مفتاح Stripe حقيقي بيفرض تمامًا. فيستورا مشروع تخرج مالوش رخصة يحرك فلوس، وأمن طريقة نضمن بيها إنه ماحصلش هي إننا نخلّي التطبيق **مش قادر يبدأ** لو حد جرب.»

دي قاعدة رقم 10 في `AGENTS.md`، ولو الدكتور سأل عن الأمان المالي دي أقوى إجابة: **الحماية معمارية، مش اعتماد على إن حد يفتكر.**

## 17.2. Two Providers, One Interface

اعرف ده كويس

`IPaymentProvider` وراه تنفيذين: `SimulatedPaymentProvider` و `StripeSandboxPaymentProvider`. الاختيار بيحصل في `Program.cs` حسب الإعدادات. التعليق بيشرح ليه المحاكى موجود: «دورة التمويل كلها بتشتغل — نفس آلة الحالة، نفس الرسوم، نفس سجلّ المراجعة — فالعرض عمره ما يبقى تحت رحمة الشبكة.» ودي نقطة قوية في المناقشة: **المحاكي مش stub بيرجع نجاح — هو بيمشي على نفس الحالات بالطبط.**

## 17.3. Rate Limiting

| السياسة       | الحد         | بتحمي إيه                          |
|---------------|--------------|------------------------------------|
| Global        | 100 / دقيقة  | كل الطلبات                         |
| Auth          | 10 / دقيقة   | الدخول والتسجيل                    |
| PasswordReset | 3 / 10 دقائق | إعادة تعيين كلمة السر والbootstrap |
| Checkout      | 8 / 5 دقائق  | فتح جلسات دفع                      |

**جدول 17** أربع سياسات، كل واحدة بحد مناسب لخطورتها.

اعرف ده كويس

التعليق على سياسة الCheckout: «فتح checkout هو الفعل الوحيد في الدفع اللي ليه تكلفة عند المزود. كريم كفاية لإعادة محاولة حقيقية، وضيق كفاية إن حلقة ماتقدرش تقصف Stripe بجلسات.»

## 17.4. Cross-Cutting Services

| الخدمة                                 | الدور                                                              |
|----------------------------------------|--------------------------------------------------------------------|
| <code>ServiceResult&lt;T&gt;</code>    | شكل موحد لرد أي خدمة — نجاح، أو نوع الفشل بالطبط.                  |
| <code>UtcDateTimeConverter</code>      | كل تاريخ بيتحوّل لـ UTC عند التسلسل — قاعدة رقم 6.                 |
| <code>StageLog</code>                  | الطريقة <b>الوحيدة</b> المسموحة لتغيير مرحلة العلاقة، بسجلّ تاريخ. |
| <code>PipelineStages</code>            | مفردات المراحل — مصدر واحد بدل نصوص متناثرة.                       |
| <code>TrustSignals</code>              | إشارات المصادقية من حقائق قابلة للإثبات بس.                        |
| <code>FileUploadSecurityService</code> | فحص نوع الملف وحجمه ومحتواه قبل الحفظ.                             |
| <code>PaymentExpirySweeper</code>      | مهمة خلفية بتقفل طلبات التمويل المنتهية.                           |

**جدول 18** سبع خدمات بتخدم كل المشروع.

اعرف ده كويس

**StageLog** مثال ممتاز على قرار معماري:

التعليق بيقول: «قبل ده، سبع أماكن كانت بتحدّد `Stage` و `StageUpdatedAt` بالإيد — كمنترولرين، خدمة الدفع، ومسار إفعال الجولة. كانوا متفقين لأن اللي كتبهم فاكر يحطّ الاتنين. وإضافة صف تاريخ لكل واحد منهم بالإيد كانت هتبقى نفس الترتيب بحاجة زيادة تنسى، وأول موقع ينسى كان هينتج خط زمني كذاب.» نفس منطق `FundingMath` بالطبط: **لو حاجة لازم تحصل مع حاجة تانية دايمًا، اعملهم في مكان واحد.**

Services/Payments/PaymentService.cs  
 PaymentService.ApplyProviderResultAsync  
 PaymentService.SettleAsync  
 Services/FundingMath.cs  
 Services/TermSheetService.cs  
 Controllers/DealRoomController.cs  
 Controllers/AdminController.cs  
 Program.cs

فمين ده في الكود  
 قلب الدفع — 1259 سطر  
 الحدث قبل الأثر + عدم التكرار  
 التسوية وتجميد الرسوم  
 كل رقم مالي — بما فيه دوال الدفعات  
 إقتراح وقبول واستبدال الشروط  
 أكبر كنترولر — 1287 سطر  
 المستخدمين والأدوار والbootstrap  
 البنية التحتية وحارس مفتاح Stripe

اختبر نفسك

1. ليه الحدث بيتسجل قبل الأثر؟  
 عشان يبقى فيه سجل دائم لكل تأكيد من المزود، والفهرس الفريد عليه هو اللي بيمنع تطبيقه مرتين.
2. ليه المعاملة (transaction) ضرورية هناك؟  
 لو العملية ماتت بين حجز المفتاح وتطبيق الأثر، المفتاح هيعيش والتسوية لأ — وإعادة المحاولة نفسها هي اللي المفتاح بيمنعها. الدفعة كانت هتضيع.
3. إيه اللي بيمنع تحصيل التزام مرتين بنفسه؟  
 طرح المدفوع من مساحة الجولة: `headroom - settled` — من غيره المؤسس يقدر يقسم التزام 50 ألف ويطلب 100.
4. إزاي أول أدمن بيتعمل؟  
 نقطة bootstrap محمية بمفتاح سرّي وrate limiting، وبتتقفل للأبد أول ما أي أدمن يوجد.
5. ليه التطبيق بيرفض مفتاح Stripe حقيقي؟  
 عشان المشروع مالوش رخصة يحرك فلوس — والحماية معمارية (التطبيق مش بيبدأ) مش اعتماد على إن حد يفتكر.

## 18. The Frontend Foundation You Built

الفرونت فيه طبقتين: **البنية المشتركة** اللي إنت بنيتها والكل بيبنى فوقها، ثم **الصفحات** اللي كل عضو بناها. الفصل ده عن الأولى.

### 18.1. The Transport Layer — One Door

اعرف ده كويس

`lib/api/client.ts` (150 سطر) هو **الناقل الوحيد** في المشروع كله. قاعدة رقم 11 في `AGENTS.md`: **ممنوع** أي `fetch` بره `lib/api/`.

ليه؟ لأن كل طلب محتاج نفس الحاجات: التوكن، تجديده لو انتهى، تحويل الأخطاء لشكل موحد، ومعرفة إمتى بيعت JSON وإمتى `FormData`. لو كل صفحة عملت `fetch` بنفسها، كل واحدة كانت هتنسى حاجة مختلفة.



اعرف ده كويس

## أذكرى تفصيلة في الملف — single-flight refresh:

```
1 // Single-flight refresh: concurrent 401s share one refresh request.
2 let refreshPromise: Promise<string> | null = null;
```

لو خمس طلبات رجعوا 401 في نفس اللحظة (وده بيحصل كتير — صفحة بتحمل كذا حاجة مع بعض)، من غير السطر ده كانوا هيبعتوا **خمس** طلبات تجديد. وأول واحد بينجح بيدور التوكن، والأربعة الباقين بيتبعثوا بتوكن قديم بقي ملغي — وساعتها **كشف السرقة** بتاع عضو الباكند بيتفعل ويطرد المستخدم.

السطر ده بيخلي الخمسة يستنوا نفس الوعد. بدون فهم إن الباكند بيعمل تدوير، الباغ ده كان هيبقى مستحيل تشخيصه.

## 18.2. The Type Contract

اعرف ده كويس

lib/types/api.ts (1736 سطر) — كل شكل بيانات بين الفرونت والباكند.

ده **عقد** مش مجرد أنواع: لو الباكند غير شكل رد، الفرونت بيوقع في وقت الترجمة (compile time) مش وقت التشغيل قدام المستخدم. وقاعدة في AGENTS.md: أي DTO بيتغير في الباكند لازم يتحدّث هنا.

## 18.3. The Design System in Code

اعرف ده كويس

app/globals.css (775 سطر · 129 توكن) — التصميم بتاع عضو الـ UI/UX اتحوّل لكود هنا.

التقسيم: الخطوط · الوضع الفاتح (ورق) · الوضع الداكن (إسبريسو) · الأرقام المالية · المؤشّر · الأزرار · الروابط · أشرطة التقدّم · الزجاج السائل.

خد بالك

**تفصيلة تصميمية دقيقة موثقة في الكود:** الذهب في الوضع الفاتح ليه توكنين مختلفين — واحد للتعبئة وواحد للنص — لأن الذهب على الورق البيج بقيس 2.86:1 في التباين والمعيار 4.5:1.

التعليق في الكود بيقول: «الذهب على الإسبريسو أصلاً 8.81:1 — الحبر والتعبئة نفس المعدن.» يعني في الوضع الداكن التوكنين بيرجعوا واحد.

ودي نقطة قوية لو الدكتور سأل عن الوصولية (accessibility).

## 18.4. Motion and 3D — Nine Components

| المكوّن     | بيعمل إيه                                            |
|-------------|------------------------------------------------------|
| depth-scene | مشهد ثلاثي الأبعاد حقيقي بالـ compositor — مش WebGL. |
| tilt        | إمالة العنصر مع حركة الماوس.                         |
| magnetic    | العنصر بينجذب ناحية المؤشّر.                         |
| cursor      | مؤشّر مخصّص للصفحات الرئيسية.                        |
| reveal      | ظهور تدريجي عند التمرير.                             |

| المكوّن                                                        | يعمل إيه                          |
|----------------------------------------------------------------|-----------------------------------|
| masked-lines                                                   | نص بيتكشف سطر سطر.                |
| animated-number                                                | رقم بيعدّ لقيّمته.                |
| brand-loader                                                   | شاشة تحميل بهوية المنتج.          |
| lazy-video                                                     | فيديو بيتحمّل لما يقرب من الشاشة. |
| جدول 19 تسع مكوّنات حركة — كلها بتحترم prefers-reduced-motion. |                                   |

## اعرف ده كويس

**أدكى مكوّن في الغرونت كله، والتعليق بيشرح القرار:** depth-scene

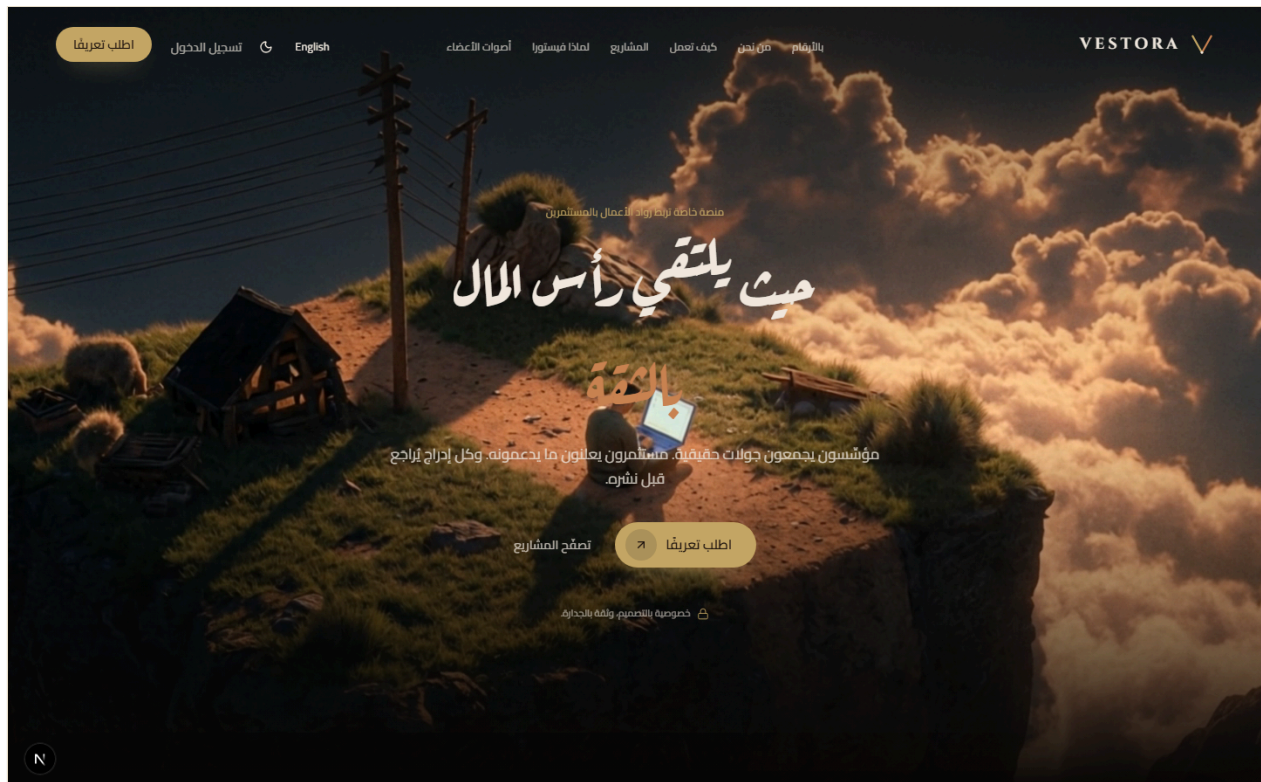
«ثلاثي أبعاد حقيقي، معمول بال-compositor بدل مُصَيّر (renderer). الطبقات متباعدة فعليًا في محور Z، فال-parallax بينهم المتصفح بيحسبه — الطبقات القريبة بتتحرك أكثر من البعيدة — بدل ما يتزوّر بإزاحات لكل طبقة بتختلف عن بعضها دايمًا.»

والتكلفة: «تحويل على كام عنصر مركّب: مفيش سياق WebGL، مفيش هندسة، مفيش shaders، مفيش أي حاجة بتتضاف للحزمة. وده بيغرق في الصفحتين اللي في عمق المسار، حيث مكتبة 3D كانت هتبقى أثقل حاجة في الصفحة وأول حاجة تلغثم على موبايل متوسط.»

دي إجابة ممتازة لو الدكتور سأل «ليه ما استخدمتمش Three.js؟»

## 19. The Landing Page and Onboarding

### 19.1. The Landing — Twelve Components

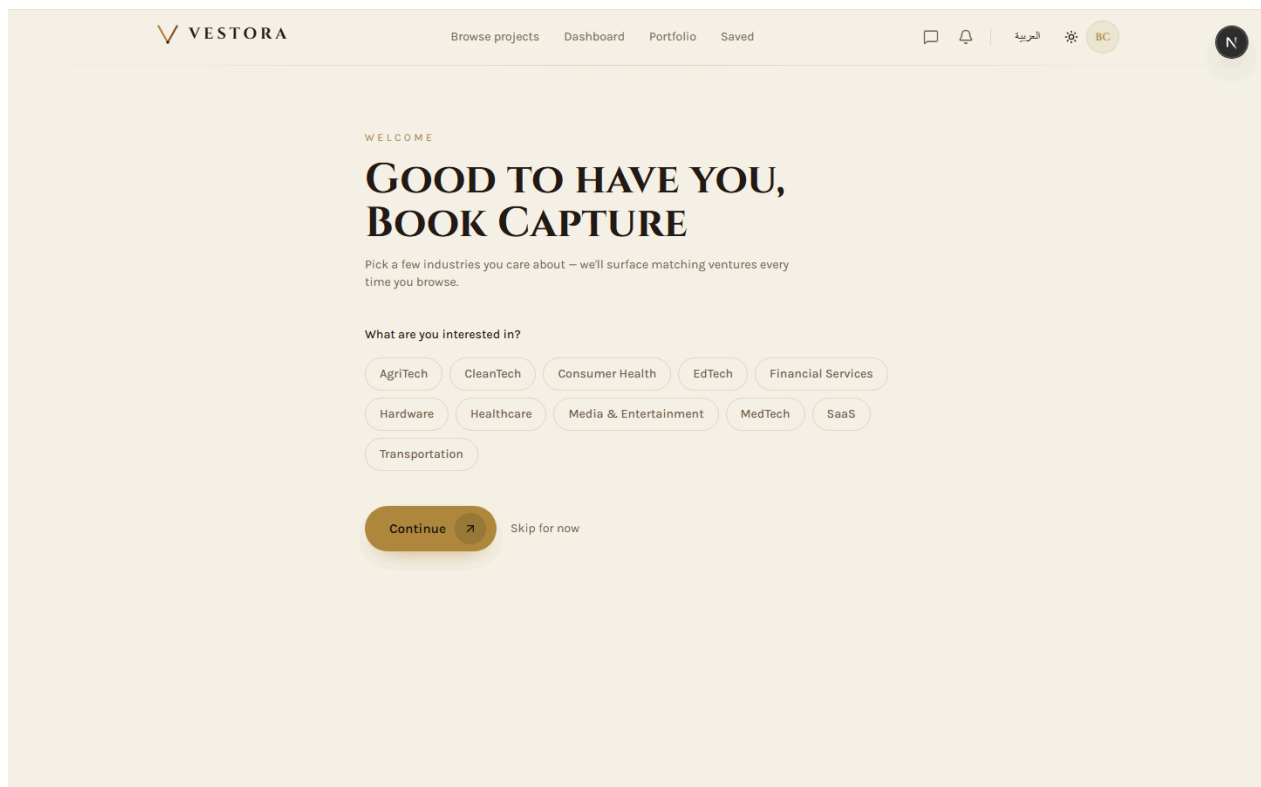


**شكل 20** صفحة الهبوط بالعربي والوضع الداكن — أول انطباع عن المنتج.

| المكوّن                                         | دوره                                            |
|-------------------------------------------------|-------------------------------------------------|
| landing-hero                                    | الهيرو (281 سطر) — الصورة والعنوان ونداء الفعل. |
| landing-nav                                     | التنقل العلوي.                                  |
| landing-splash                                  | شاشة الافتتاح.                                  |
| why-vestora                                     | ليه المنتج ده موجود.                            |
| how-it-works                                    | الخطوات الثلاث.                                 |
| featured-projects                               | مشاريع مختارة.                                  |
| stats-section                                   | الأرقام.                                        |
| testimonials-section                            | الشهادات.                                       |
| about-section                                   | عن المنصة.                                      |
| card-ground · section-surface · section-heading | مكوّنات تخطيط مشتركة.                           |

**جدول 20** اتناشر مكوّن في صفحة الهبوط.

## 19.2. Onboarding — 590 Lines



**شكل 21** شاشة أول تشغيل: اختيار القطاعات كحبيبات قابلة للاختيار المتعدّد.

**اعرف ده كويس**

الأونبوردنج بيشتغل **مرة واحدة بس** — والباكند بيحفظ `OnboardedAtUtc` ، والدخول بيرجّع `hasOnboarded` عشان الفرونت يعرف بوجه فين.

التفصيلية الذكية في الباكند: النقطة **idempotent** وبتحتفظ بأول توقيت مش آخر واحد — «إعادة تشغيل الفلو (أو طلب مكرّر من ضغطة مزدوجة) ماينفعش تعيد كتابة إمتى الحساب ده اتعمله أونبوردنج فعلاً.»

## 20. The Admin Interface — Thirteen Pages

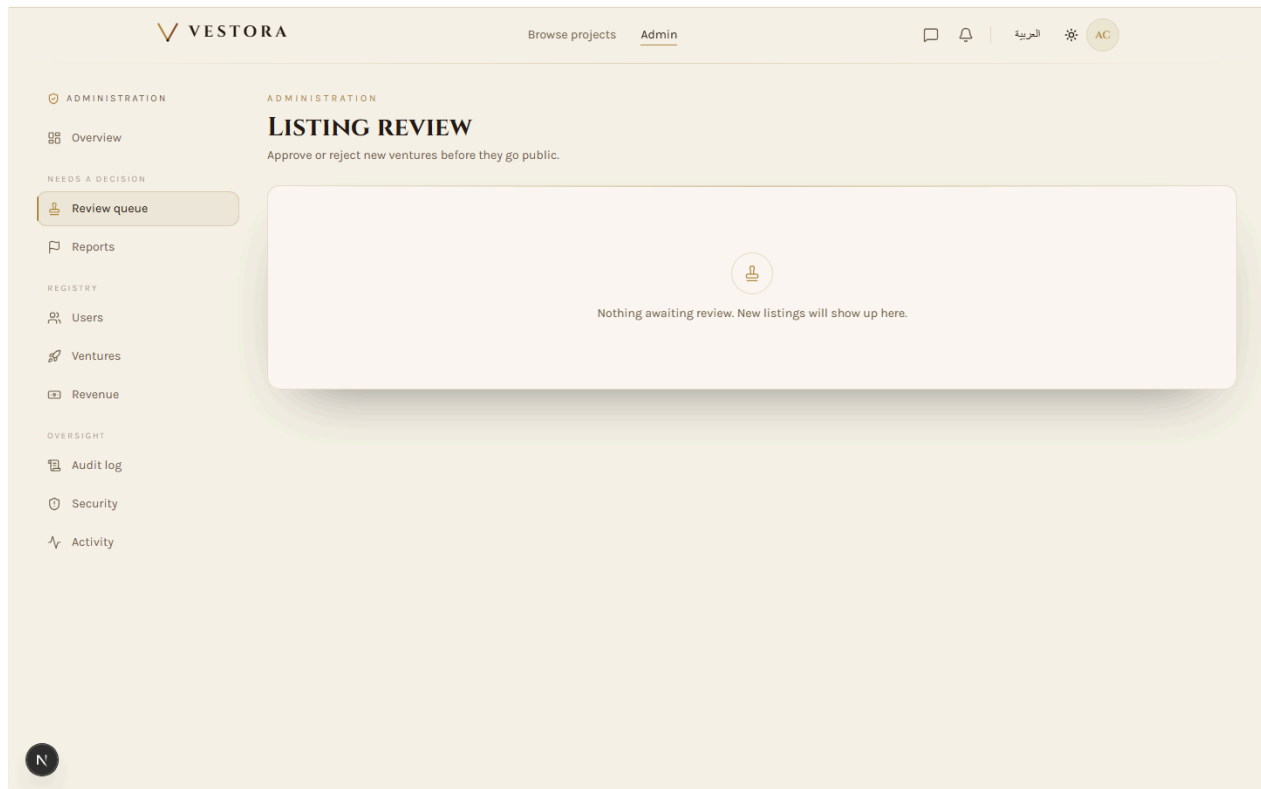
كل شاشة أدمن في المنتج إنت بنيتها.

| الوظيفة                                     | الصفحة                            |
|---------------------------------------------|-----------------------------------|
| النظرة العامة — العدّادات والرسوم البيانية. | <code>admin/page.tsx</code>       |
| قائمة المستخدمين بالبحث والفلتر.            | <code>admin/users</code>          |
| نظرة شاملة على حساب واحد.                   | <code>admin/users/[id]</code>     |
| كل المشاريع.                                | <code>admin/ventures</code>       |
| طابور المراجعة — الموافقة والرفض.           | <code>admin/review</code>         |
| البلاغات.                                   | <code>admin/reports</code>        |
| الإيرادات والرسوم.                          | <code>admin/revenue</code>        |
| التسوية المالية.                            | <code>admin/reconciliation</code> |
| التحليلات التاريخية.                        | <code>admin/analytics</code>      |
| سجلّ المراجعة.                              | <code>admin/audit</code>          |
| الأحداث الأمنية.                            | <code>admin/security</code>       |
| النشاط العام.                               | <code>admin/activity</code>       |
| التخطيط المشترك والشريط الجانبي.            | <code>admin/layout.tsx</code>     |

جدول 21 تلتاشر صفحة أدمن.

The screenshot displays the 'USERS' management interface in the VESTORA admin panel. The sidebar on the left contains navigation options under 'ADMINISTRATION', 'NEEDS A DECISION', 'REGISTRY', and 'OVERSIGHT'. The 'USERS' section is highlighted. The main area features a search bar, a 'New admin' button, and a list of users. Each user entry includes a profile picture, name, email, role (Entrepreneur or Investor), and icons for editing or deleting the user. The list includes users like Abdalwanes Alsayed, Omar Ali, Alsayed Ashref, Omer Ashref, Oday Alsayed, Abdo Gamal, and others.

شكل 22 إدارة المستخدمين: بحث وفلتر، وأدوار واضحة لكل حساب.

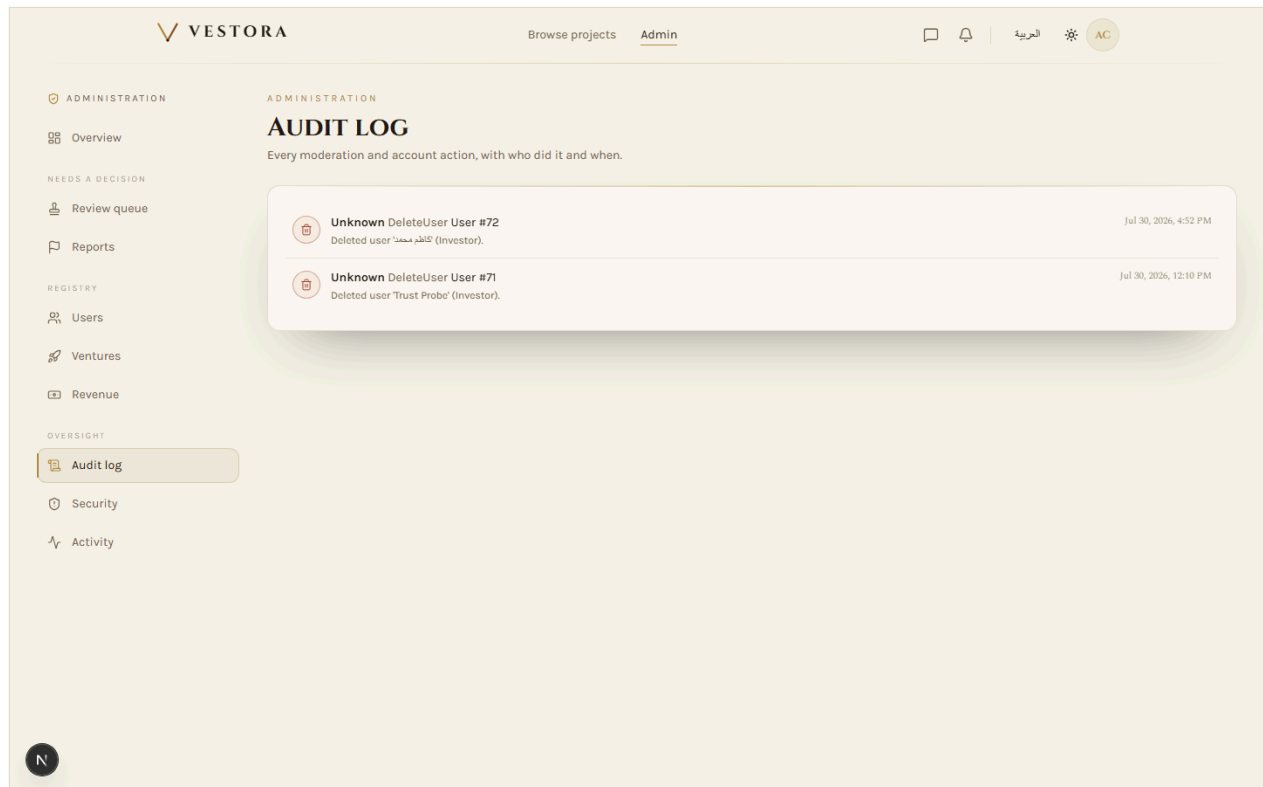


شكل 23 طابور المراجعة — بوابة النشر: مغيث مشروع يظهر للعامة قبل ما يعدّي من هنا.

#### اعرف ده كويس

**الرفض بسبب مكتوب** — مش مجرد زرار رفض.

السبب بيتخزن في `ModerationNote`، ويوصل للمؤسس، ويفضل موجود بعد الموافقة — عشان لو المؤسس عدّل ورجع للطابور، المراجع يشوف السبب الأصلي. والتعليق في الباكند بيحدّر: «ده السبب الداخلي للأدمن — عمره ما يسافر للعامة.»



شكل 24 سجلّ المراجعة: كل فعل إداري، بسببه والتغيير اللي حصل وعنوان الـ IP.

#### خد بالك

آخر تحديث في السجلّ ده صاف ثلاث حاجات لكل فعل إداري: **السبب** اللي الأدمن كتبه، و\*الفرق (before/after) **للي اتغير، و\*عنوان الـ IP** بتاع الأدمن. من غيرهم، السجلّ كان بيقول «الأدمن غيّر حاجة» — دلوقتي بيقول «مين، غيّر إيه بالضبط، من إيه لإيه، ليه، ومن فين».

#### فين ده في الكود

```
lib/api/client.ts
lib/types/api.ts
lib/i18n/dictionaries.ts
app/globals.css
components/motion/depth-scene.tsx
components/landing/
app/(app)/onboarding/page.tsx
app/(app)/admin/
```

الناقل الوحيد — التوكن والتجديد والأخطاء  
عقد الأنواع — 1736 سطر  
كل نص ظاهر بلغتين — 4896 سطر  
نظام التصميم — 129 توكن  
ثلاثي أبعاد بالـ compositors  
إتناشر مكوّن لصفحة الهبوط  
أول تشغيل — 590 سطر  
تلتاشر صفحة أدمن

اختبر نفسك

1. **ليه ممنوع fetch بره lib/api؟**  
كل طلب محتاج نفس الحاجات — التوكن وتجديده وتوحيد الأخطاء. لو كل صفحة عملت fetch، كل واحدة كانت هتنسى حاجة مختلفة.
2. **إيه الـ single-flight refresh وليه مهم؟**  
خمس طلبات بترجع 401 مع بعض بتشارك طلب تجديد واحد. من غيره، التوكنات القديمة كانت هتفعل كشف السرقة وتطرد المستخدم.
3. **ليه ثلاثي الأبعاد بالـ compositor مش Three.js؟**  
الطبقات متباعدة فعلياً في Z فالمتصفح بيحسب الـ parallax بنفسه — من غير WebGL ولا shaders ولا أي زيادة في حجم الحزمة.
4. **إيه اللي اتضاف لسجلّ المراجعة مؤخرًا؟**  
السبب المكتوب، والفرق before/after، وعنوان الـ IP — عشان السجلّ يقول مين وإيه وليه ومن فين مش «حد غير حاجة».

## 21. Bilingual and RTL — The Whole Product, Twice

**dictionaries.ts** عندك 4896 سطر — وده مش ملف ترجمة عادي، ده كل نص ظاهر في المنتج، مكتوب مرتين.

### 21.1. The Contract

اعرف ده كويس

قاعدة رقم 11 في **AGENTS.md**: ممنوع نص مكتوب في الكود — كل نص بمفتاح عن طريق **t()**.

```
1 const t = useCallback(
2   (key: string) => dictionaries[locale][key] ?? dictionaries.en[key] ?? key,
```

لاحظ التدرّج الثلاثي: اللغة الحالية ← الإنجليزي ← المفتاح نفسه. يعني مفتاح ترجمة ناقص بيظهر كنص المفتاح، مش شاشة فاصية أو **undefined**.

### 21.2. Direction Is a System Concern, Not a Page Concern

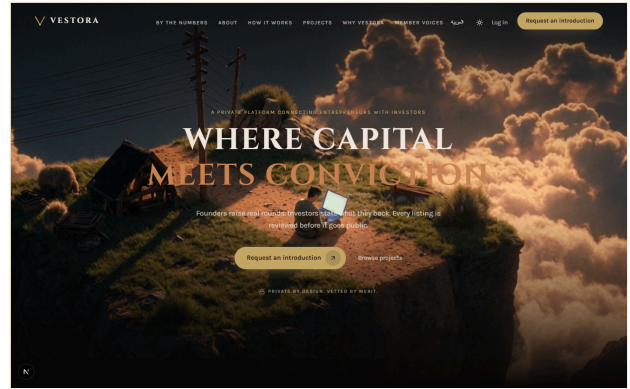
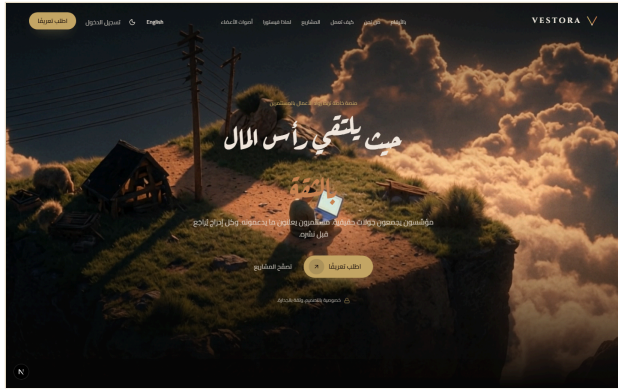
اعرف ده كويس

```
1 useEffect(() => {
2   document.documentElement.lang = locale;
3   document.documentElement.dir = dirFor(locale);
4 }, [locale]);
```

سطين بس، وهما اللي بيقبلوا المنتج كله. لأن الاتجاه متحدّد على **<html>** نفسه، كل CSS منطقي في المشروع (**ms-** بدل **ml-**، **start-** بدل **left-**) بينقلب لوحده.

ودي القاعدة الثالثة في رقم 11: **ممنوع اتجاه فيزيائي**. الالتزام بيها هو اللي خلّى دعم العربي شغل يوم واحد بدل إعادة كتابة كل صفحة.





شكل 25 نفس الصفحة بالظبط — إنجليزي فاتح، وعربي داكن. نفس المكوّنات، نفس الكود، اتجاه معكوس بالكامل.

## 21.3. The Honest-Copy Rule Lives Here Too

خد بالك

المفاتيح في `dictionaries.ts` هي اللي بتفرض قاعدة رقم 8 عملياً. مثال حقيقي من الملف:

«كل مبلغ على المنصة هو التزام معلن بين شخصين، متسجلّ عشان الطرفين يمسكوا نفس السجلّ... عشان كده مغيث حاجة هنا بتقول **raised** أو **funded**.»

النص ده مكتوب في الترجمة نفسها — يعني الالتزام الأخلاقي مش في الكود بس، هو في الكلام اللي المستخدم بيقرأه.

## 22. Authorization — The Model You Designed

نموذج الصلاحيات في المشروع كله قرارك إنت، وكل عضو بينقّده في مجاله.

### 22.1. Four Levels

| المستوى                    | المعنى والاستخدام                                                             |
|----------------------------|-------------------------------------------------------------------------------|
| [AllowAnonymous]           | مفتوح لكل — التصفح، الملف العام، صور المشاريع.                                |
| [Authorize]                | أي مستخدم مسجل — الحفظ، المتابعة، التعليق.                                    |
| [Authorize(Roles = "...")] | دور محدّد — <b>Innovator</b> ينشر، <b>Investor</b> يدفع، <b>Admin</b> يراجع.  |
| فحص الملكية في الكود       | <code>project.OwnerId != GetCurrentUser().Id</code> ⇒ <code>Forbid()</code> . |

جدول 22 أربع مستويات — والرابع مش attribute، لازم يتكتب بالإيد.

اعرف ده كويس

قاعدة رقم 4: فحص الدور مش فحص الملكية.

[Authorize(Roles = "Innovator")] بتأكد إنك مؤسس — مش إن المشروع ده بتاعك. مؤسس ثاني دوره **Innovator** برضه.

عشان كده الفحص التالت لازم يتكتب صراحةً في كل endpoint بيلمس حاجة مملوكة. ودي القاعدة اللي عضو التستينج أثبتها باختبارين حقيقيين (`UpdateProject_ByNonOwnerInnovator_IsForbidden` و `CloseRound_ByNonOwner_IsForbidden`).



## 22.2. Viewer-Relative Visibility

اعرف ده كويس

حاجة أعمق من مجرد سماح ومنع: نفس endpoint بيرجع بيانات مختلفة حسب مين بيص.

```
1 var privileged = isOwner || isAdmin;
```

المالك والأدمن بيشفوا كل الحالات؛ الزائر بيشف المراجع وغير الموقوف بس. والتعليق في الكود بيحذر إن العدادات لازم تستخدم نفس الشرط — «ترويسة مكتوب فيها 5 مشاريع فوق 3 بطاقات دي كذبة، وكمان بتقول لغريب إن المؤسس ده مخبي كام إدراج».

## 22.3. The Ownership Chain

| الكيان             | سلسلة الملكية اللي بتفحصها                                             |
|--------------------|------------------------------------------------------------------------|
| Project            | Project.OwnerId مباشرة.                                                |
| Investment         | Investment.InvestorId للمستثمر، أو Investment.Project.OwnerId للمؤسس.  |
| FundingRequest     | FundingRequest.InvestorId أو FundingRequest.Investment.Project.OwnerId |
| PaymentTransaction | InvestorId أو FundingRequest.Investment.Project.OwnerId                |
| TermSheet          | عن طريق Investment — الطرفين بس.                                       |
| ProjectDocument    | Project.OwnerId للرفع، والطرفين للقراءة.                               |

جدول 23 سلاسل الملكية — إزاي بتوصل من الكيان لصاحبه.

خد بالك

الجدول ده بيحدد إزاي بتفحص الملكية في كل حالة. مثال: عشان تعرف لو المستخدم ده يقدر يشوف معاملة دفع، لازم تعدي:

```
OwnerId ← Project ← Investment ← FundingRequest ← PaymentTransaction
```

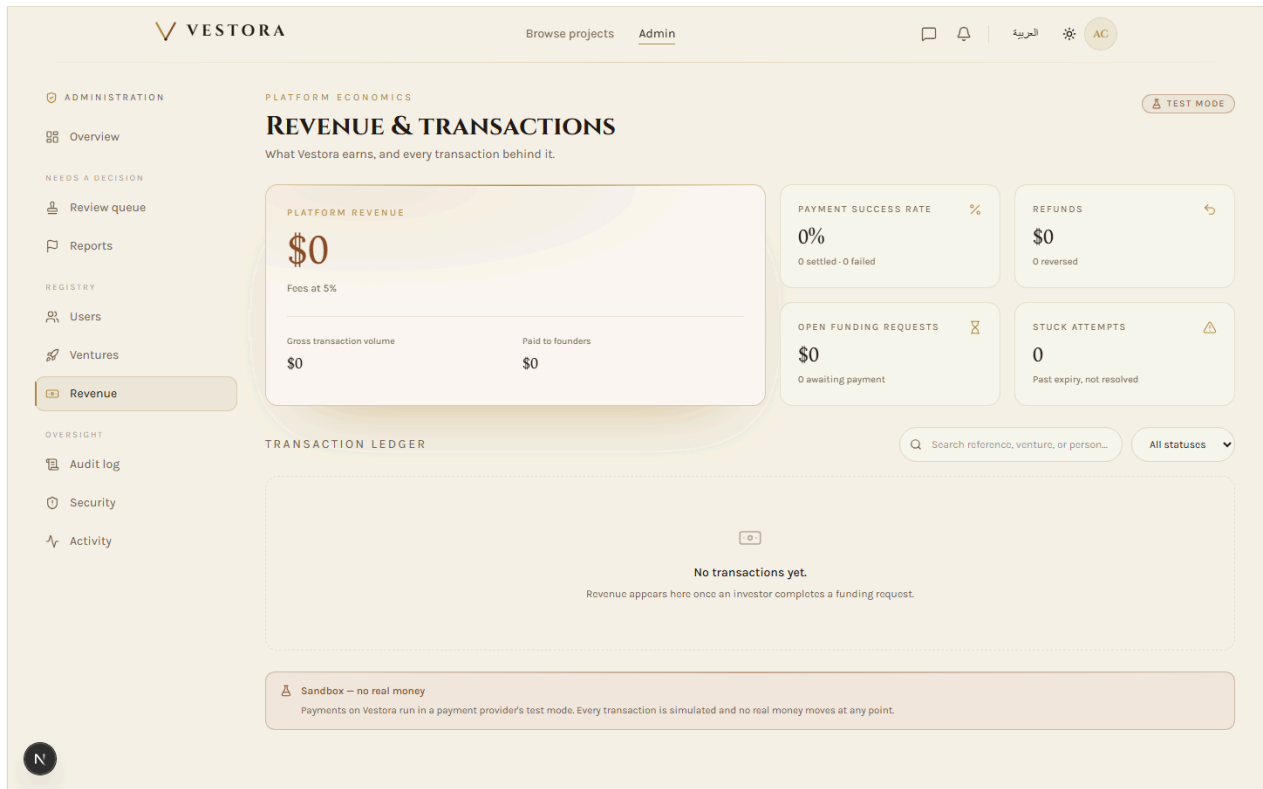
وعشان كده الكود بيعمل `.Include()` بالسلسلة دي قبل الفحص:

```
1 var isFounder = transaction.FundingRequest.Investment.Project.OwnerId == me;
2 if (transaction.InvestorId != me && !isFounder && !isAdmin())
3     return Forbid();
```

## 23. The Admin Screens — A Gallery

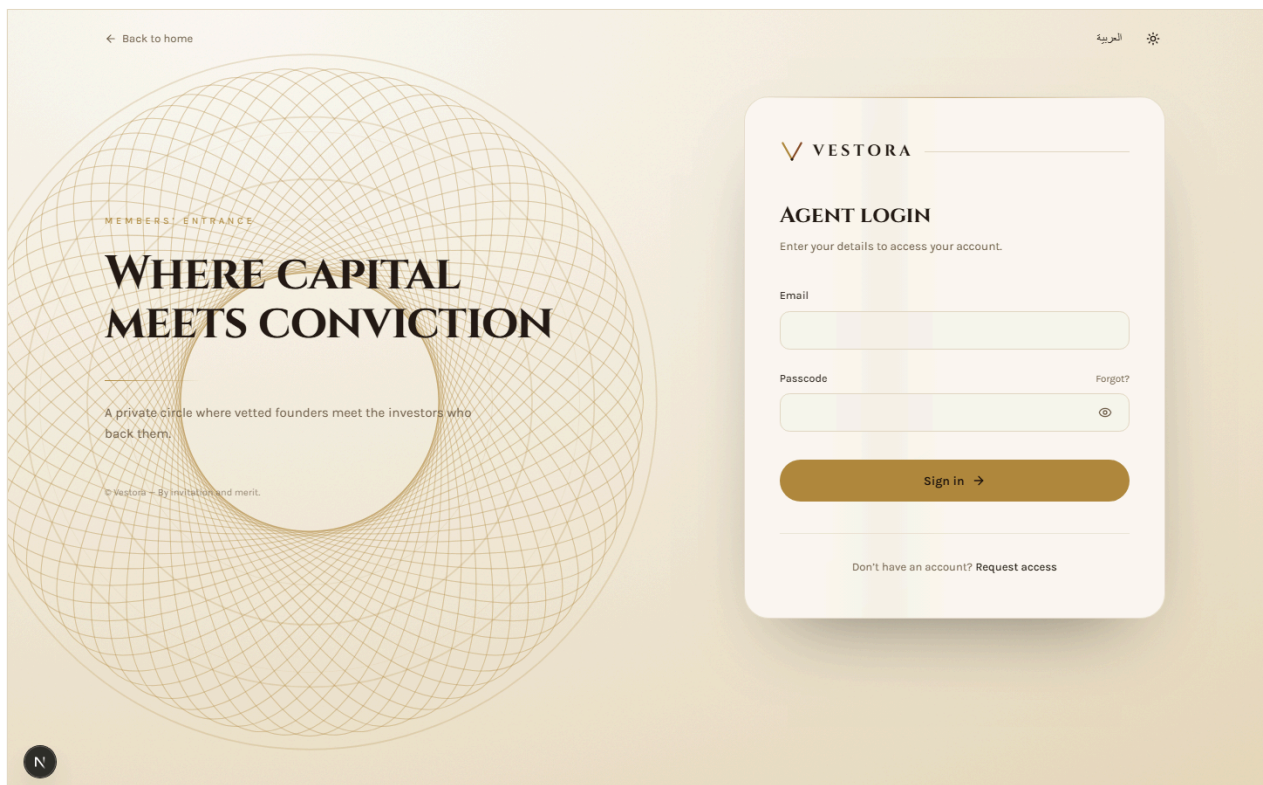
كل شاشة أدمن في المنتج إنت بنيتها. دي أهمهم.

## 23.1. Revenue and Reconciliation



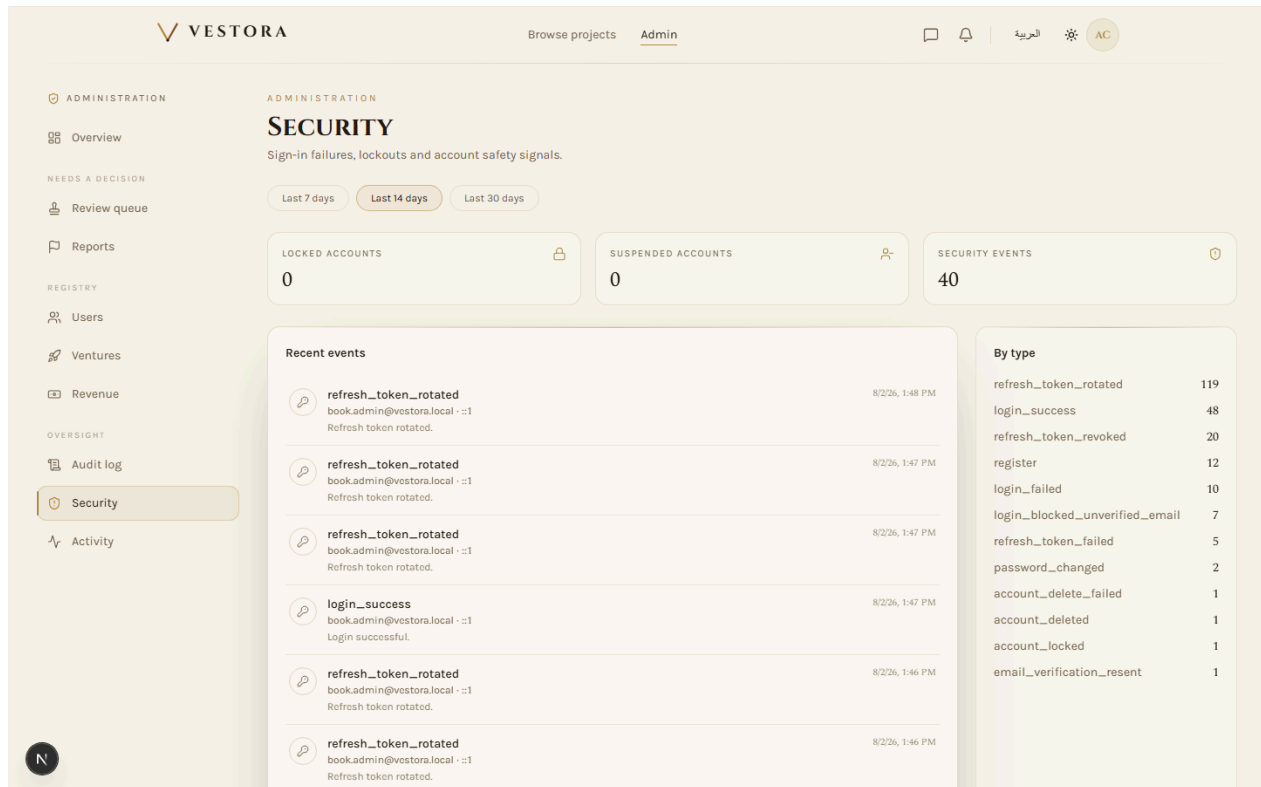
شكل 26 الإيرادات: رسوم المنصة من كل معاملة ناجحة، بالتفصيل.

اعرف ده كويس  
الرسوم هنا بتيجي من `PaymentTransaction.FeeAmount` — القيمة اللي اتجمدت وقت التسوية، مش اللي في الإعدادات دلوقتي. عشان كده الشاشة دي تفضل صحيحة حتى لو نسبة الرسوم اتغيرت بعدين.



شكل 27 البلاغات: المستخدمين يبلغوا عن محتوى، والأدمن يتصرّف.

## 23.2. Security and Activity

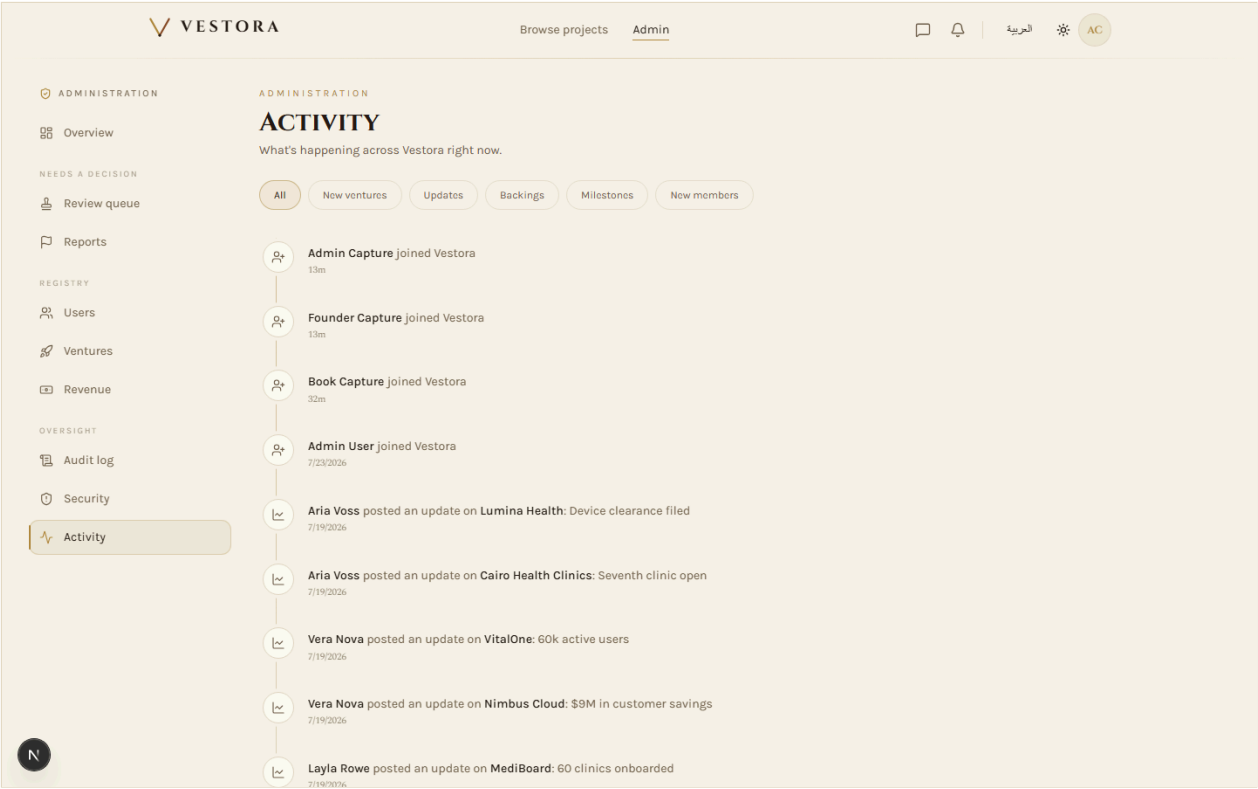


شكل 28 الأحداث الأمنية: محاولات الدخول، تدوير التوكنات، أقفال الحسابات.

خد بالك

البيانات في الشاشة دي كلها بتيجي من `SecurityLogs` — والجدول ده كود عضو الباكند هو اللي بيكتب فيه `AddSecurityEvent` في `AuthService`.)

يعني الشاشة بتاعتك، والبيانات بتاعته. ودي واحدة من الحدود المشتركة اللي ذكرناها في فصل تقسيم الفريق — لو الدكتور سأل عن الأرقام دي، وجّهه له.



شكل 29 النشاط العام: كل حدث في المنصة في خط زمني واحد.

## 24. Terms You Must Know

المصطلحات التي يمكن تتسأل فيها مباشرة كقائد.

| المصطلح                             | المعنى                                                             |
|-------------------------------------|--------------------------------------------------------------------|
| Idempotency                         | العملية تنفذ مرة واحدة مهما اتنادت كام مرة — هنا بفهرس فريد.       |
| Tranche                             | دفعة من التزام أكبر. التزام 50 ألف ممكن يتدفع على ثلاث دفعات.      |
| Term Sheet                          | ورقة الشروط المتفق عليها بين مؤسس ومستثمر.                         |
| Webhook                             | المزوّد بينده السيرفر بتاعنا لما حاجة تحصل — موثّق بالتوقيع.       |
| Sandbox                             | بيئة تجريبية — فلوس وهمية، نفس السلوك بالطبط.                      |
| TPH                                 | Table-Per-Hierarchy — ثلاث أنواع مستخدمين في جدول واحد بعمود مميز. |
| Soft delete                         | إخفاء الصف بعمود بدل مسحه فعليًا.                                  |
| Global query filter                 | شرط بيتطبّق تلقائيًا على كل استعلام لكيان معيّن.                   |
| Optimistic concurrency              | <code>RowVersion</code> بيمينع تعديلين متزامنين من يكتبوا فوق بعض. |
| Rate limiting                       | حد أقصى لعدد الطلبات في فترة — حماية من الإساءة.                   |
| SignalR                             | تقنية .NET للاتصال اللحظي (WebSocket) — الرسائل والإشعارات.        |
| DI Container                        | بييني الكائنات ويحقن تبعياتها بدل <code>new</code> في كل مكان.     |
| <code>ServiceResult&lt;T&gt;</code> | شكل موحد لرد الخدمة: نجاح بقيمة، أو فشل بنوعه.                     |

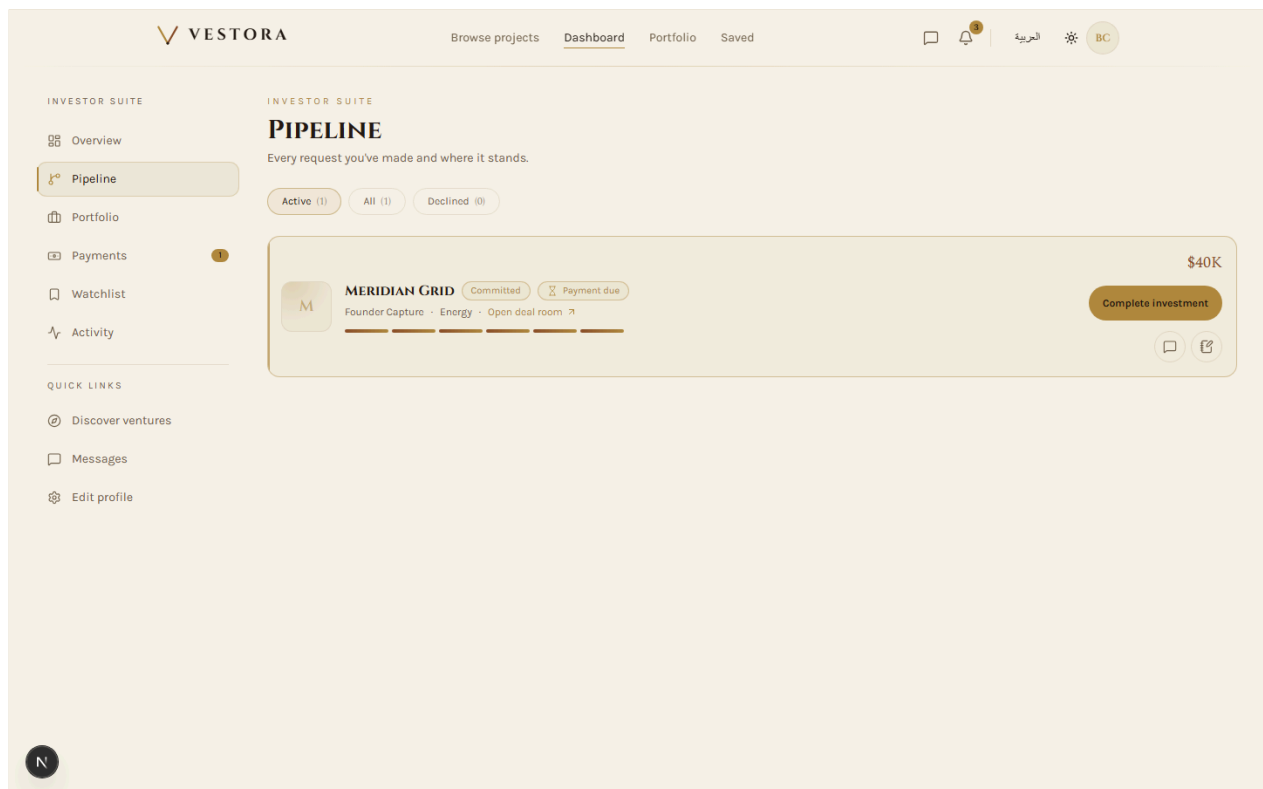
| المصطلح                | المعنى                                                                                                    |
|------------------------|-----------------------------------------------------------------------------------------------------------|
| Logical CSS properties | بدل <code>me-</code> / <code>ms-</code> بـ <code>ml-</code> / <code>mr-</code> — بتقلب لوحدها مع الاتجاه. |
| Compositor             | طبقة المتصفح التي ترسم التحويلات على كارت الشاشة — أسرع من JS.                                            |

**جدول 24** خـمـسـتـاشـر مـصـطـلـح.

## 25. The Money Model, Proven on Screen

الفصل ده بيوري **نفس الشاشات قبل وبعد** ما الفلوس توصل — وده أقوى دليل بصري على إن المفردات المالية مش مجرد كلام.

### 25.1. Before — Committed but Not Funded



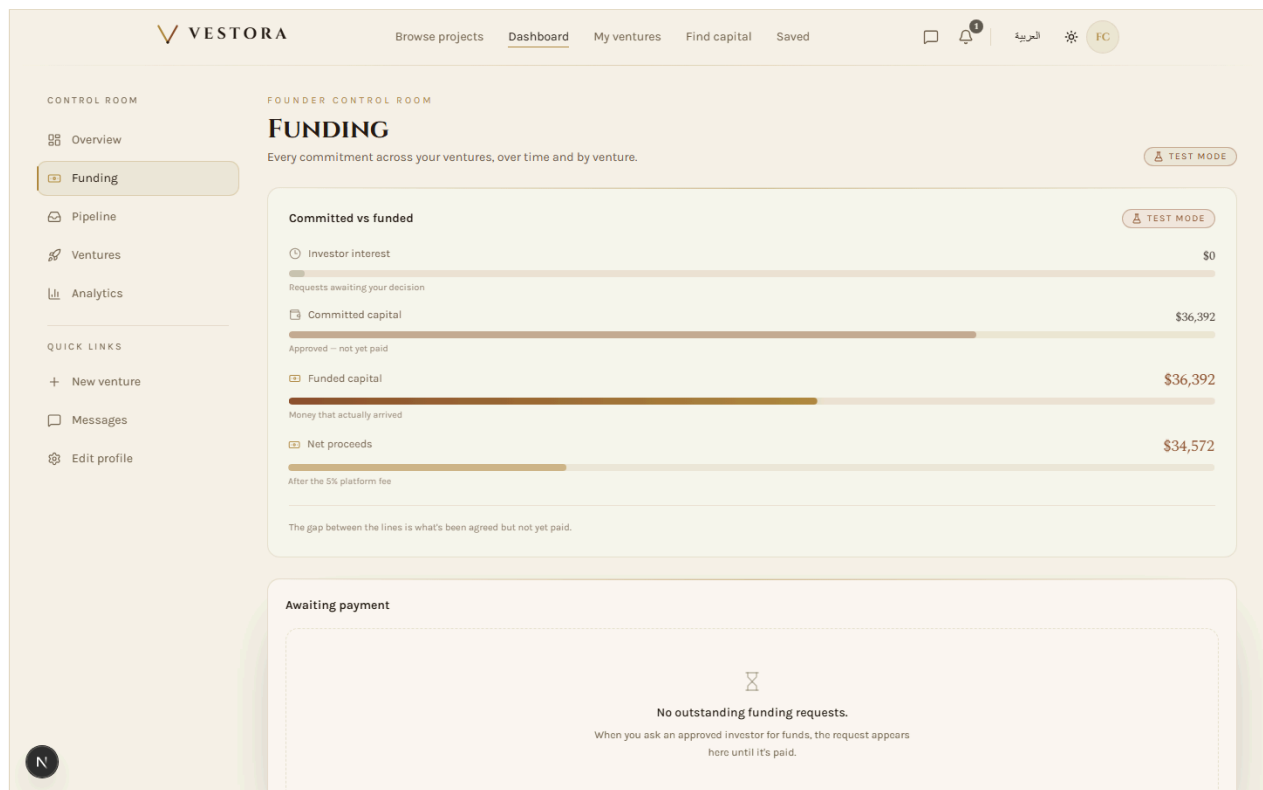
**شكل 30** خط أنابيب المستثمر: العلاقة **ملتزمة** بـ 40 ألف، والشارة بتقول «Committed» و«Payment due» — والزرار بيقول «Complete investment».

#### اعرف ده كويس

لاحظ الحالة دي كويس: المبلغ **ظاهر** (\$40K)، والعلاقة **معتمدة**، بس الفلوس **ماوصلتش**. الشارتين بيقلوا الحقيقة بالظبط: ملتزم، والدفع مستحق.

ده بالظبط الفرق اللي `AGENTS.md` قاعدة رقم 1 بتفرضه — ولو المنتج كان بيقل «تم جمع 40 ألف» هنا، كان هيبقى بيكذب.

## 25.2. After — The Numbers Line Up



**شكل 31** شاشة تمويل المؤسس بعد التسوية: ملتزم 36,392 ثم ممول 36,392 — والصافي 34,572 بعد رسوم المنصة 5%.

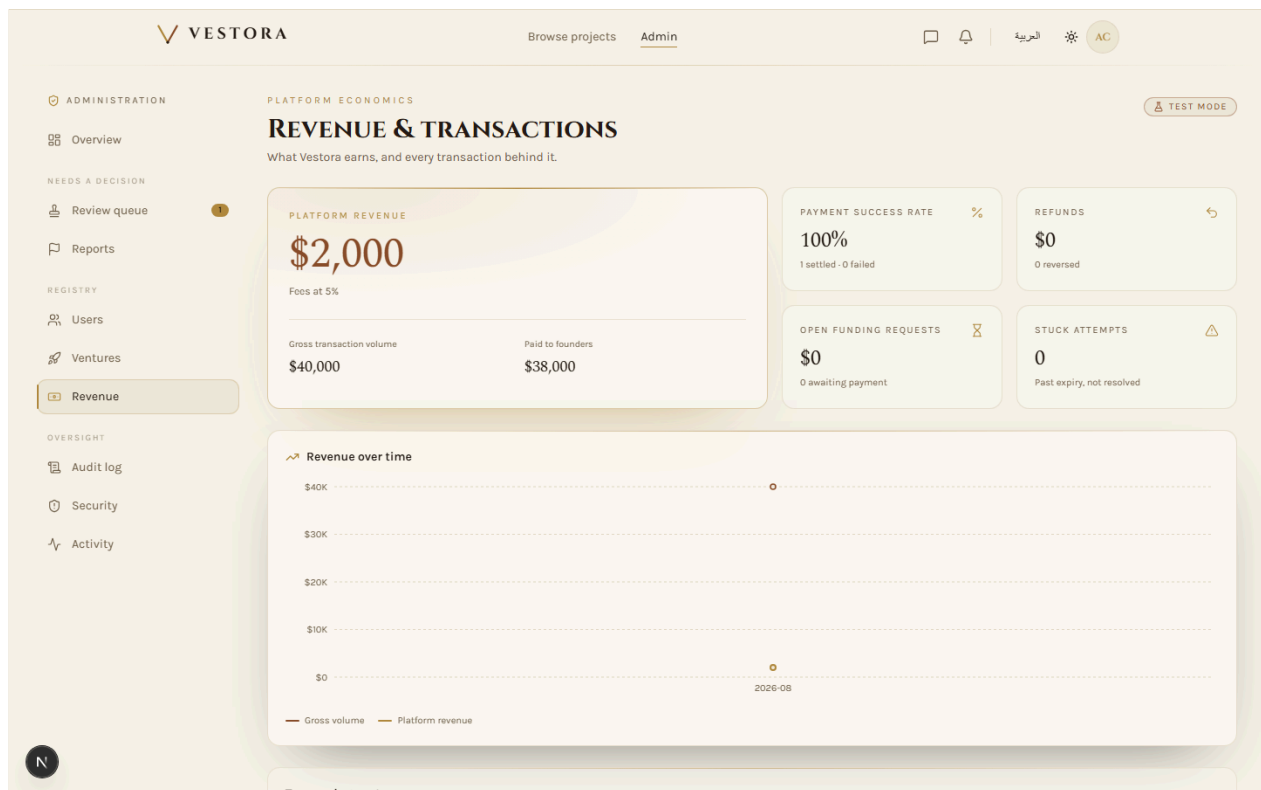
### اعرف ده كويس

دي أهم صورة في الملف كله. الأربع أرقام بيحكوا القصة كاملة:

- **اهتمام المستثمرين** 0\$ — مفيش طلبات مستنية قرار.
- **رأس المال الملتزم** 36,392\$ — «معتمد، لسه ما اتدفعش».
- **رأس المال الممول** 36,392\$ — «الفلوس اللي وصلت فعلاً».
- **صافي العائد** 34,572\$ — «بعد رسوم المنصة 5%».

والجملة تحتيهم مكتوبة في المنتج نفسه: «الفرق بين الخططين هو اللي اتفق عليه وما اتدفعش لسه.»  
لو الدكتور سأل «إزاي بتفرق بين الملتزم والممول؟» — افتح الصورة دي وخلص.

## 25.3. The Platform Side



**شكل 32** إيرادات المنصة: حجم المعاملات 40 ألف، رسوم 5% = 2,000، والمدفوع للمؤسسين 38 ألف.

### اعرف ده كويس

الحساب هنا شفاف بالكامل: \$40,000 حجم إجمالي ← \$2,000 رسوم المنصة (5%) ← \$38,000 للمؤسسين. ولاحظ المؤشرات التالية: **معدل نجاح الدفع 100%** (واحد تسوّى، صفر فشل)، بالإضافة لـ **محاولات عالقة: 0** — ودي اللي الـ **sweeper** مسؤول عنها. وشارة **TEST MODE** ظاهرة في الشاشتين — المنتج بيفكر المستخدم إن ده صندوق تجريبي حتى في شاشة الإيرادات.

### خد بالك

الأرقام في الصورتين مختلفة (36,392 مقابل 40,000) لأنهم من لقطتين في وقتين مختلفين ولحسابات مختلفة — واحدة لمؤسس واحد والثانية للمنصة كلها. **ماتقولش إنهم نفس الصفحة** لو اتسألت.

## 26. Endpoints Reference — Your Areas

كل نقطة نهاية في نصيكي. لو الدكتور سأل «إيه الـ API بتاعك؟» — افتح الصفحات دي.

### 26.1. Payments — `api/payments`

| Method + Route          | مين  | بيعمل إيه                               |
|-------------------------|------|-----------------------------------------|
| <code>GET config</code> | مسجل | إعدادات الدفع الظاهرة (المزود، العملة). |

| Method + Route                            | مين         | يُعمل إيه                                   |
|-------------------------------------------|-------------|---------------------------------------------|
| POST investments/{id}/funding-request     | Innovator   | المؤسس يبطلب مبلغ — أو دفعة منه.            |
| GET investments/{id}/funding-request      | الطرفين     | الطلب الحالي للعلاقة.                       |
| POST funding-requests/{id}/counter        | Investor    | المستثمر يقترح رقم مضاد.                    |
| POST funding-requests/{id}/counter/answer | Innovator   | المؤسس بيقبل أو يرفض المقابل.               |
| POST funding-requests/{id}/cancel         | المؤسس/أدمن | سحب الطلب.                                  |
| POST funding-requests/{id}/checkout       | Investor    | فتح جلسة دفع — بحد 8 كل 5 دقائق.            |
| POST transactions/{id}/verify             | الأطراف     | سؤال المزود وتطبيق النتيجة — دي اللي بتقرر. |
| POST transactions/{id}/cancel             | Investor    | التخلي عن محاولة دفع.                       |
| GET transactions/{id}                     | الأطراف     | تفاصيل معاملة.                              |
| GET mine                                  | Investor    | كل فلوس المستثمر في رد واحد.                |
| POST sandbox/sessions/{id}/resolve        | Investor    | محاكاة نتيجة الدفع (الصندوق التجريبي).      |
| POST webhook/stripe                       | مفتوح       | قناة التأكيد — موثقة بالتوقيع مش بتوكن.     |

جدول 25 نقاط الدفع.

خد بالك  
لاحظ POST webhook/stripe — دي النقطة الوحيدة المفتوحة في نظام الدفع كله، والسبب مذكور في الكود: «Stripe مالوش JWT — فبتتوثق بالتوقيع بدالها. أي طلب غير موقع أو قديم بيترفض قبل ما الجسم يتبصله أصلاً.»

## 26.2. Deal Room — api/deals

| Method + Route                     | مين          | يُعمل إيه                            |
|------------------------------------|--------------|--------------------------------------|
| GET {investmentId}                 | الطرفين/أدمن | غرفة الصفقة كاملة.                   |
| GET                                | مسجل         | قائمة الصفقات.                       |
| POST {id}/questions                | الطرفين      | سؤال جديد.                           |
| PUT questions/{id}/answer          | المسؤول      | الرد على سؤال.                       |
| POST questions/{id}/withdraw       | صاحبه        | سحب سؤال.                            |
| POST {id}/document-requests        | Investor     | طلب مستند.                           |
| PUT document-requests/{id}         | الطرفين      | تحديث الطلب.                         |
| POST document-requests/{id}/upload | Innovator    | رفع المستند.                         |
| GET document-requests/{id}/file    | الطرفين      | تحميل المستند.                       |
| POST {id}/terms                    | الطرفين      | اقتراح ورقة شروط.                    |
| POST terms/{id}/accept             | الطرف الثاني | قبول الشروط.                         |
| POST terms/{id}/decline            | الطرف الثاني | رفض الشروط.                          |
| PUT {id}/note                      | الطرفين      | ملاحظة خاصة (كل طرف يشوف بتاعته بس). |

جدول 26 نقاط غرفة الصفقة.



26.3. Admin — `api/admin`

| Method + Route                | مين        | يُعمل إيه                           |
|-------------------------------|------------|-------------------------------------|
| POST bootstrap                | مفتوح+سرّ  | أول أدمن — بيتقفل للأبد بعدها.      |
| POST admins                   | الأساسي بس | إنشاء أدمن جديد.                    |
| POST admins/{id}/make-primary | الأساسي بس | نقل الصلاحية الأساسية.              |
| GET users                     | أدمن       | قائمة المستخدمين بفلتر.             |
| POST users                    | أدمن       | إنشاء مستخدم.                       |
| PUT users/{id}                | أدمن       | تعديل مستخدم.                       |
| DELETE users/{id}             | أدمن       | حذف ناعم.                           |
| POST users/{id}/suspend       | أدمن       | إيقاف حساب.                         |
| POST users/{id}/restore       | أدمن       | استعادة حساب.                       |
| GET security                  | أدمن       | الأحداث الأمنية.                    |
| GET growth                    | أدمن       | نمو المستخدمين.                     |
| GET alerts                    | أدمن       | التنبيهات.                          |
| GET audit-log                 | أدمن       | سجلّ المراجعة بالسبب والفرق والـIP. |
| GET analytics                 | أدمن       | التحليلات.                          |

**جدول 27** نقاط الأدمن الأساسية (وفيه ثلاث كترولرز ثانية بنقاطهم).

## 27. Request and Response Examples

أمثلة حقيقية تنفع تفتحها قدام الدكتور.

### 27.1. Creating a Funding Request

```
1 POST /api/payments/investments/42/funding-request
2 {
3   "amount": 25000,
4   "note": "First tranche - covers the pilot build."
5 }
```

الرد — 201:

```
1 {
2   "id": 17,
3   "reference": "VST-FR-2026-000017",
4   "investmentId": 42,
5   "amount": 25000,
6   "currency": "USD",
7   "status": "Open",
8   "note": "First tranche - covers the pilot build.",
9   "expiresAtUtc": "2026-09-01T12:00:00Z"
10 }
```

اعرف ده كويس

لاحظ `reference` — مرجع بشري يتقال في التليفون («المرجع VST-FR-2026-000017») بدل رقم قاعدة بيانات. كل كيان مالي في المشروع عنده واحد.

## 27.2. Opening a Checkout

1 POST /api/payments/funding-requests/17/checkout

الرد — 200:

```
1 {
2   "transactionId": 88,
3   "reference": "VST-TX-2026-000088",
4   "checkoutUrl": "http://localhost:3000/payments/sandbox-checkout?session=sim_
cs_...",
5   "provider": "simulated",
6   "isSandbox": true,
7   "amount": 25000,
8   "currency": "USD",
9   "resumed": false
10 }
```

خد بالك

`resumed: false` معناها اتفتحت جلسة جديدة. لو كانت `true`، يبقى فيه محاولة حيّة موجودة أصلاً واطرّجت زي ما هي. التعليق في الكود بيشرح ليه: «محاولة حيّة بتتعاود بدل ما تتكرّر. اتنين checkout مفتوحين لطلب واحد هو أقصر طريق لتمويل نفس الحاجة مرتين.»

## 27.3. Tranche Refusal — A Real Guard

لو المؤسّس طلب أكثر من الفاضل في الالتزام المتفق عليه:

```
1 {
2   "message": "The agreed terms are 50,000 USD and 25,000 has settled.
3               Ask for at most 25,000, or propose new terms."
4 }
```

اعرف ده كويس

الرسالة دي بتعلّم مش بس بترفض: بتقول الاتفاق كام، واتدفع كام، والمسموح كام، وإيه البديل (اقتراح شروط جديدة). ودي نفس فلسفة رسائل الخطأ في فصل نصوص الواجهة — قول المشكلة والمخرج.

## 28. The Counter-Offer — Negotiating the Number

المؤسّس بيطلب رقم، والمستثمر ممكن ما يوافقش عليه. الميزة دي بتدي المستثمر نصّه من الجملة.

## 28.1. Why It Exists

اعرف ده كويس

التعليق في `FundingRequest.cs` يشرح المشكلة بدقّة:

«المؤسّس بيحدّد الرقم، وخيارات المستثمر الوحيدة كانت إنه يدفعه أو يسيبه يخلص في صمت. كل حاجة قبل النقطة دي في المنتج مبنية لمحادثة، وبعدين آخر خطوة — اللي عن الفلوس — كانت خد أو سيب.»

والقرار المعماري: «بتقعد على الطلب مش في جدول لوحدها، لأنها خاصية لطلب واحد: على الأكثر مقابل واحد حيّ في المرة، وقبوله بينهي الطلب اللي اتعمل ضده.»

## 28.2. The Flow

| الخطوة     | مين                    | بيحصل إيه                                                       |
|------------|------------------------|-----------------------------------------------------------------|
| 1. الطلب   | <code>Innovator</code> | بيطلب 50 ألف.                                                   |
| 2. المقابل | <code>Investor</code>  | بيقتراح 35 ألف بسبب مكتوب. <code>CounterStatus: Proposed</code> |
| 3. الرد    | <code>Innovator</code> | بيقبل أو يرفض.                                                  |
| 4. لو قبل  | النظام                 | بيقل الطلب القديم ويفتح واحد جديد بالرقم المتفق عليه.           |

جدول 28 أربع خطوات — والرقمين بيضلوا في السجلّ.

خد بالك

ليه طلب جديد مش تعديل القديم؟

التعليق بيحاجب: «الأرقام على صف مالي عمرها ما بتتعد كتابتها. قبول المقابل بيقل الطلب القديم ويفتح واحد جديد؛ و `SupersedesRequestId` هو الخيط بينهم.»

يعني السجلّ بيفضل شايل الاتنين: الرقم اللي اطلب، والرقم اللي اتفق عليه. ودي نفس فلسفة «الصفوف النهائية ثابتة» بتاعة المعاملات.

## 29. Running the Whole Thing

كقائد، إنت اللي المفروض تعرف تشغل المشروع كامل قدام اللجنة.

### 29.1. The Two Commands

```
1 cd MyAppApi/MyAppApi
2 dotnet run
```

```
1 cd Frontend/vestora
2 npm install
3 npm run dev
```

الفرونت بيفتح على `http://localhost:3000` ، والAPI بيفتح Swagger على `/swagger`.

خد بالك

لو الـ API رفض يشتغل وقال حاجة عن مفتاح Stripe — ده مقصود وإنت اللي عملته. المفتاح لازم يبدأ بـ `sk_test_`. اعرف الجملة دي كويس عشان لو حصلت قدام اللجنة تشرحها كميزة مش كعطل.

اعرف ده كويس

الأسرار مش في المستودع (قاعدة 12). يعني عشان المشروع يشتغل على جهاز جديد محتاج:

- `ConnectionStrings:InvestContextDB` — الاتصال بقاعدة البيانات
  - `JwtSettings:Key` — مفتاح توقيع التوكنات (32 بايت على الأقل)
  - `AdminBootstrap:SecretKey` — سرّ إنشاء أول أدمن
- وكلمهم بيتخطّوا في `user secrets` أو متغيّرات بيئة، مش في `appsettings.json` — اللي فيه أماكن فاضية عن قصد.

## 29.2. The Five-Minute Demo

لو الدكتور قال «وَرَيْنِي»، ده المسار اللي يوري أقوى حاجات في نصيبك:

| الخطوة           | اللي تعمله وتقول                                                                                                          |
|------------------|---------------------------------------------------------------------------------------------------------------------------|
| 1. اللاندينج     | افتح الصفحة الرئيسية وحرك الماوس على الهيرو — وريهم الـ 3D وقول إنه بالـ <code>compositor</code> مش <code>WebGL</code> .  |
| 2. بدّل اللغة    | دوس على العربي — التخطيط كله بينقلب. قول: «سطرين في <code>LocaleProvider</code> بيعملوا ده، لأن كل CSS في المشروع منطقي.» |
| 3. لوحة الأدمن   | ادخل كأدمن ووري النظرة العامة والمراجعة والإيرادات.                                                                       |
| 4. المراجعة      | ارفض مشروع بسبب مكتوب — ووري إن السبب بيوصل للمؤسّس.                                                                      |
| 5. غرفة الصفقة   | افتح غرفة صفقة حقيقية ووري الأقسام الستة.                                                                                 |
| 6. اقتراح شروط   | اقتراح ورقة شروط واقلها من الطرف الثاني.                                                                                  |
| 7. طلب دفعة      | اطلب جزء من الالتزام — ووري إن النظام بيقبل، ولما تطلب أكثر من الفاضل بيرفض برسالة بتقول الفاضل كام.                      |
| 8. الدفع         | افتح <code>checkout</code> ووري شارة الصندوق التجريبي، وادفع، ووري التأكيد.                                               |
| 9. سجّل المراجعة | ارجع للأدمن ووري إن الفعل اتسجّل بالسبب والفرق والـ IP.                                                                   |

جدول 29 تسع خطوات بتغطّي أقوى حاجات في نصيبك.

اعرف ده كويس

أقوى لحظتين في العرض:

- **الخطوة 7** — طلب دفعة جزئية ورفض الزيادة. دي بتوري إن النظام فاهم الفرق بين الالتزام والمدفوع، وإن فيه حارس بيمنع التحصيل مرتين.
- **الخطوة 2** — قلب اللغة. بسيطة بصرياً، بس بتوري انضباط معماري في كل سطر CSS في المشروع.

## 30. The Documentation Suite

كتبت **أحد عشر ملف توثيق** (4336 سطر) — ودي حاجة قليل ما تتعمل في مشاريع التخرّج.

| الملف                        | بيجاب سؤال                            |
|------------------------------|---------------------------------------|
| 01-OVERVIEW.md               | إيه المنتج ده؟ وإيه معنى كل كلمة فيه؟ |
| 02-ARCHITECTURE.md           | إزاي النظام متوصّل؟                   |
| 03-DATA-MODEL.md             | إيه الأعمدة والقيود الموجودة؟         |
| 04-API-REFERENCE.md          | إيه endpoints؟                        |
| 05-BUSINESS-RULES.md         | إيه القواعد اللي ماينفعش تتكسر؟       |
| 06-FRONTEND.md               | المسارات والمكوّنات ونظام التصميم؟    |
| 07-SETUP.md                  | إزاي أشغله؟                           |
| 08-CONVENTIONS.md            | إزاي أضيف فيتنشر جديد؟                |
| 09-STATUS.md                 | إيه الناقص؟                           |
| 10-ROLES-AND-CAPABILITIES.md | كل دور يقدر يعمل إيه بالطبط؟          |
| 11-INVESTMENT-FLOW.md        | رحلة استثمار واحدة من أولها لآخرها؟   |

**جدول 30** أحد عشر ملف — لكل واحد سؤال واحد بيجاب عليه.

اعرف ده كويس

**ليه التقسيمة دي بالذات؟** كل ملف بيجاب **سؤال واحد**. الفرق بين ده وبين ملف واحد كبير إن العضو اللي عنده سؤال محدّد بيلاقى إجابته من غير ما يقرأ حاجة ثانية.

ولو الدكتور سأل «إزاي عضو جديد كان هيدخل على المشروع؟» — الإجابة: **AGENTS.md** بيقوله يبدأ بـ **01-OVERVIEW.md**، وبعدها كل ملف بيوجّهه للي بعده.

## 31. The Schema History — Twenty-Nine Migrations

عضو الداتا بيز ماسك ميكانيكا الترحيلات، بس **التسلسل نفسه** بيحكى قصة المشروع — ودي حاجة القائد بس اللي يعرف يقصّها.

### 31.1. The Story in Phases

| المرحلة  | الترحيلات وإيه اللي اتضاف                                                      |
|----------|--------------------------------------------------------------------------------|
| البداية  | initialCreate — المخطط الأول.                                                  |
| الأمان   | Phase4ProductionAuthSecurity — التوكنات وسجلّ الأمان والقفل.                   |
| الاكتشاف | Phase5SearchAndProjectImages · BrowseDiscoveryIndexes — البحث والصور والفهارس. |

| المرحلة  | الترجيلات وإيه اللي اتضاف                                                                                                                                                                       |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| العلاقات | <ul style="list-style-type: none"> <li>Phase2_InvestmentPipeline</li> <li>AddInvestmentApprovalStatus</li> <li>ExpansionRelationshipWorkspace</li> </ul> <p>— الحالة والمراحل وغرفة الصفقة.</p> |
| المحتوى  | <ul style="list-style-type: none"> <li>AddBookmarks</li> <li>AddTeamAndDocuments</li> <li>AddUpdatesAndMilestones</li> <li>AddFollowsAndProfileMedia</li> </ul>                                 |
| الإشراف  | <ul style="list-style-type: none"> <li>Phase5_SuspensionAndModerationNotes</li> <li>AddModerationAndAnalytics</li> <li>Phase6_VentureLifecycle</li> </ul>                                       |
| الفلوس   | <ul style="list-style-type: none"> <li>FundingRequestsAndSandboxPayments</li> </ul> <p>— أكبر تحوّل في المشروع.</p>                                                                             |
| التفاوض  | <ul style="list-style-type: none"> <li>StageHistoryRemindersAndReconciliation</li> <li>TermSheetsCounterOffersAndTwoWayDocs</li> </ul>                                                          |
| الحوكمة  | <ul style="list-style-type: none"> <li>AddMustChangePasswordFlag</li> <li>AddPrimaryAdminFlag</li> <li>AuditTrailReasonDiffAndIp</li> </ul> <p>— آخر ثلاثة.</p>                                 |

**جدول 31** تسعة وعشرين ترحيل — كل مجموعة بتحكى مرحلة من عمر المنتج.

#### اعرف ده كويس

**أهم ترحيل في المشروع كله:** FundingRequestsAndSandboxPayments

قبله، «الاستثمار» كان رقم على علاقة. بعده، بقى فيه FundingRequest و PaymentTransaction و PaymentEvent — يعني الفرق بين وعد وبين فلوس وصلت بقى موجود في السكيميا نفسها، مش في التسمية بس.

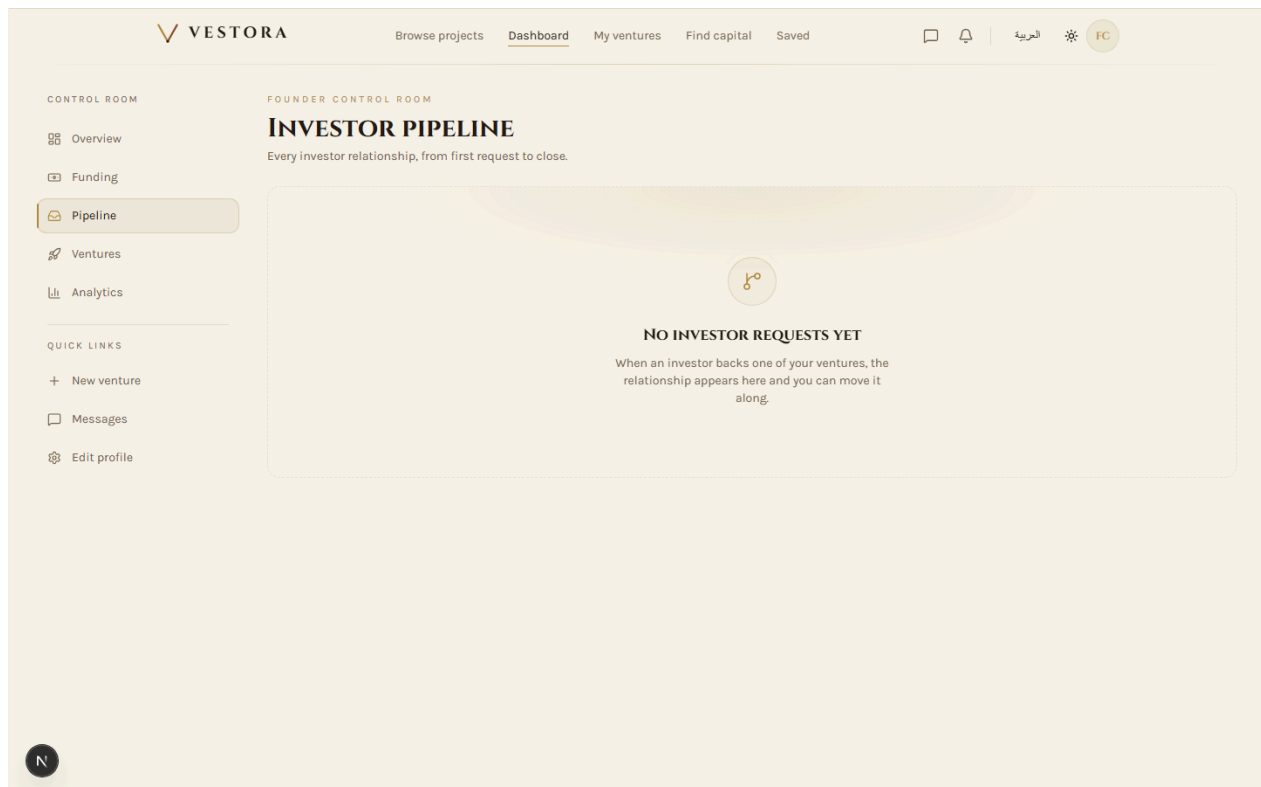
لو الدكتور سأل «إيه أكبر تغيير معماري في المشروع؟» — ده هو.

#### خد بالك

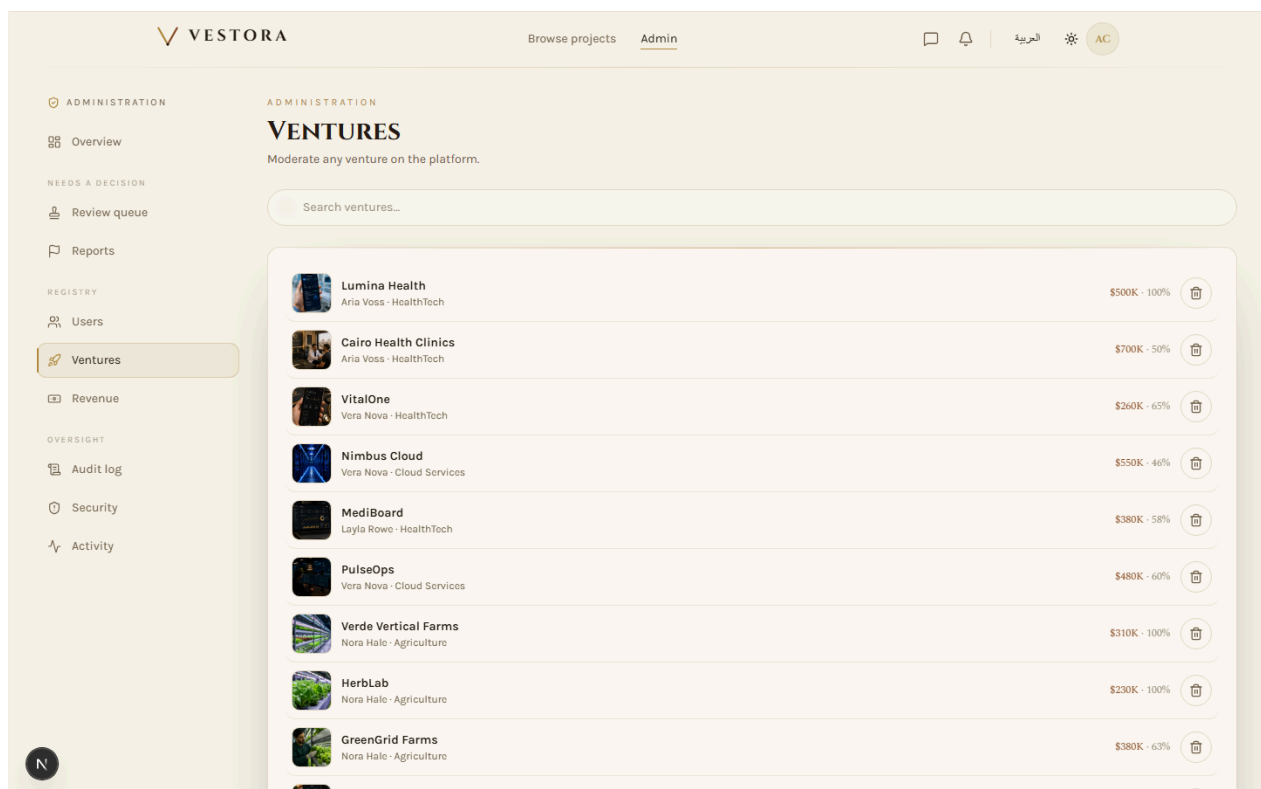
ترجيل OneLiveInvestmentPerInvestor بيضيف فهرس فريد مفلتر: مستثمر واحد ماينفعش يكون عنده أكثر من علاقة حيّة على نفس المشروع.

القيد ده في الداتايز مش في الكود — يعني حتى لو الكود غلط، القاعدة مستحيل تتكسر. ودي فلسفة متكررة في المشروع: القواعد الحرجة تتفرض بالبنية مش بالانضباط، زي فهرس عدم تكرار الدفع بالطبط.

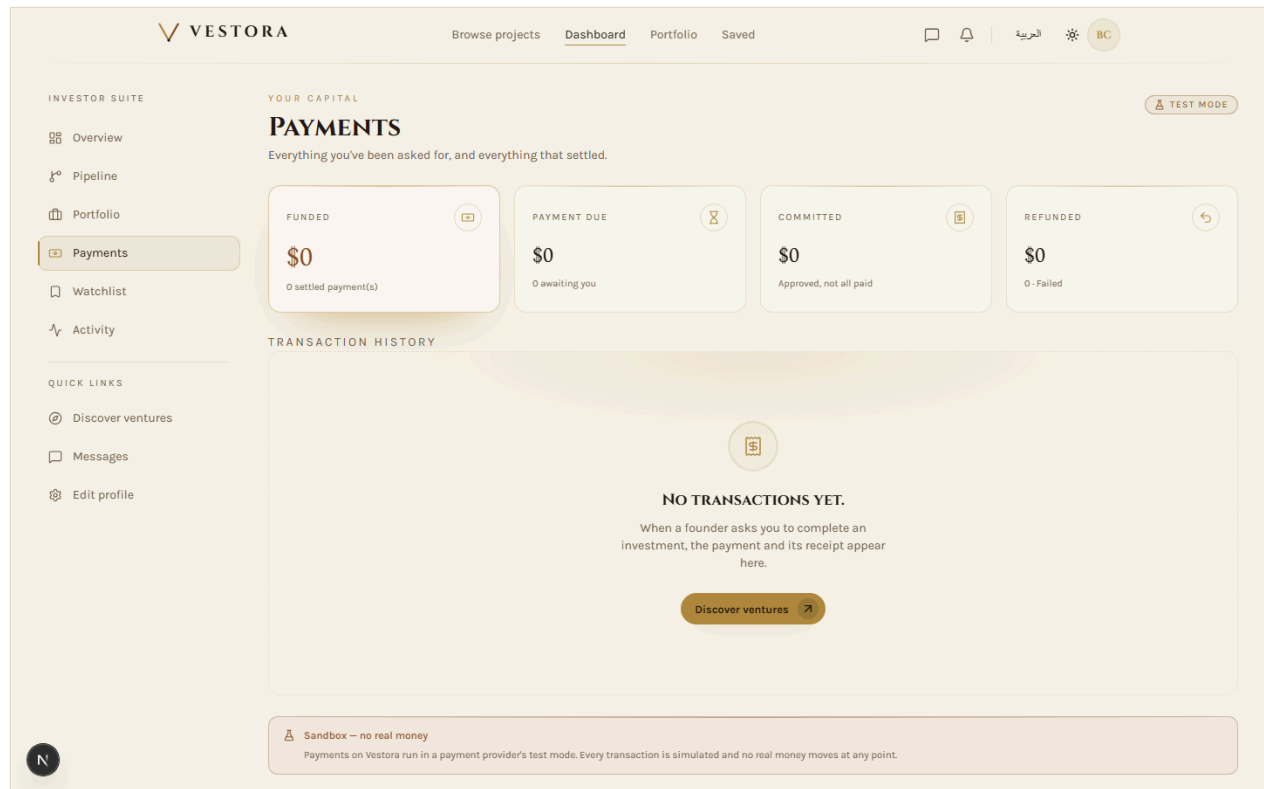
## 32. More Screens You Built



**شكل 33** خط أنابيب المؤسس: كل علاقة مستثمر من أول طلب لحد الإقفال — بحالة فاضية موجهة بتشرح إمتى هتتملى.



**شكل 34** كل المشاريع من عين الأدمن — بحالة المراجعة ودورة الحياة لكل واحد.



شكل 35 مدفوعات المستثمر: المستحق، المسوّى، والفاشل في مكان واحد.

#### اعرف ده كويس

لاحظ التكرار المقصود في الشاشات دي كلها: نفس الشريط الجانبي، نفس الحالة الفاضية الموجهة، نفس شكل الكارت.

ده مش كسل — ده نظام التصميم شغّال. المستخدم اللي اتعلم يقرأ شاشة واحدة يقدر يقرأ الباقي، والمكونات دي بتتكتب مرة وتستخدم في كل مكان.

## 33. Governance — The Rules You Enforced

في جذر المشروع فيه **اتناشر قاعدة** إنت كتبتها والفريق كله ملتزم بيها. الفصل ده عن **ليه** كل واحدة موجودة.

| القاعدة                                  | ليه                                                                                                   |
|------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 1. مفردات الفلوس                         | $\text{Interest} \geq \text{Committed} \geq \text{Funded}$<br>الخلط بينهم هو الفرق بين حقيقة وادّعاء. |
| 2. مغيث حساب مالي بره <b>FundingMath</b> | كان مكتوب في 15 مكان واتفقوا بالصدفة.                                                                 |
| 3. <b>Status</b> و <b>Stage</b> محوريين  | النقاش حاجة والفلوس حاجة تانية.                                                                       |



| القاعدة                                                                               | ليه                                                      |
|---------------------------------------------------------------------------------------|----------------------------------------------------------|
| 4. فحص الدور ≠ فحص الملكية                                                            | مؤسس ثاني دوره<br><code>Innovator</code> برضه.           |
| 5. الحدث قبل الأثر                                                                    | الصفوف النهائية ثابتة — إعادة المحاولة صف جديد.          |
| 6. <code>DateTime.UtcNow</code> بس                                                    | كل تاريخ محفوظ UTC.                                      |
| 7. كل عمود فلوس <code>HasPrecision(18, 2)</code>                                      | الكسور العشرية في الفلوس مش رفاهية.                      |
| 8. ماتدّعيش اللي ماتقدرش تثبته                                                        | مفیش شارة «موثّق»، مفیش درجة ثقة. «مُراجَع» مش «vetted». |
| 9. متمسّحش صف علاقة عند الرفض                                                         | التاريخ بتاع الطرفين حقيقة مشتركة.                       |
| 10. الدفع صندوقي بس                                                                   | التطبيق بيرفض يشتغل على مفتاح حقيقي.                     |
| 11. الفرونط: مفیش <code>fetch</code> بره الطبقة، مفیش نص في الكود، مفیش اتجاه فيزيائي | عشان الاتساق والترجمة والـRTL.                           |
| 12. الأسرار مش في ملف متسلّم                                                          | فيه <code>appsettings.json</code> أماكن فاضية عن قصد.    |

جدول 32 اتناشر قاعدة — دستور المشروع.

اعرف ده كويس

ليه كتبت الملف ده أصلاً؟

في فريق ستّة أفراد بيشغلوا على نفس الكود، القواعد اللي في دماغ شخص واحد بتتكسر أول ما حد ثاني يكتب سطر. الملف بيحوّل القرارات المعمارية لحاجة **مكتوبة وقابلة للمراجعة** — أي عضو يقدر يفتحه ويعرف ليه الحاجة كده. ودي إجابة قوية لو الدكتور سأل «إزاي ضمنت جودة الكود في فريق؟»

خد بالك

أهم قاعدة أخلاقية — رقم 8:

مفیش شارة «موثّق» في المنتج كله، لأن فيستورا **مبتتحققش** من هوية حد ولا بتدقّق شركة. الشارة كانت هتبقى كذب بصري. وبدالها `TrustSignals` بيني المصدقية من حفايق **قابلة للإثبات بس**: الإيميل أتأكّد، عمر الحساب، الأدمن راجع الإدراج، المؤسس نشر مستندات، داعمين تانيين زكّوه.

## 34. Known Limitations

الأمانة هنا أقوى من التجميل. لو اتسألت «إيه العيوب؟» — دي إجابتك.

| الحدّ                                           | التفصيل                                                                            |
|-------------------------------------------------|------------------------------------------------------------------------------------|
| الدفع صندوق بالكامل                             | مفیش فلوس حقيقية بتتحرك — وده قرار مقصود مش نقص، المشروع مالوش رخصة مالية.         |
| الصور في قاعدة البيانات                         | البايتس متخزنة في القاعدة نفسها مش تخزين خارجي — مقايضة مقصودة لحجم المشروع.       |
| التحقق مكرّر يدويًا                             | <code>AccountRules.cs</code> و <code>rules.ts</code> بيتزامنوا بالانضباط مش بأداة. |
| البحث بـ <b>LIKE</b>                            | مش محرك بحث حقيقي — كفاية للحجم الحالي مش لأضعافه.                                 |
| مفیش قياس Code Coverage                         | التغطية مبنية على السيناريوهات والendpoints المختبرة، مش نسبة مقيسة بأداة.         |
| مفیش أتمتة متصفح للفرونت                        | الاعتماد على اختبار يدوي منظم بـ 65 سيناريو.                                       |
| <b>جدول 33</b> حدود معروفة — قرارات نطاق واعية. |                                                                                    |

**اعرف ده كويس**

الطريقة الصح تقول بيها أي حدّ من دول:

«ده قرار واعٍ للنطاق، مش حاجة فاتتنا. البديل كان يكلف [كذا]، والحجم الحالي ما يستاهلش.»

الفرق بين القائد اللي عارف حدود مشروعه والقائد اللي مش شايفها = الفرق في التقدير.

## 35. Doctor Questions — The Leader Edition

أسئلتك مختلفة عن أسئلة الأعضاء: هما بيتسألوا «إزاي عملت ده؟» وإنت بتسأل «ليه قرّرت كده؟».

**مثال**

«إنت عملت إيه بالطبط في المشروع؟»

الإجابة: «صمّمت المعمارية ووّزعت الفريق، وبنيت أضخم نصيب تقني: الدفع كامل بالتمويل على دفعات، غرفة الصفقة والتفاوض، الأدمن بستّة كنترولرز، البنية التحتية للباكند، وطبقة الفرونت المشتركة كلها — أكثر من 9000 سطر في الباكند لوحده.»

لو ضغط: «طيب الأعضاء عملوا إيه؟» — استعرض الجدول: كل واحد ومجاله الكامل. مهم تعرف تجاوب عن شغل كل عضو، مش شغلك بس.

**مثال**

«على أي أساس ووّزعت الشغل؟»

الإجابة: «كل عضو ياخذ مجال كامل يقدر يشرحه من أوله لآخره. عشان كده عضو الباكند خد المصادقة كاملة مش نصّها، وعضو الفرونت خد التصفّح كامل.»

لو ضغط: «وليه مسكت إنت أصعب الحاجات؟» — «لأن الدفع وغرفة الصفقة فيهم قرارات لو اتاخذت غلط بتكسر سلامة البيانات المالية — ودي مسؤولية القائد.»

## مثال

«ليه مغيش Repository Pattern؟»  
 الإجابة: «DbContext» نفسه أصلًا Repository و Unit of Work. طبقة فوقه كانت هتبقى غلاف بينده على غلاف من غير ما يضيف حاجة.»  
 لو ضغط: «طيب إمتى بتعمل خدمة؟» — «لما يبقى فيه منطق حقيقي مشترك بين أكثر من مكان: AuthService، PaymentService، TermSheetService. المعيار واضح مش مزاج.»

## مثال

«اشرحلي إزاي الدفع بيشتغل»  
 الإجابة: «العلاقة ← طلب تمويل ← معاملة ← حدث المزود. المؤسس بيطلب رقم، المستثمر بيفتح checkout، والتسوية مبتحصلش لما المتصفح يرجع — بتحصل لما السيرفر يسأل المزود أو يستقبل webhook موقع.»  
 لو ضغط: «وايه اللي بيمنع الدفع المزدوج؟» — «فهرس فريد على (المزود، معرف الحدث). الحدث بيتسجل قبل أي أثر، وأي تكرار بيصطدم بالفهرس قبل ما أي رقم يتحرك.»  
 لو ضغط أكثر: «وليه ده جوّه معاملة؟» — «لأن حجز مفتاح عدم التكرار كتابة دائمة. لو العملية ماتت بعد الحجز وقبل الأثر، المفتاح هيعيش والتسوية لأ، وإعادة المحاولة هي بالظبط اللي المفتاح بيمنعها — الدفعة كانت هتضيع.»

## مثال

«إيه أصعب قرار تقني خدته؟»  
 الإجابة: «التمويل على دفعات. الكود كان ببسأل هل نجحت دفعة واحدة؟ ولو أيوه يقفل الطلب — يعني التزام 50 ألف بيتقفل على أول قسط 10 آلاف، والباقي يبقى مستحيل تحصيله والرصيد لسه محسوب كالتزام.»  
 لو ضغط: «إزاي حلّيتها؟» — «حوّلت السؤال من Any() لـ Sum(): هل مجموع المدفوع وصل للالتزام؟ وضفت طرح المدفوع من مساحة الجولة، عشان المؤسس ما يقدرش يقسم التزام ويطلبه مرتين.»

## مثال

«ليه الفلوس مش حقيقية؟»  
 الإجابة: «المشروع مالوش رخصة مالية يحرك فلوس ناس. والحماية معمارية: التطبيق بيرمي استثناء ومش بيشتغل أصلًا لو المفتاح مش sk\_test\_»  
 لو ضغط: «طيب إزاي بتثبت إن النظام هيشغل بفلوس حقيقية؟» — «المزود الوهمي بيمشي على نفس آلة الحالة ونفس الرسوم ونفس عدم التكرار ونفس سجلّ المراجعة. الفرق الوحيد إنه مش بيكلّم الإنترنت.»

## مثال

«إزاي ضمنت جودة الكود في فريق سنّة؟»  
 الإجابة: «كتبت AGENTS.md — اتناشر قاعدة ملزمة، وكل واحدة مكتوب معاها ليه. القواعد اللي في دماغ شخص واحد بتتكسر أول ما حد ثاني يكتب سطر.»  
 لو ضغط: «وإزاي تتأكد إن حد التزم؟» — «القواعد الحرجة مفروضة بالكود مش بالثقة: FundingMath مصدر واحد، StageLog الطريقة الوحيدة لتغيير المرحلة، والفهرس الفريد هو اللي بيمنع الدفع المزدوج.»

## مثال

«ليه مغيش شارة موثّق؟»  
 الإجابة: «لأن فيستورا مبتتحققش من هوية حد ولا بتدقّق شركة. الشارة كانت هتبقى كذب بصري. وبدالها TrustSignals بيبنّي المصادقية من حقايق قابلة للإثبات بس — الإيميل اتأكد، عمر الحساب، الأدمن راجع، فيه تركيزات.»

## مثال

«إليه الفرق بين المشروع ده وأي منصة تمويل ثانية؟»  
 الإجابة: «ثلاث حاجات. الأولى: مفردات مالية صارمة — المنتج بيرفض يقول جُمع على فلوس ما وصلتش. الثانية: التمويل على دفعات — الالتزام ينفع يتدفع على مراحل زي الواقع. الثالثة: المنتج مبيدّعيش حاجة ماينفعش يثبتها — مفيششارة موثّق ولا درجة ثقة.»

لو ضغط: «طيب دي مش مميزات تقنية» — «هي قرارات تقنية بنتيجتها منتج: FundingMath بيمنع الحساب الغلط بالكود، والدفعات محتاجة تغيير في السكيميا وفي أربع دوال، و TrustSignals بيبيّن المصدقية من حقايق قابلة للإثبات بس.»

## مثال

«إزاي ضمنت إن الفلوس ماتتحسبش غلط؟»  
 الإجابة: «ثلاث طبقات. الأولى: FundingMath هو المصدر الوحيد — مفيش حساب مالي في أي مكان ثاني. الثانية: الفهرس الفريد على أحداث الدفع بيمنع تطبيق نفس التأكيد مرتين. الثالثة: طرح المدفوع من مساحة الجولة بيمنع تحصيل نفس الالتزام مرتين بتقسيمه.»

لو ضغط: «وايه اللي بيثبت ده؟» — «سويت اختبارات التكامل بتاعة عضو التستينج: فيه اختبار بيدفع طلب وبعدين يحاول يفتح checkout ثاني عليه، والنظام بيرفض.»

## مثال

«لو عندك وقت أكثر، كنت هتعمل إيه؟»  
 الإجابة: «ثلاثة بالترتيب: أتمتة اختبار للفرون (Playwright)، ونقل الصور لتخزين خارجي بدل قاعدة البيانات، وأداة تفرض تطابق قواعد التحقق بين الفرون والباكند بدل الانضباط اليدوي.»  
 لو ضغط: «وليه دول بالذات؟» — «لأنهم الثلاثة اللي موثّقين عندي كحدود معروفة، وكل واحد فيهم بيسدّ فجوة بتكبر مع حجم المشروع مش بتفضل ثابتة.»

## مثال

«إزاي اتعاملت مع خلاف تقني في الفريق؟»  
 الإجابة: «القواعد الحرجة كتبها في AGENTS.md بالسبب بتاعها، مش كأوامر. لما يبقى فيه خلاف، النقاش يبقى على السبب المكتوب مش على الرأي — وده بيخلّي القرار قابل للمراجعة بدل ما يبقى سلطة.»  
 لو ضغط: «وطلع حد اعترض على حاجة؟» — «وجّه لأي قاعدة وقول إن اللي فرضها هو الكود مش الاتفاق: FundingMath مصدر واحد، StageLog طريقة وحيدة، والفهرس الفريد.»

## مثال

«إليه العيوب في المشروع؟»  
 الإجابة: «الدفع صندوقي بالكامل — قرار مقصود لأننا مالناش رخصة. الصور في قاعدة البيانات مش تخزين خارجي. التحقق مكرّر بين الفرون والباكند بالانضباط مش بأداة. والفرون لسه بلا أتمتة اختبار.»  
 لو ضغط: «وليه ما عملتوش الأفضل؟» — «كل واحدة كانت هتكلف وقت وتعقيد أكثر مما الحجم يستاهل. دي قرارات نطاق واعية موثّقة، مش حاجات فاتت.»

## 36. Answering For Your Team

إنت الوحيد اللي ممكن يتسأل عن شغل أي عضو. الفصل ده مش عشان تحفظ شغلهم — عشان تعرف تجاوب جملتين وتوجّه بثقة.

### 36.1. The Backend Member

| لو اتسأل عن    | قول                                                                                      |
|----------------|------------------------------------------------------------------------------------------|
| نصيبه          | المصادقة والحسابات كاملة، المشاريع والاكتشاف، القصة والتفاعل — تسع كترولرز وأربع خدمات.  |
| أقوى حاجة عنده | كشف إعادة استخدام الـ refresh token: توكن مُلغى بيتستعمل = سرقة، فبنلغي كل جلسات الحساب. |
| أذكى تفصيلة    | رسالة الدخول الغلط <b>موحّدة</b> بين إيميل مش موجود وكلمة سر غلط — منع تسريب المستخدمين. |

**جدول 34** عضو الباكند في ثلاث جمل.

### 36.2. The Frontend Member

| لو اتسأل عن    | قول                                                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------|
| نصيبه          | صفحة التصقّح (أعقد صفحة في المنتج)، النماذج الخمسة، صفحة المشروع، الإعدادات، الدليل، البحث المحفوظ.               |
| أقوى حاجة عنده | حالة الفلاتر كلها في الـ URL — الرابط يتشارك، والتحديث مبيضيّعش الفلاتر، وزارار الرجوع بيشغل.                     |
| أذكى تفصيلة    | العدّادات جنب كل فلتر بتتسبب مع كل الفلاتر <b>ما عدا فلترها هي</b> — عشان الرقم يجاوب «هيطلعلني إيه لو اخترت ده». |

**جدول 35** عضو الفرونت في ثلاث جمل.

### 36.3. The Database Member

| لو اتسأل عن    | قول                                                                                                              |
|----------------|------------------------------------------------------------------------------------------------------------------|
| نصيبه          | المخطط والعلاقات والفهارس والحذف الناعم وتسعة وعشرين ترحيل.                                                      |
| أقوى حاجة عنده | الفهارس اللي <b>بتفرض قواعد</b> مش بس بتسرّع — زي الفهرس الفريد المفلتر اللي بيمنع علاقتين حيّتين لنفس المستثمر. |
| أذكى تفصيلة    | الحذف الناعم بفلتر استعلام عام: الصف بيتخفي تلقائيًا من كل استعلام من غير ما أي كترولر يفتكر.                    |

**جدول 36** عضو الداتابيز في ثلاث جمل.

### 36.4. The UI/UX Member

| لو اتسأل عن    | قول                                                                                                         |
|----------------|-------------------------------------------------------------------------------------------------------------|
| نصيبه          | تصميم كل الشاشات، نظام التصميم، الخطوط والألوان، التصميم ثنائي اللغة، والتسليم للفرونت. <b>مبيكتبش كود.</b> |
| أقوى حاجة عنده | اكتشف إن الذهب على الورق البيج بيقيس 2.86:1 والمعيار 4.5:1 — فعمل توكن ذهبي أعمق للنصوص.                    |

| لو اتسأل عن                    | قول                                                                                                                           |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| أذكى تفصيلة                    | الزوايا <b>مقصوفة</b> مش مدوّرة — قصّ زاويتين متقابلتين ببيان «لوحة»، وده اللي بيخلّي أي شاشة تتعرف إنها فيستورا من غير شعار. |
| جدول 37 عضو UI/UX في ثلاث جمل. |                                                                                                                               |

## 36.5. The Testing Member

| لو اتسأل عن                       | قول                                                                                          |
|-----------------------------------|----------------------------------------------------------------------------------------------|
| نصيبه                             | تمنية وعشرين اختبار تكامل، توثيق تلتاشر محاولة اختراق، وخطة اختبار يدوي بخمسة وستين سيناريو. |
| أقوى حاجة عنده                    | الاختبارات بتنده الـ API <b>الحقيقي</b> على قاعدة بيانات معزولة — مش دوال منفصلة.            |
| أذكى تفصيلة                       | اختبارات الأمان بتتأكد من <b>حالة الداتايز</b> بعد المحاولة، مش بس من الـ status code.       |
| جدول 38 عضو التستينج في ثلاث جمل. |                                                                                              |

### اعرف ده كويس

#### الجملة اللي تقولها لو اتسألت في تفصيلة عميقة في شغل حد:

«ده بتاع [فلان] وهو اللي هيشرحه بالتفصيل — أنا عارف القرار المعماري وراه وليه اتاخذ كده، بس التنفيذ بتاعه.»  
دي إجابة **قائد**، مش تهزّب. القائد المفروض يعرف الصورة الكبيرة وحدود كل جزء — مش كل سطر كتبه كل واحد.

## 37. Suggested Study Plan

| الجلسة  | تقرا                                         | تعمل على الجهاز                                                                    |
|---------|----------------------------------------------|------------------------------------------------------------------------------------|
| الأولى  | الفصل 1 و 2 و 3 — دورك، المنتج، المعمارية    | شغل المشروع كامل — الـ API والفرونت — وافتح كل شاشة أدمن.                          |
| الثانية | الفصل 4 و 5 — الفريق والمخطط                 | افتح الـ ERD الثلاثة واتبع علاقة واحدة من أولها لآخرها.                            |
| الثالثة | الفصل 6 — الدفع                              | اقرا <code>ApplyProviderResultAsync</code> سطر سطر — ده أهم كود عندك.              |
| الرابعة | الفصل 7 — الدفعات (Tranches)                 | اتبع <code>IsFullySettled</code> ومين بينده عليها، وجرب دفعة جزئية.                |
| الخامسة | الفصل 8 و 9 — غرفة الصفقة والأدمن            | افتح غرفة صفقة حقيقية وجرب اقتراح شروط وقبولها.                                    |
| السادسة | الفصل 10 و 11 و 12 — البنية التحتية والفرونت | افتح <code>Program.cs</code> و <code>client.ts</code> و <code>globals.css</code> . |
| السابعة | الفصل 13 و 14 — القواعد والحدود              | اقرا <code>AGENTS.md</code> كامل، وجّهز إجابة «إيه العيوب».                        |
| الأخيرة | الفصل 15 — أسئلة الضغط                       | جاوب على كل مثال بصوت عالي، وبعدها الـ Cheat Sheet.                                |

جدول 39 تمن جلسات — ملفك أكبر من ملفات الأعضاء.

## اعرف ده كويس

**نصيحة خاصة بـك كقائد:** إنت الوحيد اللي ممكن يتسأل عن شغل أي عضو. فبعد ما تخلص ملفك، خذ نظرة سريعة على Cheat Sheet بتاع كل عضو — مش عشان تحفظ شغلهم، بس عشان تعرف تقول «ده بتاع فلان وهو اللي هيشرحه» بثقة.

## 38. Final Code Map — Where to Find Everything

| What it does                                                    | Where in Code                                                                                                                                                     | What                    |
|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| كل ما يتعلق بالفلوس — 1259 سطر.                                 | <code>Services/<br/>Payments/PaymentService.cs</code>                                                                                                             | Payment engine          |
| الحدث بيتسجل قبل الأثر، جوّه معاملة واحدة.                      | <code>PaymentService.ApplyProviderResultAsync</code>                                                                                                              | Event before effect     |
| التسوية، تجميد الرسوم، وتحريك المرحلة.                          | <code>PaymentService.SettleAsync</code>                                                                                                                           | Settlement              |
| المصدر الوحيد لأي رقم مالي — 443 سطر.                           | <code>Services/FundingMath.cs</code>                                                                                                                              | All money figures       |
| أربع دوال بتخلي الالتزام يتدفع على مراحل.                       | <code>FundingMath.IsFullySettled</code><br>· <code>UnsettledCommitment</code> ·<br>· <code>CommitmentTargetAsync</code><br><code>SettledForInvestmentAsync</code> | Tranche logic           |
| اقتراح وقبول واستبدال الشروط.                                   | <code>Services/TermSheetService.cs</code>                                                                                                                         | Term sheets             |
| أكبر كمنترولر — الأسئلة والمستندات والتفاوض.                    | <code>Controllers/<br/>DealRoomController.cs</code>                                                                                                               | Deal room               |
| طلب التمويل، checkout، التحقق، webhook.                         | <code>Controllers/<br/>PaymentsController.cs</code>                                                                                                               | Payment endpoints       |
| المستخدمين، الأدمن الأساسي، bootstrap.                          | <code>Controllers/<br/>AdminController.cs</code>                                                                                                                  | Admin — users and roles |
| · الإيرادات، نظرة الحساب، المراجعة،<br>· التحليلات، البحث.<br>· | <code>AdminRevenue</code><br><code>AdminUserOverview</code><br><code>AdminModeration</code><br><code>AdminSearch</code> · <code>AdminInsights</code>              | Admin — the other five  |
| الطريقة الوحيدة لتغيير مرحلة العلاقة.                           | <code>Services/StageLog.cs</code>                                                                                                                                 | Stage transitions       |
| مصادقية من حقائق قابلة للإثبات بس.                              | <code>Services/TrustSignals.cs</code>                                                                                                                             | Trust signals           |
| الـ CORS، الـ rate limiting، وحارس مفتاح Stripe.                | <code>Program.cs</code>                                                                                                                                           | Infrastructure          |
| الناقل الوحيد — التوكن والتجديد والأخطاء.                       | <code>lib/api/client.ts</code>                                                                                                                                    | API transport           |
| عقد الأنواع بين الفرونت والباكند — 1736 سطر.                    | <code>lib/types/api.ts</code>                                                                                                                                     | Type contract           |
| 129 توكن — الألوان والخطوط والحركة.                             | <code>app/globals.css</code>                                                                                                                                      | Design system           |

| What it does                             | Where in Code                                  | What       |
|------------------------------------------|------------------------------------------------|------------|
| ثلاثي أبعاد بالـcompositor من غير WebGL. | <code>components/motion/depth-scene.tsx</code> | 3D scene   |
| اتناشر مكّون.                            | <code>components/landing/</code>               | Landing    |
| تلتاشر صفحة.                             | <code>app/(app)/admin/</code>                  | Admin UI   |
| اتناشر قاعدة ملزمة للفريق كله.           | <code>AGENTS.md</code>                         | Governance |

**جدول 40** كل حاجة في نصيكم، في جدول واحد.

## 39. Cheat Sheet — Review Before You Walk In

### خلاصة

**دورك:** قائد الفريق — صمّمت المعمارية، وزّعت الشغل، وبنيت أضخم نصيب تقني. أكثر من 9000 سطر باكند (تلتين الباكند) + طبقة الفرونت المشتركة.

**المنتج:** منصّة تمويل من ناحيتين. المؤسّس ينشر، المستثمر يطلب، المؤسّس يوافق، يتفاوضوا في غرفة صفقة، ويدفع في صندوق تجريبي.

**المفردات (الأهم):** `Interest ≥ Committed ≥ Funded`. الموافقة **التزام** مش فلوس. فلوس = `Succeeded` بس.

**ثلاث قرارات معمارية:** مغيّش Repository (الـ DbContext نفسه هو) `Status` و `Stage` محورين منفصلين · كل رقم مالي في `FundingMath` بس.

**Investment** **كيان مش جدول ربط:** لأن فيه حالة ومرحلة وملاحظات وتاريخ.

**الدفع — قاعدتين:** التأكيد مرة واحدة بفهرس فريد مش بكود · المتصفّح الراجع **مش دليل**، السيرفر ببسأل المزود. **الحدث قبل الأثر:** جوّه معاملة واحدة — لأن حجز المفتاح كتابة دائمة، ولو العملية ماتت بعده الدفعة كانت هتضيع للأبد.

**الدفعات (Tranches):** السؤال اتغيّر من «هل نجحت دفعة؟» لـ«هل مجموع المدفوع وصل للالتزام؟» — وطرح المدفوع من المساحة بيمنع التحصيل مرتين.

**الأدمن:** ستّة كينترولرز (2093 سطر) · الأدمن الأساسي بس يعمل أدمن · الـ bootstrap بيتقفّل للأبد بعد أول أدمن. **Stripe:** التطبيق **بيرفض يشتغل** على مفتاح مش `sk_test_` — حماية معمارية.

**الفرونت:** الناقل الوحيد `client.ts` (بالـ single-flight refresh) · 129 توكن تصميم · ثلاثي أبعاد بالـcompositor مش 13 WebGL · صفحة أدمن.

**الحوكمة:** `AGENTS.md` — اتناشر قاعدة، كل واحدة معاها **ليه**.

**مغيّش شارة موثّق:** لأن المنصّة مبيتتحقّقش من حد — `TrustSignals` بيبيني من حقايق قابلة للإثبات بس.

**العيوب:** الدفع صندوقي (مقصود) · الصور في القاعدة · التحقق مكّرر بالانضباط · مغيّش أتمتة فرونت. **كلها قرارات نطاق واعية.**

**لو اتسألت عن شغل عضو:** اعرف مجاله وقول «ده بناع فلان» بثقة — إنت عارف التقسيمة، مش لازم تعرف كل سطر.